Chapter 2
Application Layer



Chapter 2: Outline

- Principles of network applications
  - app architectures
  - app requirements
- Web and HTTP
- Electronic mail
  - SMTP, POP3, IMAP
- The Domain Name System (DNS)
- P2P applications
- video streaming and content distribution networks
- socket programming with UDP and TCP

Chapter 2: Application Layer

Our goals:

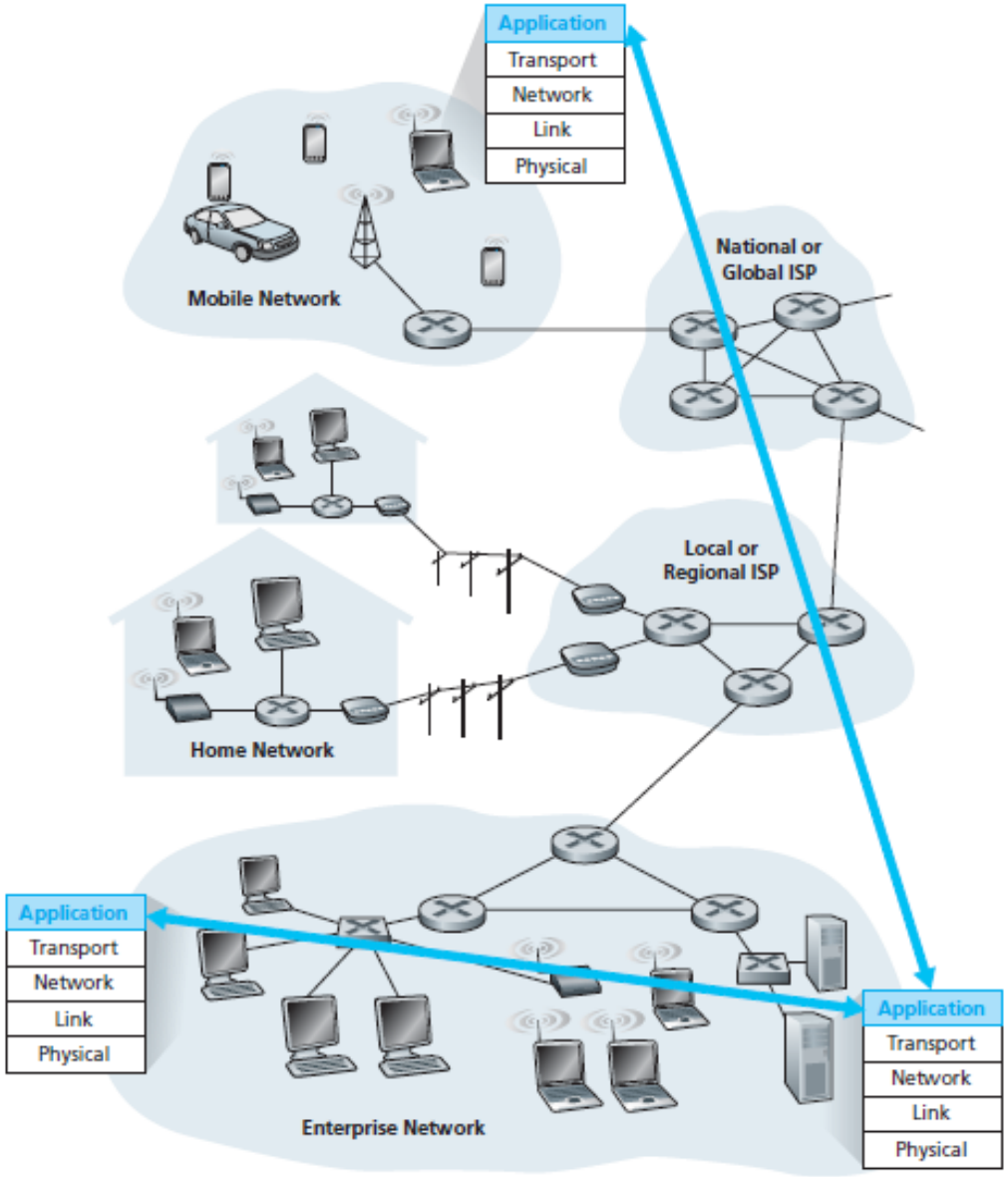
- Conceptual, implementation aspects of network application protocols
  - transport-layer service models
  - client-server paradigm
  - peer-to-peer paradigm
- Learn about protocols by examining popular application-level protocols
  - HTTP
  - FTP
  - SMTP / POP3 / IMAP
  - DNS
  - video streaming systems, CDNs

Some network apps

- E-mail
- Web
- Text messaging
- Remote login
- P2P file sharing
- Multi-user network games
- Streaming stored video (YouTube, Hulu, Netflix)
- Voice over IP (e.g., Skype)
- Real-time video conferencing
- Social networking
- Search
- ***this list is NOT exhaustive*

Principles of Network Applications

- At the core of network application development ➔ writing programs that run on end systems and communicate over the network
  - These end systems are sometimes DIFFERENT
    - e.g., web server software communicates with browser software
- No need to write software for network-core devices
  - Network-core devices do not run user applications
  - Applications on end systems allows for rapid application development and propagation



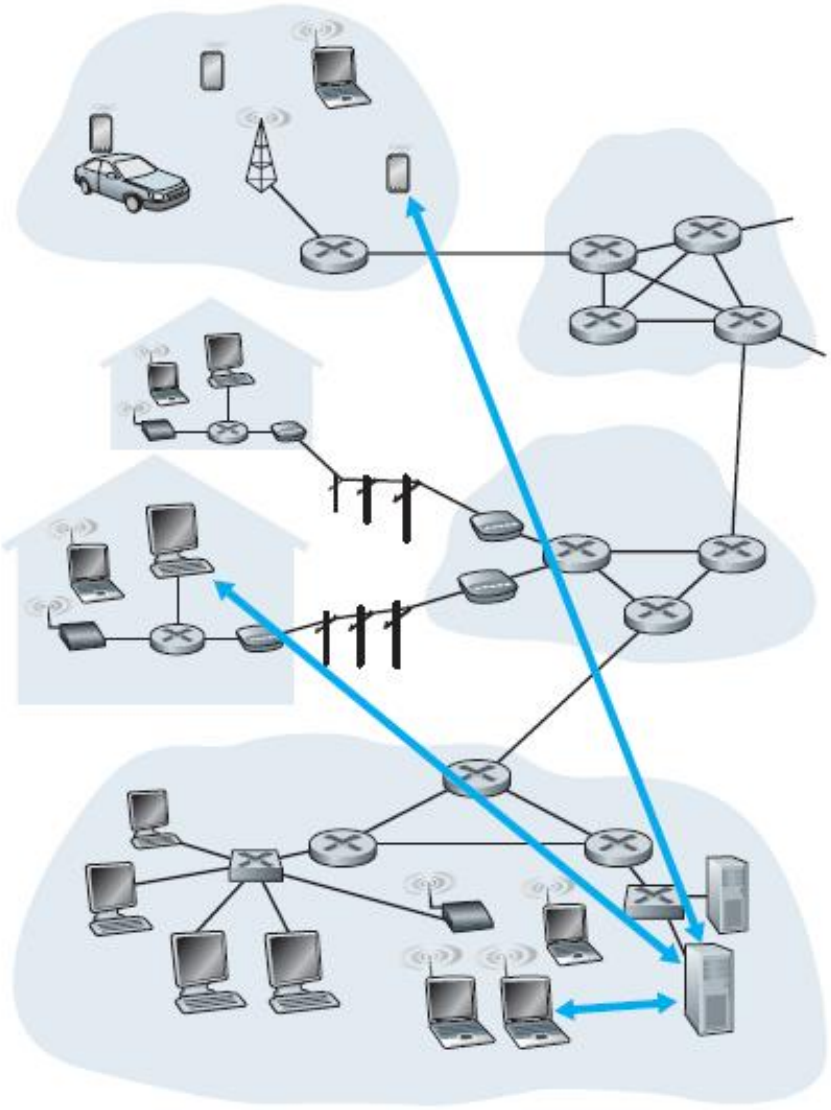
Network apps communicate between end systems at the application layer over the network

Network Application Architectures

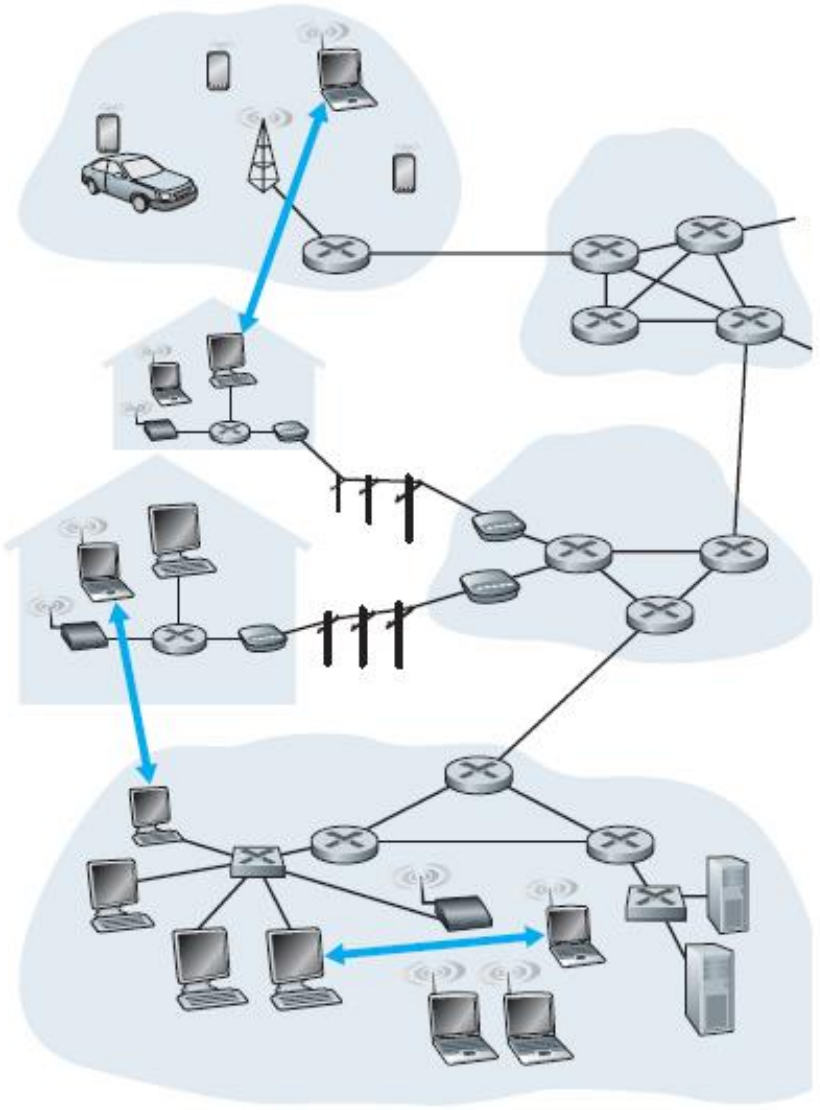


- Application's architecture is different from the network architecture
  - Network architecture ➔ the 5 layers
- Application architecture
  - designed by the application developer
  - Sets the structure of the application
- Possible structure of applications:
  - client-server
  - peer-to-peer (P2P)

Spot the difference.

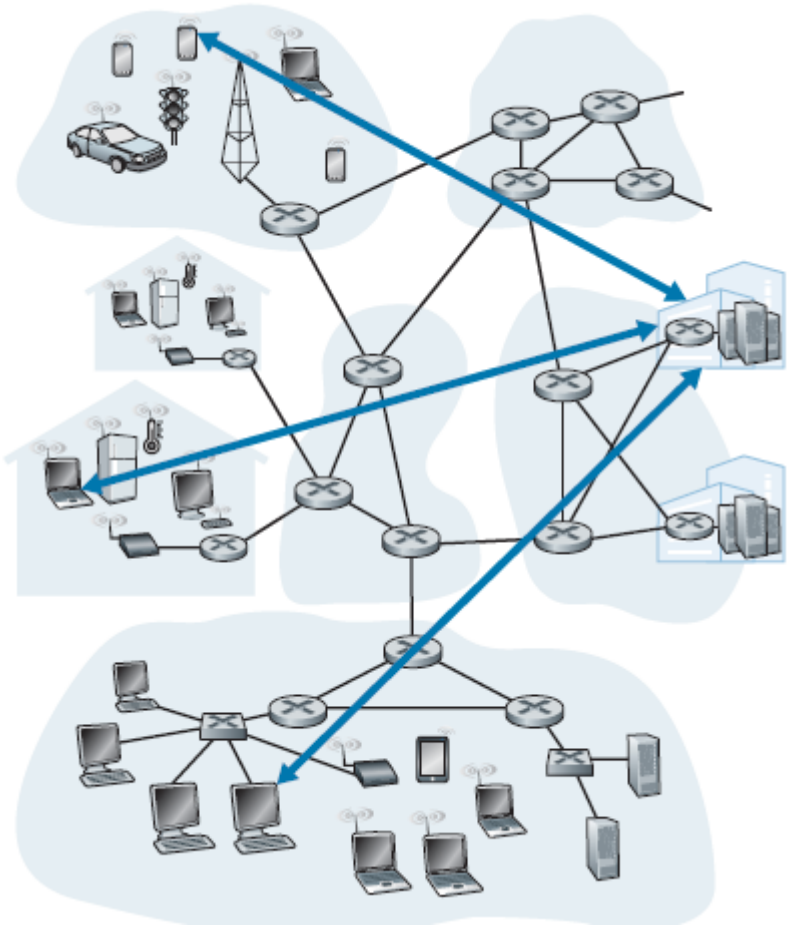


a. Client-server architecture



b. Peer-to-peer architecture

Client-server paradigm



Server:

- Always-on host
- Permanent IP address
- often in Data centers, for scaling

Clients:

- Communicate with server
- May be intermittently connected
- May have dynamic IP addresses
- Do not communicate directly with each other

•examples: HTTP, IMAP, FTP

Client-server architecture

Search engine

Internet
e-commerce

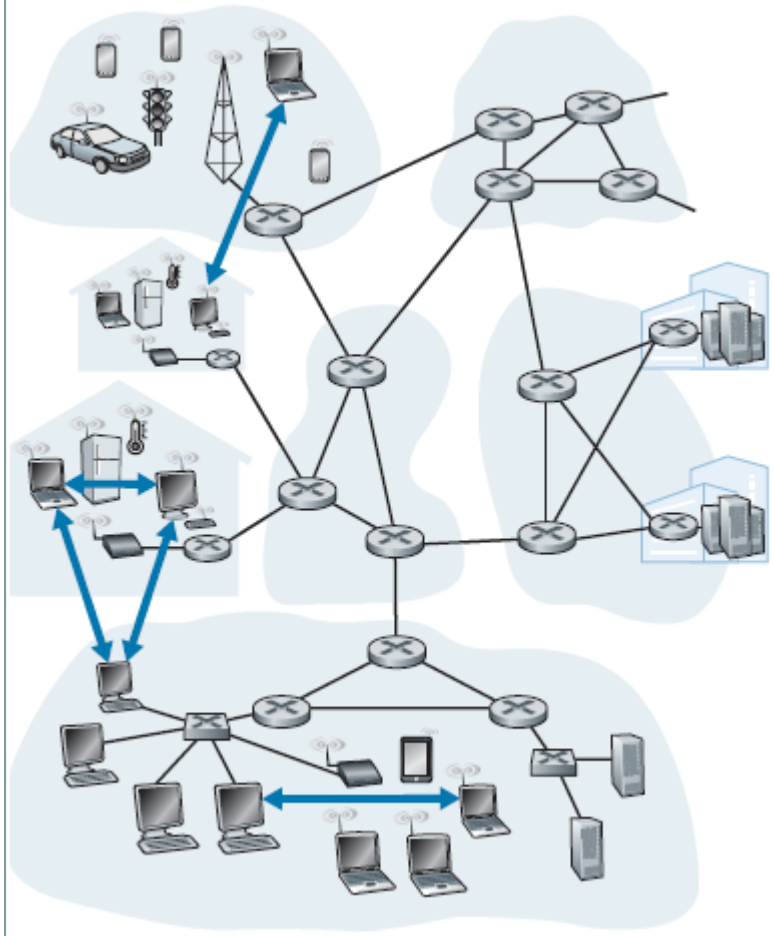
Social
networking



Data center:

- housing a large number of hosts
- create a powerful virtual server
- popular Internet services employ one or more data centers
- A data center can have hundreds of thousands of servers

Peer-peer architecture



- *No* always-on server
- Arbitrary end systems directly communicate
- Peers request service from other peers, provide service in return to other peers
  - *Self scalability* – new peers bring new service capacity, as well as new service demands
- Peers are intermittently connected and change IP addresses
  - complex management
- example: P2P file sharing

Application Architecture Analogy : The class

- Before: client-server
- Teacher always on
- The one to get info/data/resources from
- Students always connects to teachers

- Today: P2P
- Teacher is always on BUT….
- Students intermittently connect to get info/data/resources from each other
- Student-teacher connections lessens

Processes communicating

Process: program running within a host

- within same host, two processes communicate using inter-process communication (defined by OS)
- processes in different hosts communicate by exchanging messages

- In client-server:

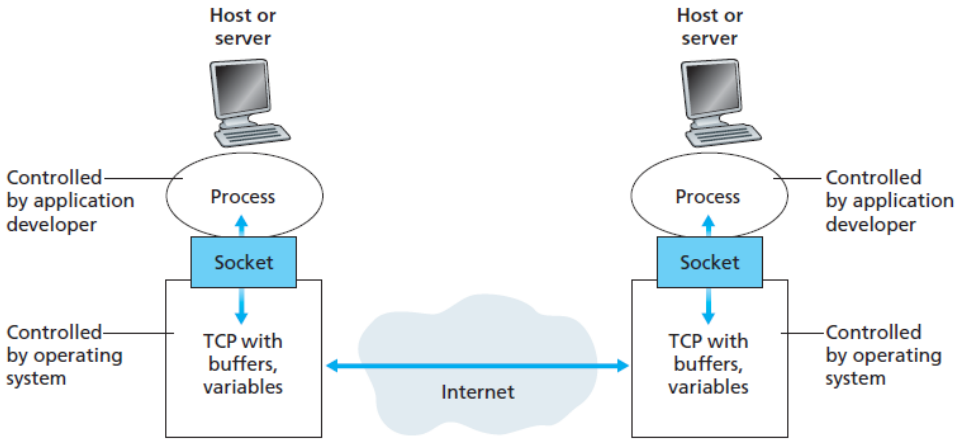
client process: process that initiates communication

server process: process that waits to be contacted

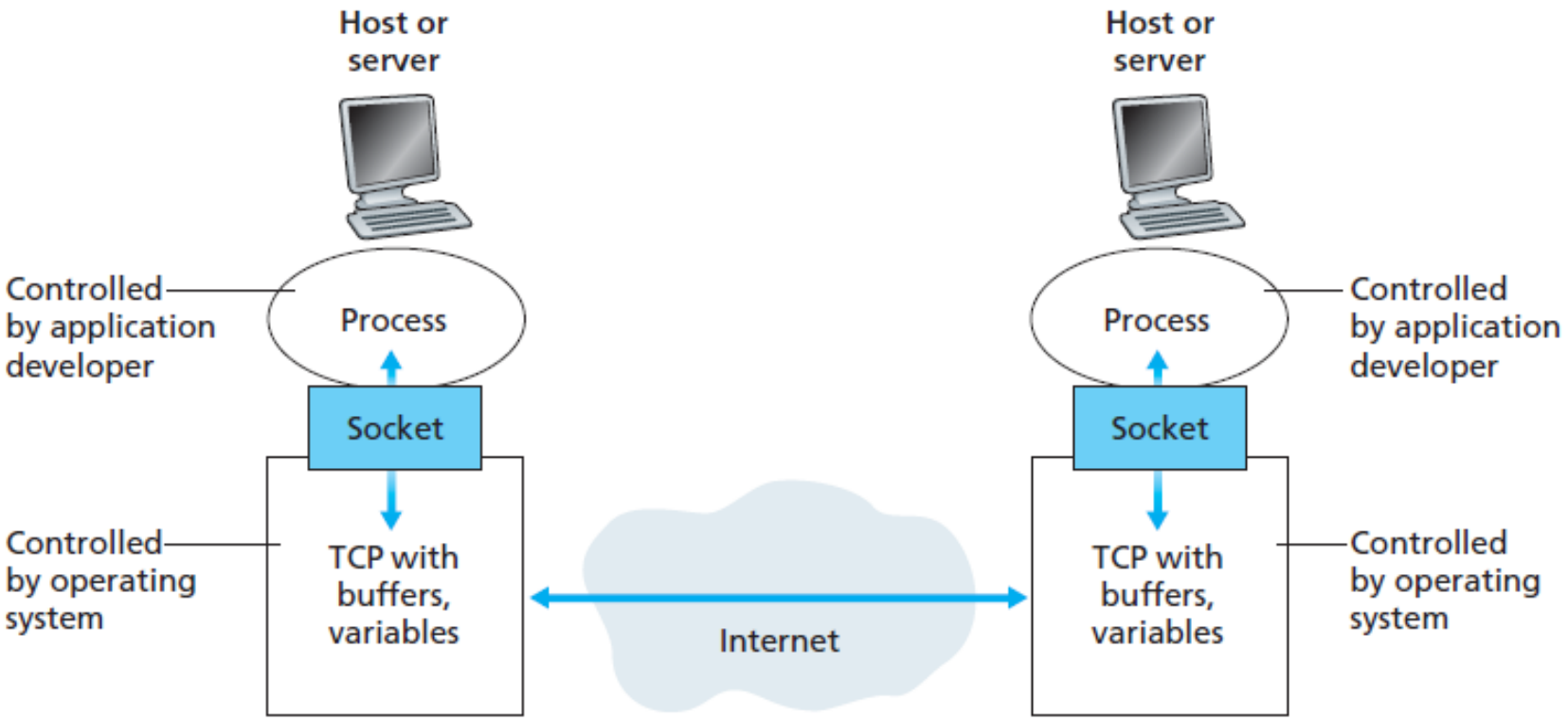
- Aside: applications with P2P architectures have client processes & server processes

Sockets

- Process sends/receives messages to/from its socket
- Socket analogous to door
  - Sending process shoves (push) message out door
  - Sending process relies on transport infrastructure on other side of door to deliver message to socket at receiving process
  - two sockets involved: one on each side



Sockets



A socket is the interface between the application process and the transport-layer protocol

Addressing processes

- To receive messages, process must have *identifier*
- Host device has unique 32-bit IP address
- *Q:* Does IP address of host on which process runs suffice (be adequate) for identifying the process?
- *A:* No, *many* processes can be running on same host

- *Identifier* includes both IP address and port numbers associated with process on host.
- Example port numbers:
  - HTTP server: 80
  - mail server: 25
- To send HTTP message to www.utm.my web server:
  - IP address: 161.139.20.177
  - port number: 80

App-layer protocol defines

- types of messages exchanged,
  - e.g., request, response
- message syntax:
  - what fields in messages & how fields are delineated (explained)
- message semantics
  - meaning of information in fields
- rules for when and how processes send & respond to messages

open protocols:

- defined in RFCs (everyone has access to protocol definition)
- allows for interoperability
- e.g., HTTP, SMTP

proprietary protocols:

- e.g., Skype, Zoom

What transport service does an app need?

Reliable data transfer (data integrity)

- some apps (e.g., file transfer, web transactions) require 100% reliable data transfer
- other apps (e.g., audio) can tolerate some loss

Timing

- some apps (e.g., Internet telephony, interactive games) require low delay to be "effective"

Throughput

- some apps (e.g., multimedia) require minimum amount of throughput to be "effective"
- other apps ("elastic apps") make use of whatever throughput they get

Security

- encryption, data integrity

Requirements of selected network applications

Transport service requirements: common apps

Application	Data Loss	Throughput	Time-Sensitive
File transfer/download	No loss	Elastic	No
E-mail	No loss	Elastic	No
Web documents	No loss	Elastic (few kbps)	No
Internet telephony/ Video conferencing	Loss-tolerant	Audio: few kbps–1Mbps Video: 10 kbps–5 Mbps	Yes: 100s of msec
Streaming stored audio/video	Loss-tolerant	Same as above	Yes: few seconds
Interactive games	Loss-tolerant	Few kbps–10 kbps	Yes: 100s of msec
Instant messaging	No loss	Elastic	Yes and no

Internet transport protocols services

TCP service:

- *reliable transport*
  - between sending and receiving process
- *flow control:*
  - sender won't overwhelm receiver
- *congestion control:*
  - throttle (choke) sender when network overloaded
- *does not provide:*
  - timing, minimum throughput guarantee, security
- *connection-oriented:*
  - setup required between client and server processes

UDP service:

- *unreliable data transfer*
  - between sending and receiving process
- *does not provide:*
  - reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup,

Q: Why bother? Why is there a UDP?

Internet apps: application, transport protocols

Application	Application-Layer Protocol	Underlying Transport Protocol
Electronic mail	SMTP [RFC 5321]	TCP
Remote terminal access	Telnet [RFC 854]	TCP
Web	HTTP 1.1 [RFC 7230]	TCP
File transfer	FTP [RFC 959]	TCP
Streaming multimedia	HTTP (e.g., YouTube), DASH	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], or proprietary (e.g., Skype)	UDP or TCP
interactive games	WOW, FPS (proprietary)	UDP or TCP

Securing TCP

TCP & UDP

- no encryption
- cleartext passwords sent into socket traverse Internet in cleartext

Transport Layer Security (TLS)

- provides encrypted TCP connection
- data integrity
- end-point authentication

TLS is implemented at application layer

- apps use TLS libraries, that use TCP in turn
- cleartext sent into "socket" traverse Internet encrypted

Chapter 2: Outline

- Principles of network applications
  - app architectures
  - app requirements
- Web and HTTP
- Electronic mail
  - SMTP, POP3, IMAP
- DNS
- P2P applications
- CDN

Web and HTTP



Web and HTTP : The attraction

- The Web operates *on demand* ➔ Users receive what they want, when they want it.
- Everyone can be publishers of content ➔ a variety of content is now available
- Hyperlinks and search engines help us find all these published contents ➔ we learn, experience new things (though *not necessarily always good*)
- Contents are exhibited in beautiful graphics, with interaction via forms and scripts.
- The Web serves as a platform for many killer applications (e.g. YouTube, Gmail, Facebook) with many more to come.
  - It is also where ideas are born from the different contents, discussion forums, groups

Web and HTTP

- *A web page* consists of *objects each of which can be stored on different Web servers*
  - object can be HTML file, JPEG image, Java applet, audio file,…
  - An object is a collection of items that can be called from a single URL
- A web page consists of *base HTML-file* which includes *several referenced objects*
  - each object is addressable by a *URL (Uniform Resource Locator or Web address)* , e.g.,

`www.someschool.edu/someDept/pic.gif`

host name | path name

A Web page (also called a document) consists of objects

Web pages consist of a base HTML file and several referenced objects

An object is simply a file addressable by a single URL.

Examples of files: HTML file, a JPEG image, a Java applet, or a video clip

Each URL has two components: the hostname of the server that houses the object and the object's path name.

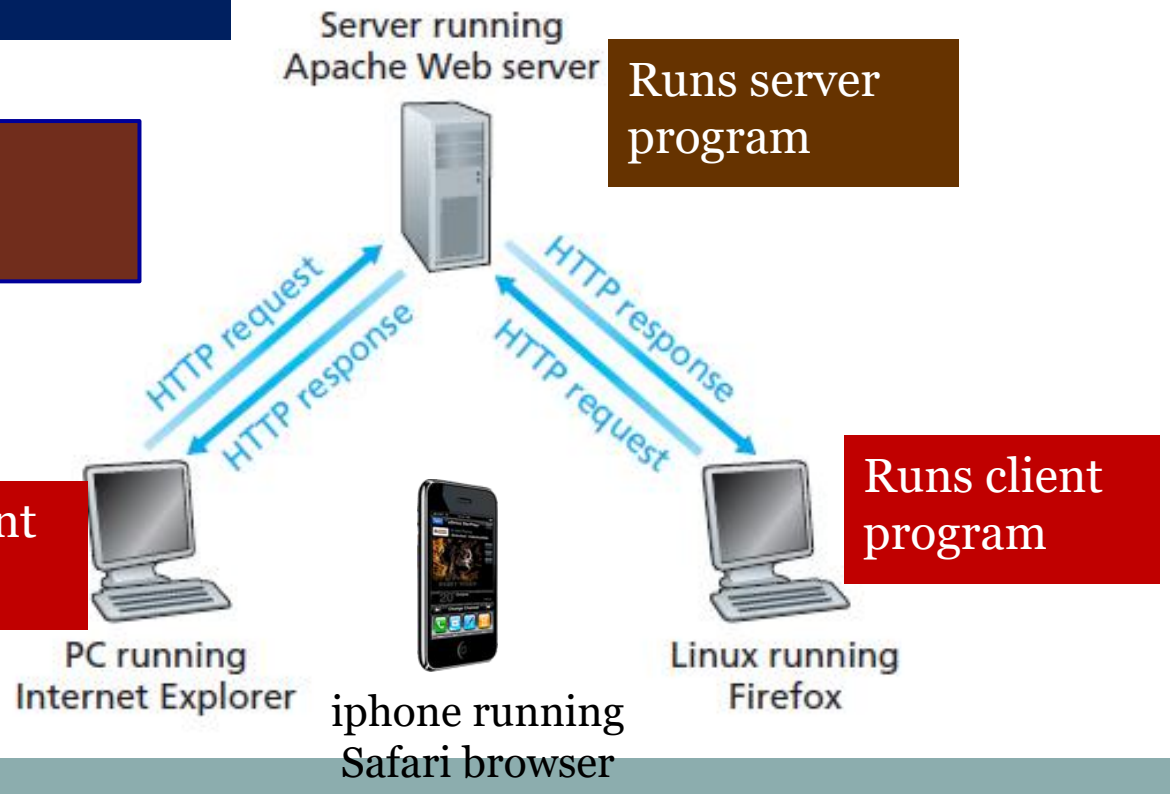
http://www.someSchool.edu/someDepartment/picture.gif

HTTP overview

HTTP is implemented in 2 programs:
- - a client program
- - a server program

browser that requests, receives, "displays" Web objects

Web server sends objects in response to requests



HTTP overview (continued)

- *HTTP uses TCP:*
  - The client sends HTTP request messages into its socket interface and receives HTTP response messages from its socket interface
    - The server does the same
- The communication:
  - Client initiates TCP connection (creates socket) to server, port 80
  - Server accepts TCP connection from client
  - HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
  - TCP connection closed

HTTP overview (continued)

- HTTP is a "*stateless*" protocol
  - the server sends requested files to clients without storing any state information about the client
  - server maintains no information about past client requests
  - When client ask for the same object repeatedly ➔ the server will resend

aside

protocols that maintain "state" are complex!

- past history (state) must be maintained
- if server/client crashes, their views of "state" may be inconsistent, must be reconciled

HTTP connections

During this client-server interactions over TCP, decisions need to be made

should **each** request/response pair be sent over a **separate TCP connection**

non-persistent connections

*should **all** of the* request/response pair be sent over the **same TCP connection**

persistent connections

The default mode

HTTP connections

non-persistent HTTP

- at most one object sent over TCP connection
  - Then connection is closed
- downloading multiple objects required multiple connections

persistent HTTP

- multiple objects can be sent over single TCP connection between client, server

suppose user enters URL:

`www.someSchool.edu/someDepartment/home.index`

(contains text, references to 10 jpeg images)

1a. HTTP client initiates TCP connection to HTTP server (process) at *www.someSchool.edu* on port 80

1b. HTTP server at host *www.someSchool.edu* waiting for TCP connection at port 80. "accepts" connection, notifying client

2. HTTP client sends HTTP *request message* (containing URL) into TCP connection socket. Message indicates that client wants object *someDepartment/home.index*

3. HTTP server receives request message, forms *response message* containing requested object, and sends message into its socket

5. The HTTP client receives the response message. The TCP connection terminates.

4. The HTTP server process tells TCP to close the TCP connection.

6. HTTP client parsing html file, finds 10 referenced jpeg objects ➔ repeat steps 1-5 for each object

Non-persistent HTTP: response time

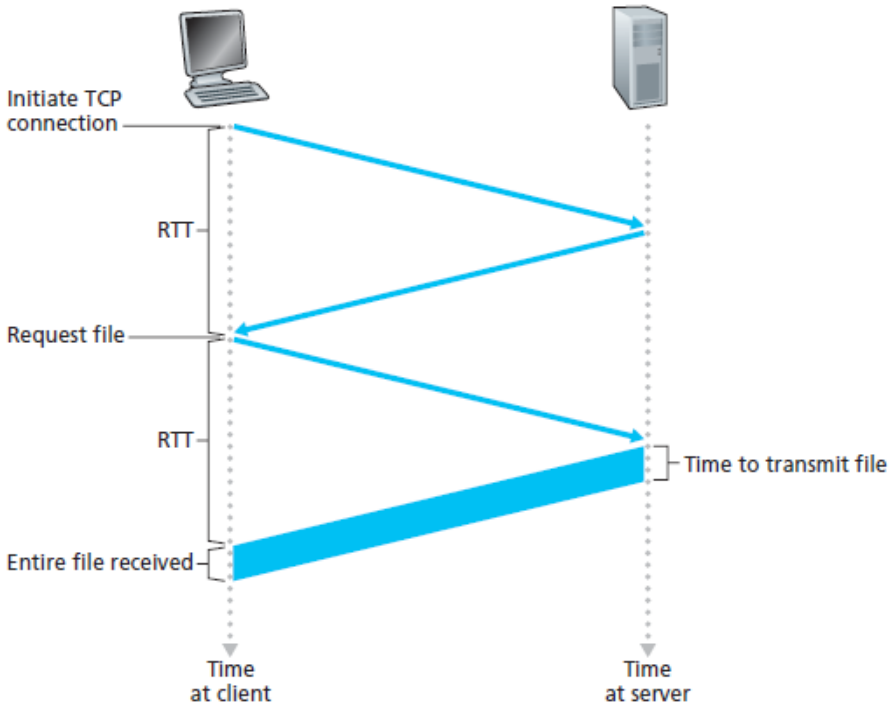
RTT (definition): time for a small packet to travel from client to server and back

HTTP response time (per object):

- one RTT to initiate TCP connection
- one RTT for HTTP request and first few bytes of HTTP response to return
- object/file transmission time

non-persistent HTTP response time =

2RTT+ file transmission time



Round-Trip Time (RTT)

Persistent HTTP

non-persistent HTTP issues:

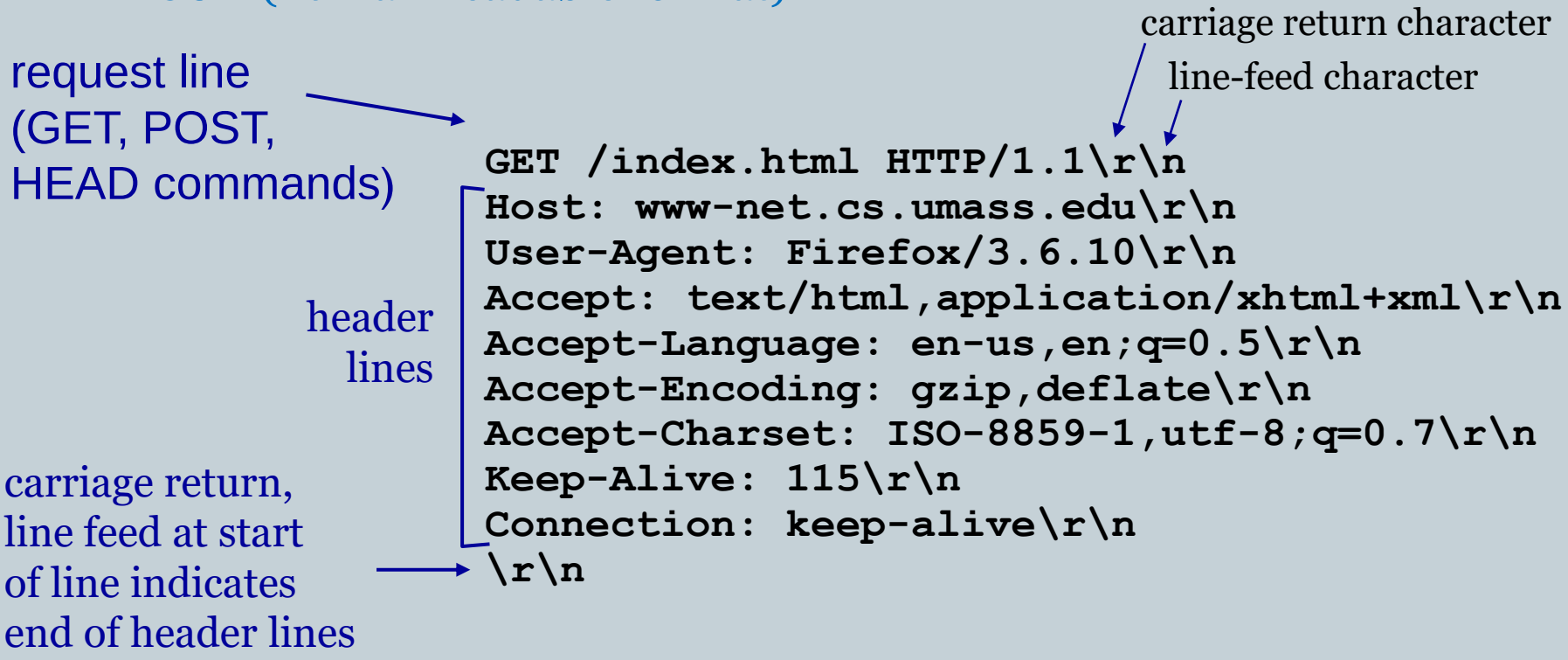
- Each object requires 2 RTT delay
- Operating System (OS) overhead for *each* TCP connection – can burden the server
  - For **each of these connections**, TCP buffers must be allocated and TCP variables must be kept in both the client and server.
- browsers often open multiple parallel TCP connections to fetch referenced objects (e.g. index.html contains text, references to 10 jpeg images)

persistent HTTP (HTTP1.1):

- server leaves connection open after sending response
- subsequent HTTP messages between same client/server sent over open connection
- client sends requests as soon as it encounters a referenced object
- as little as one RTT for all the referenced objects (cutting response time in half)

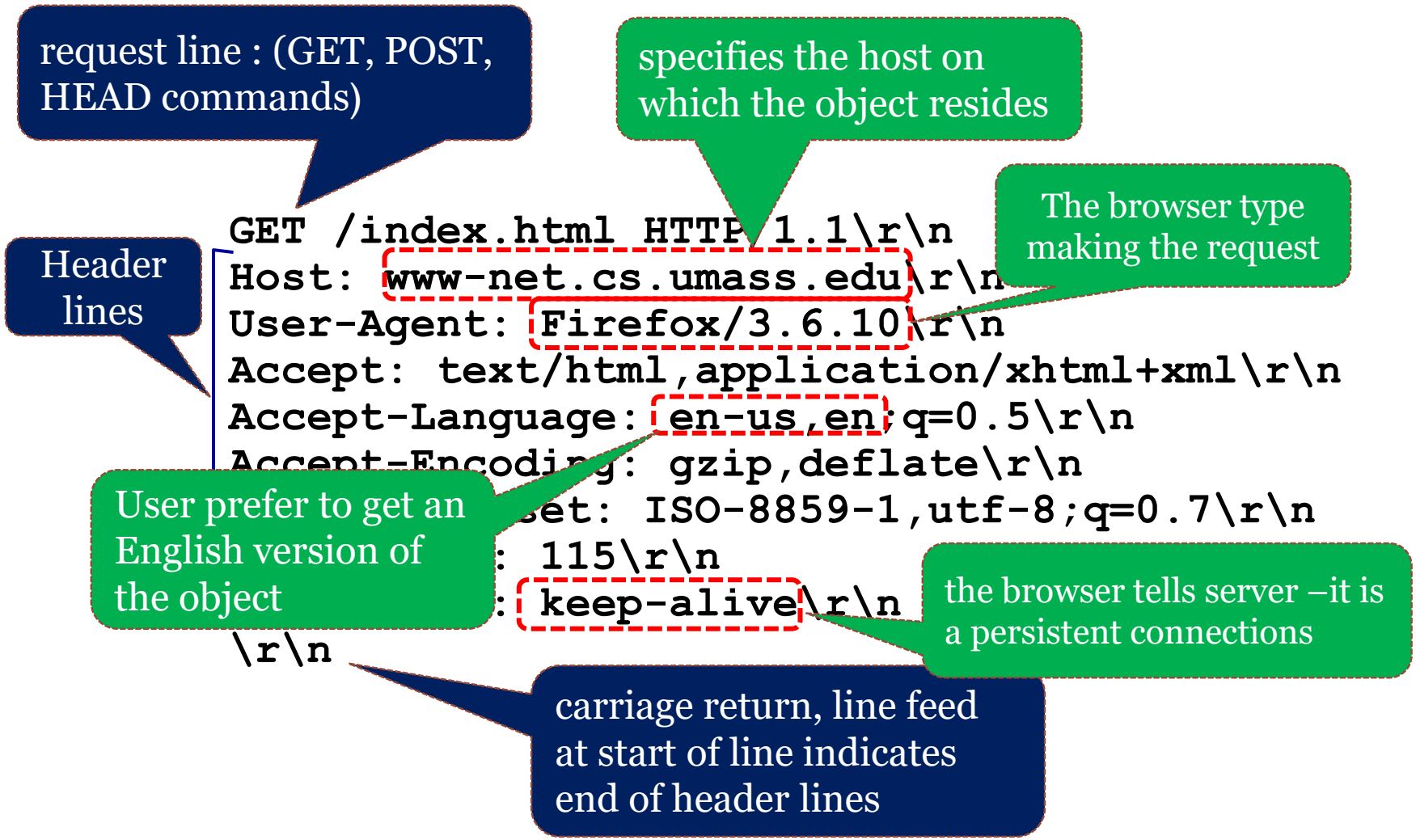
HTTP request message

- two types of HTTP messages: *request, response*
- HTTP request message:
  - ASCII (human-readable format)

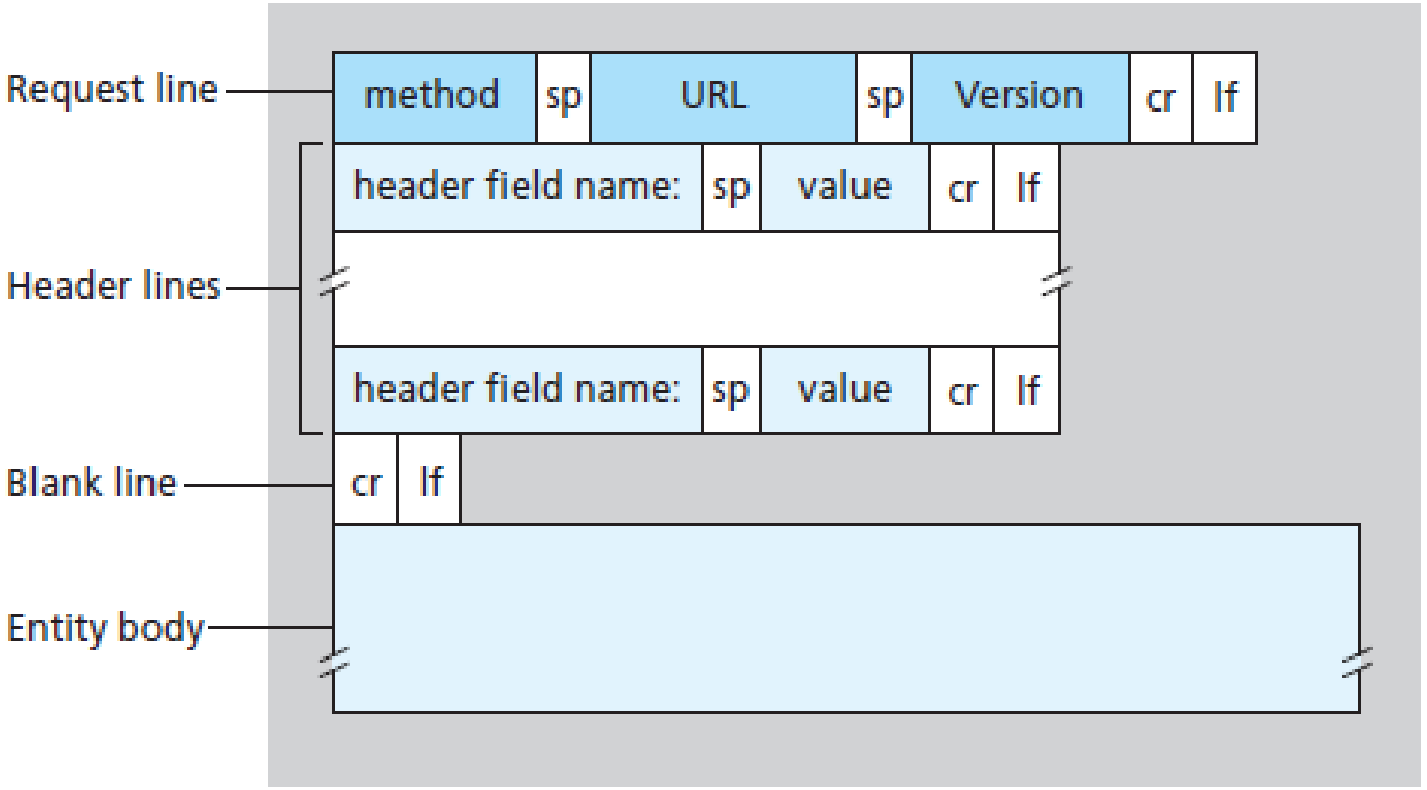


HTTP request message

The document requested: www-net.cs.umass.edu/index.html



HTTP request message: general format



Other HTTP request messages

POST method:

- web page often includes form input
- user input sent from client to server in entity body of HTTP POST request message

GET method:

- uses GET method
- include user data in URL field of HTTP GET request message (following a '?'):

`www.somesite.com/animalsearch?monkeys&banana`

Other HTTP request messages

HEAD method:

- requests headers (only) that would be returned if specified URL were requested with an HTTP GET method.

PUT method:

- uploads new file (object) to server
- completely replaces file that exists at specified URL with content in entity body of POST HTTP request message

Method types

HTTP/1.0:

- GET
  - GET is used to request data from a specified resource.
- POST
  - POST is used to send data to a server to create/update a resource.
- HEAD
  - requests headers (only) that would be returned if specified URL were requested with an HTTP GET method.
  - E.g. use for debugging

HTTP/1.1:

- GET, POST, HEAD
- PUT
  - uploads file in entity body to path specified in URL field
  - uploads new file (object) to server
  - completely replaces file that exists at specified URL with content in entity body of POST HTTP request message
- DELETE
  - deletes file specified in the URL field

HTTP response message

Status line: protocol ,status code status phrase

the message was generated by an Apache Web server

```
HTTP/1.1 200 OK\r\n
Date: Sun, 26 Sep 2010 20:09:20 GMT\r\n
Server: Apache/2.0.52 (CentOS)\r\n
Last-Modified: Tue, 30 Oct 2007 17:00:02 GMT\r\n
ETag: "17dc6-a5c-bf716880"\r\n
Accept-Ranges: bytes\r\n
Content-Length: 2652\r\n
Keep-Alive: timeout=10, max=100
Connection: Keep-Alive\r\n
Content-Type: text/html; charset=ISO-8859-1\r\n
\r\n
data data data ... data ...
```

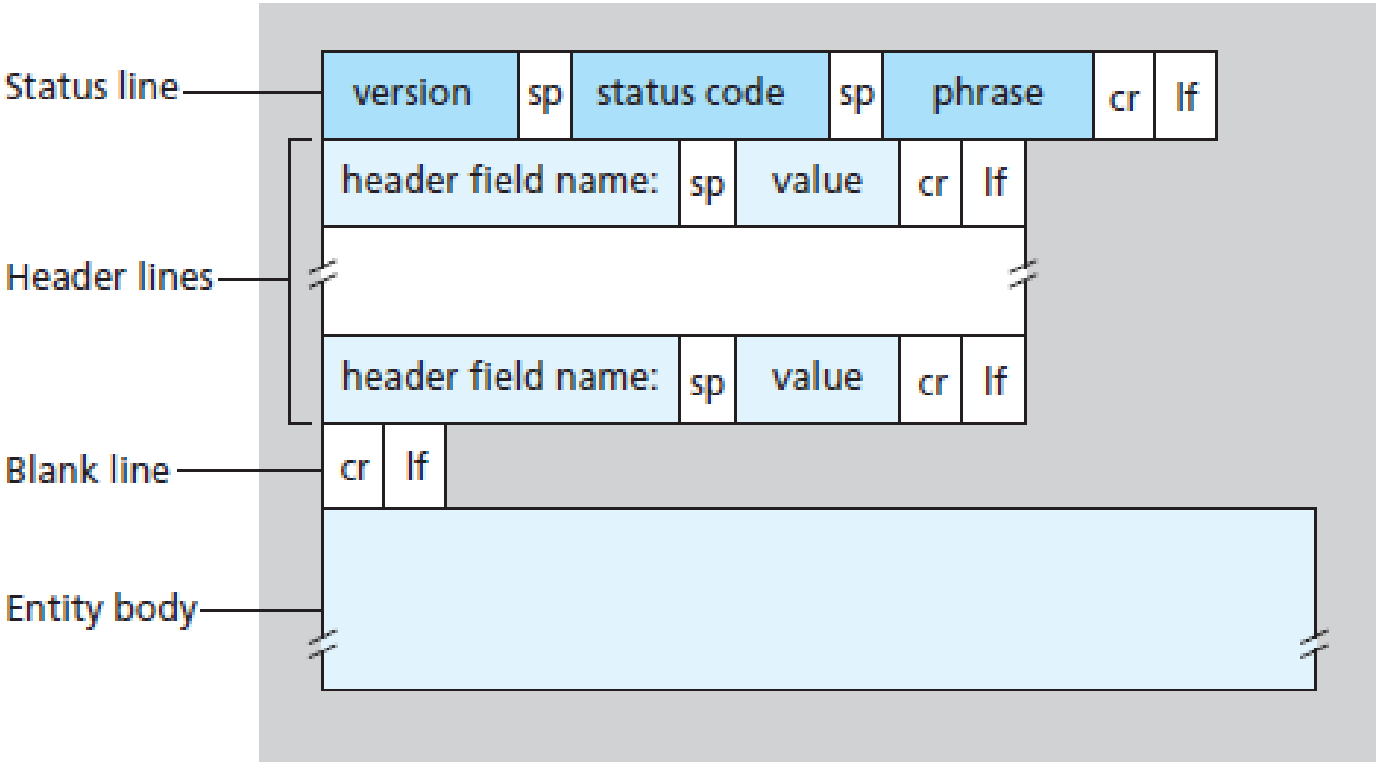
header lines

time and date when the object was created or last modified

the object in the entity body is HTML text.

Entity body

HTTP response message: general format



HTTP response status codes

- status code appears in 1st line in server-to-client response message.
- some sample codes:

200 OK : request succeeded, requested object later in this message

301 Moved Permanently : requested object moved, new location specified later in this message (in the *Location:* field)

400 Bad Request : request message not understood by server

404 Not Found : requested document not found on this server

505 HTTP Version Not Supported

Trying out http (client side) for yourself

1. Telnet to your favorite Web server:

```
telnet www.manutd.com 80
```

Opens TCP connection to port 80
(default http server port) at www.manutd.com
Anything typed in sent
to port 80 at www.manutd.com

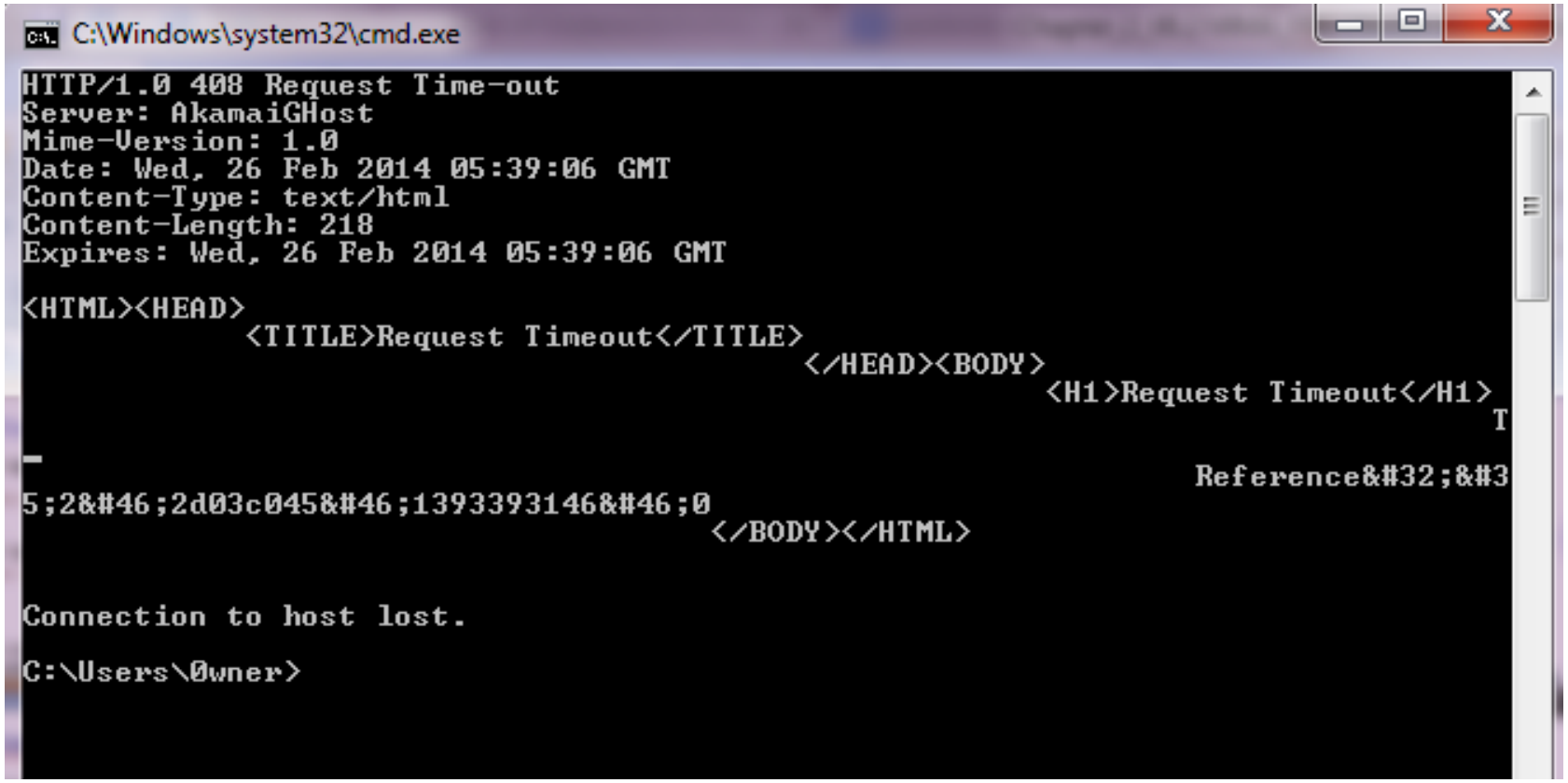
2. Type in a GET http request:

```
GET /index.html HTTP/1.0
```

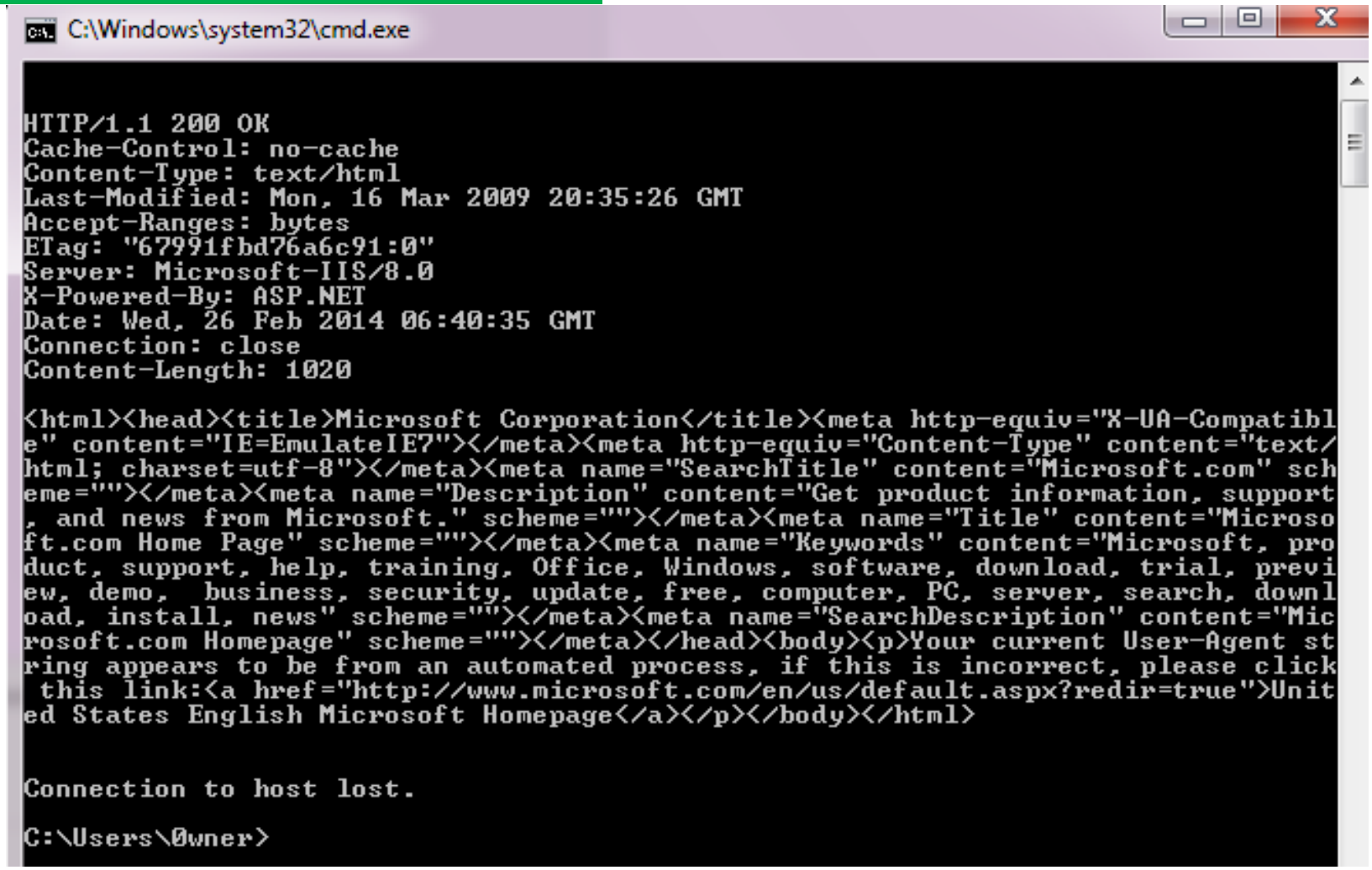
Type this and hit carriage return twice → you have sent this minimal (but complete) GET request to http server
* you can't see what you are typing

3. Look at response message sent by http server!

```
telnet www.manutd.com 80
```



```
telnet www.microsoft.com 80
→ GET / HTTP/1.0
```





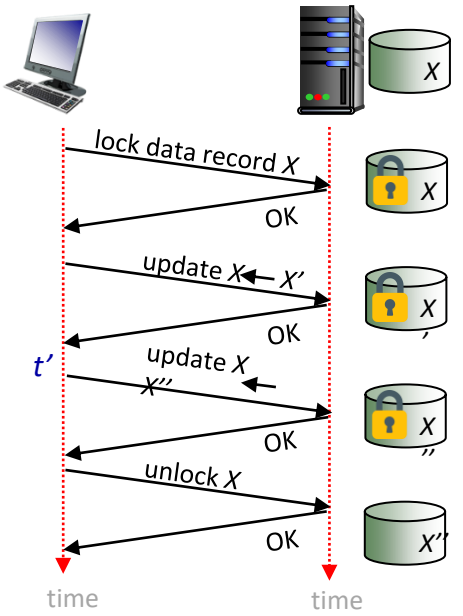
We will stop here for today.
Next class: Cookies (the web kind)

Maintaining user/server state: cookies

Recall: HTTP GET/response interaction is *stateless*

- no notion of multi-step exchanges of HTTP messages to complete a Web "transaction"
  - no need for client/server to track "state" of multi-step exchange
  - all HTTP requests are independent of each other
  - no need for client/server to "recover" from a partially-completed-but-never-completely-completed transaction

a stateful protocol: client makes two changes to X, or none at all



Q: what happens if network connection or client crashes at *t'* ?

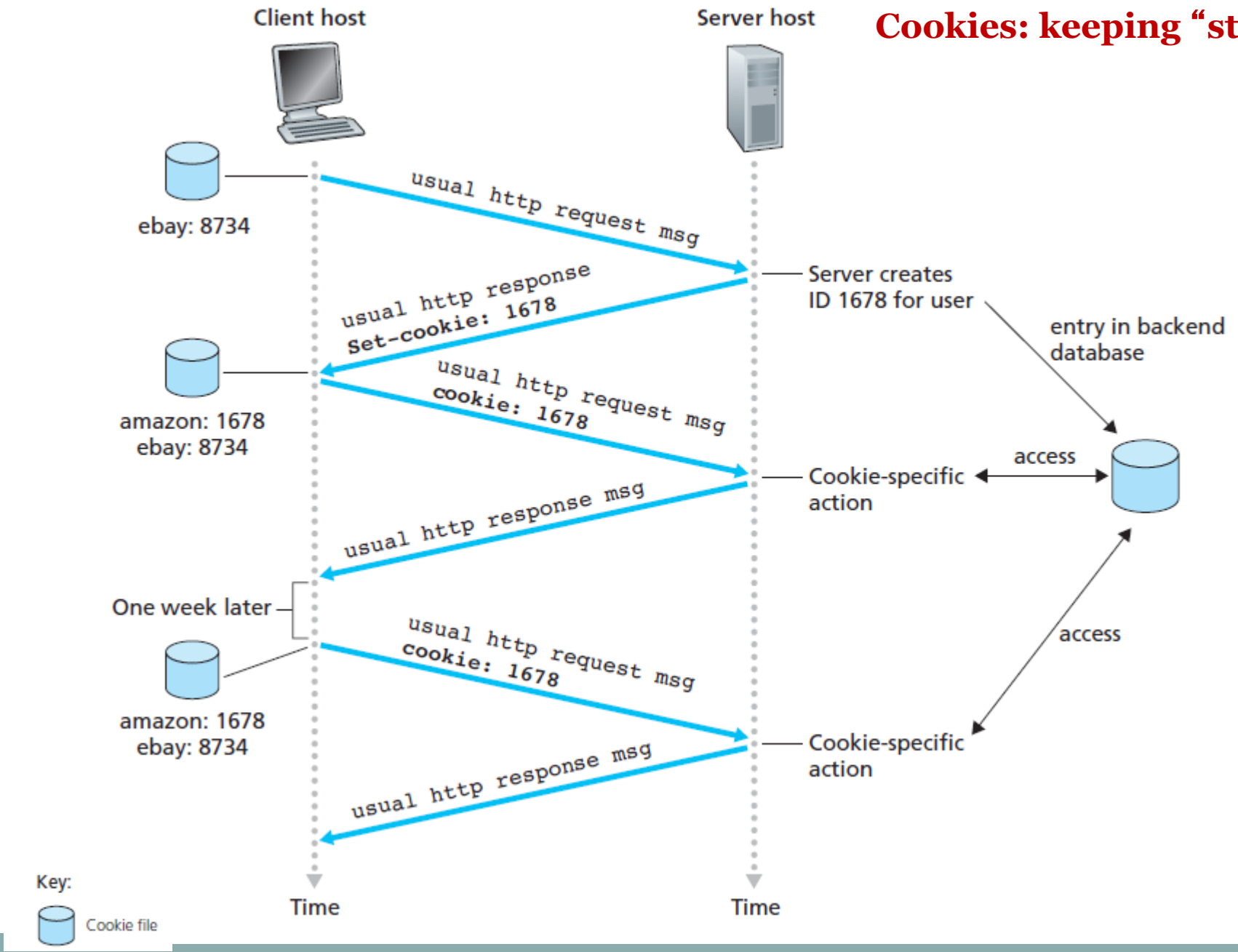
User-server state: cookies

- many Web sites and client browser use cookies – help Web servers handle thousands of simultaneous TCP connections
- *4 components of cookie technology:*
  1) cookie header line of HTTP *response* message
  2) cookie header line in next HTTP *request* message
  3) cookie file kept on user' s host, managed by user' s browser
  4) back-end database at Web site

example:

- Susan always access Internet from PC
- visits specific e-commerce site for first time
- when initial HTTP requests arrives at site, site creates:
  - unique ID (aka "cookie")
  - entry in backend database for ID
- subsequent HTTP requests from Susan to this site will contain cookie ID value, allowing site to "identify" Susan

Cookies: keeping "state"



Cookies (continued)

what cookies can be used for:

- authorization
- shopping carts
- recommendations
- user session state (Web e-mail)

Challenge: how to keep "state"?:

- At protocol endpoints: maintain state at sender/receiver over multiple transactions
- In messages: cookies in HTTP messages carry state

aside

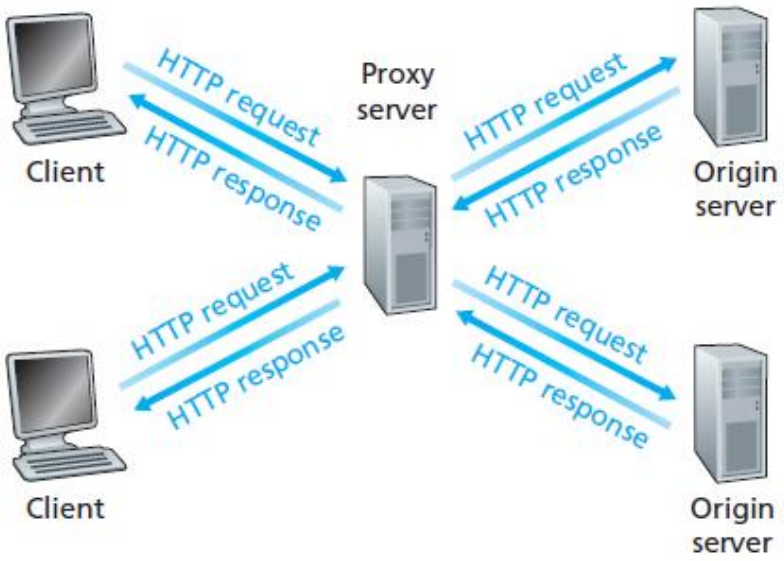
cookies and privacy:

- cookies permit sites to learn a lot about you on their site
- third party persistent cookies (tracking cookies) allow common identity (cookie value) to be tracked across multiple web sites

Web caches (aka proxy server)

goal: satisfy client request without involving origin server

- user configures browser to point to (local) Web cache
- browser sends all HTTP requests to cache
  - If object in cache: cache returns object to client
  - else cache requests object from origin server, caches received object, then returns object to client



More about Web caching

- Web cache acts as both client and server
  - server for original requesting client
  - client to origin server
- typically cache is installed by ISP (university, company, residential ISP)

  - server tells cache about object's allowable caching in response header:

```
Cache-Control: max-age=<seconds>
```

```
Cache-Control: no-cache
```

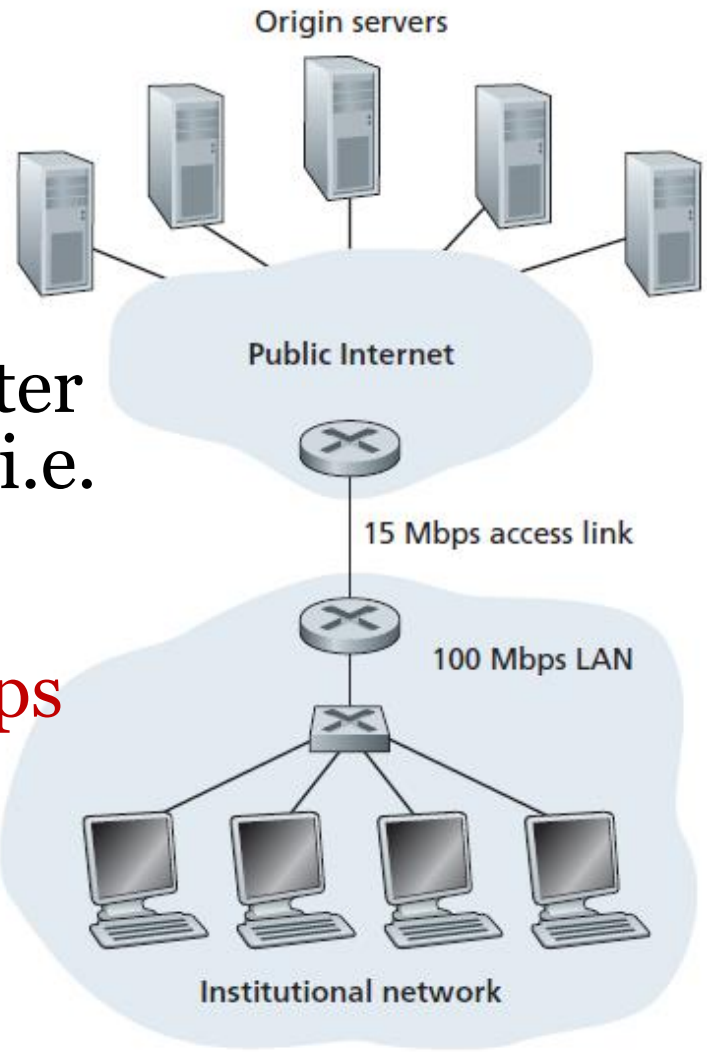
why Web caching?

- reduce response time for client request ➔ cache is closer to client
- reduce traffic on an institution's access link
- Internet dense with caches: enables "poor" content providers to effectively deliver content (so too does P2P file sharing)

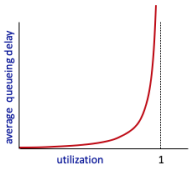
Caching example: slim access link

Assumptions:

- average object size: 1Mbits
- average request rate from browsers to origin servers:15/sec
- RTT from institutional router to any origin server: 2 sec (i.e. the Internet delay)
- access link rate: 15 Mbps
- Institutional LAN : 100Mbps



Caching example: *slim* access link



Assumptions:

- average object size: 1Mbits
- average request rate from browsers to origin servers:15/sec
- RTT from institutional router to any origin server: 2 sec (i.e. the Internet delay)
- access link rate: 15 Mbps
- Institutional LAN : 100Mbps



Network Utilization

- ***LAN Utilization***
  = Traffic Intensity x 100%
- Traffic intensity
  = L.a/R
  = (1Mbits) x (15req/sec) / 100Mbps= 0.15

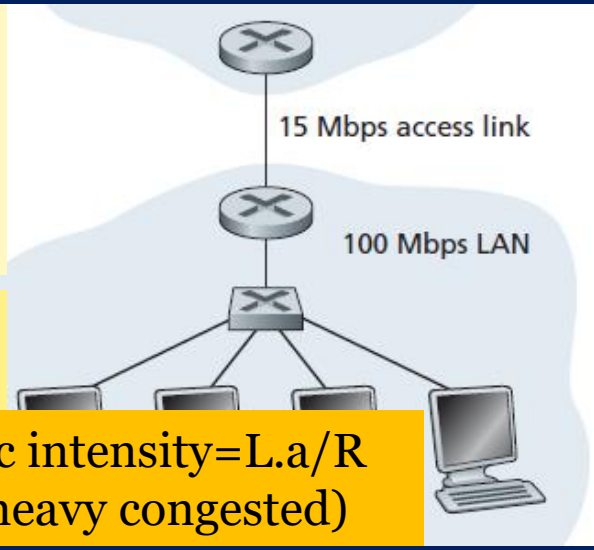
➔ LAN utilization = (0.15 x 100%) = 15%

- ***Access Link Utilization***
- Traffic intensity
  = L.a/R
  = (1Mbits) x (15req/sec) / 15Mbps
  = 1

➔ LAN utilization = (1 x 100%) = 100%

- ***Total delay***

= Internet delay + access link delay + LAN delay
= 2 sec + minutes + usecs



Traffic intensity=L.a/R
→ 1 (heavy congested)

What will happen if we make the access link bigger?

Caching example: *fatter* access link

Assumptions:

- average object size: 1Mbits
- average request rate from browsers to origin servers:15/sec
- RTT from institutional router to any origin server: 2 sec (i.e. the Internet delay)
- access link rate: ***15 Mbps ➔ to 100Mbps***
- Institutional LAN : 100Mbps



- ***Total delay***

= Internet delay + access link delay + LAN delay

= 2 sec + usecs + usecs

≈ 2 sec (Internet delay)

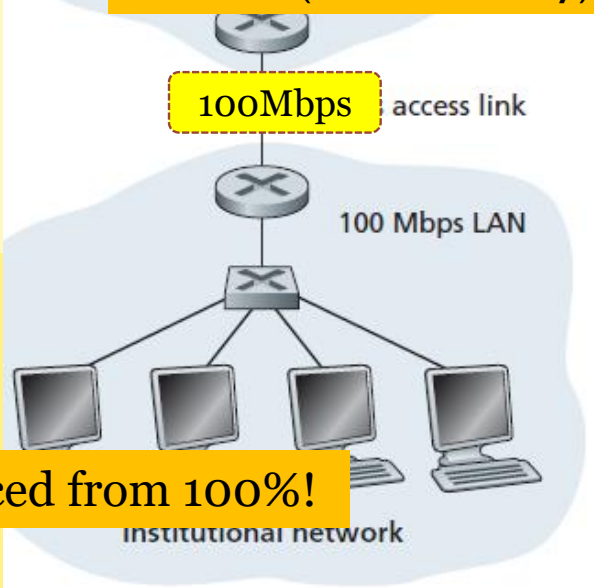
Network Utilization

- ***LAN Utilization***
- Traffic intensity
  = L.a/R
  = (1Mbits) x (15req/sec) / 100Mbps
  = 0.15

➔ LAN utilization = (0.15 x 100%) = 15%

- ***Access Link Utilization***
- Traffic intensity
  = L.a/R
  = (1Mbits) x (15req/sec) / **100Mbps**
  = 0.15

➔ LAN utilization = (0.15 x 100%) = **15%**



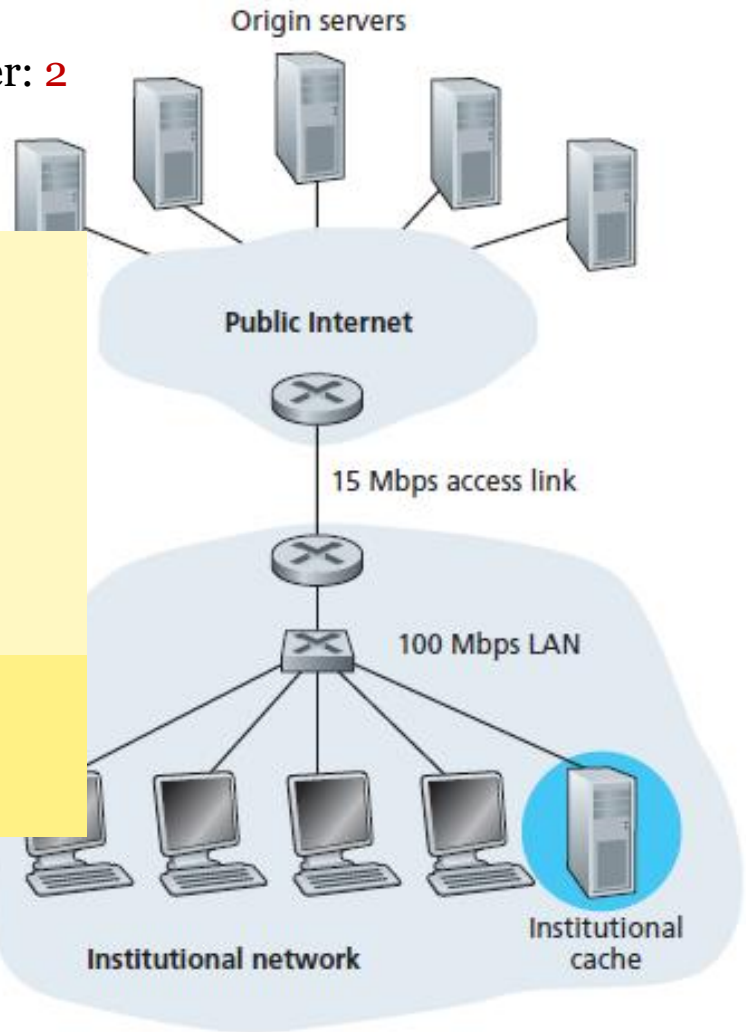
Reduced from 100%!

Unfortunately, increasing access link speed is NOT CHEAP!

Caching example: install local cache

Assumptions:

- average object size: 1Mbits
- average request rate from browsers to origin servers:15/sec
- RTT from institutional router to any origin server: 2 sec (i.e. the Internet delay)
- access link rate: 15 Mbps
- Institutional LAN : 100Mbps



Network Utilization

- ***LAN Utilization***
- Traffic intensity
  = L.a/R
  = (1Mbits) x (15req/sec) / 100Mbps
  = 0.15

➔ LAN utilization = (0.15 x 100%) = 15%

- ***Access Link Utilization***
- How to calculate?

Cost: web cache (cheap!)

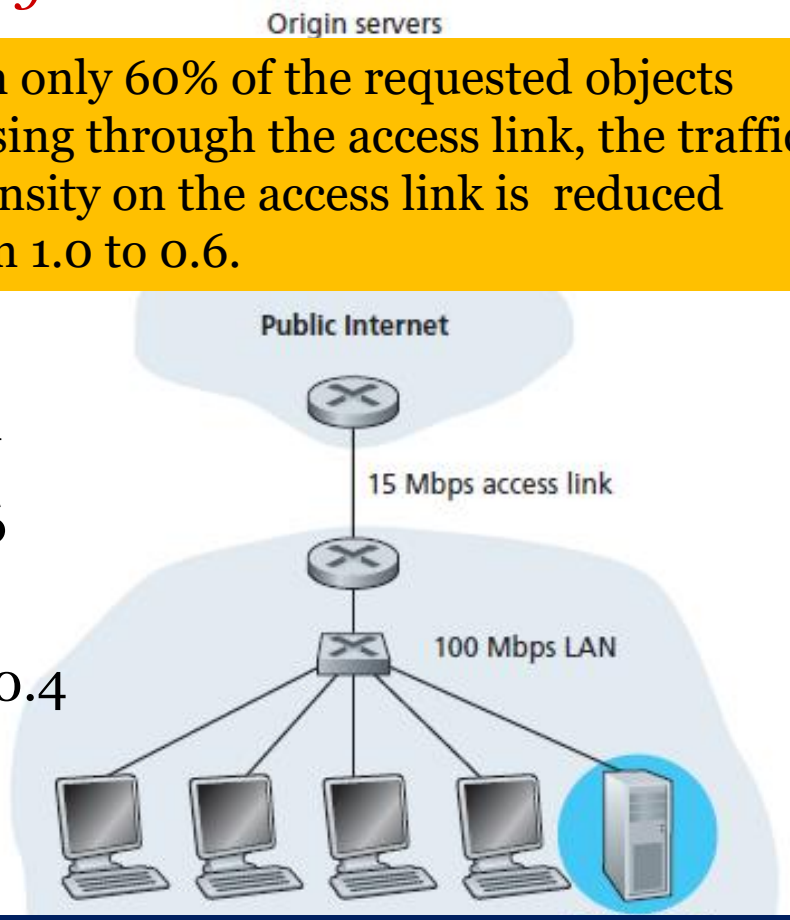
Caching example: install local cache

Calculating access link utilization, delay with cache:

- Suppose cache hit rate is 0.4
  - 40% requests satisfied at cache,
  - 60% requests satisfied at origin

with only 60% of the requested objects passing through the access link, the traffic intensity on the access link is reduced from 1.0 to 0.6.

- Access link utilization:
  - 60% of requests use access link
- Data rate to browsers over access link
  = 0.6*15 Mbps = 9 Mbps
  - Access link utilization = 9/15 = 0.6 (i.e. 60%)
- Average end-end delay
  = 0.6 * (delay from origin servers) +0.4 * (delay when satisfied at cache)
  = 0.6 (2.01) + 0.4 (~msecs)
  = 0.6 (2.01) + 0.4 (~msecs)
  = ~ 1.21 secs



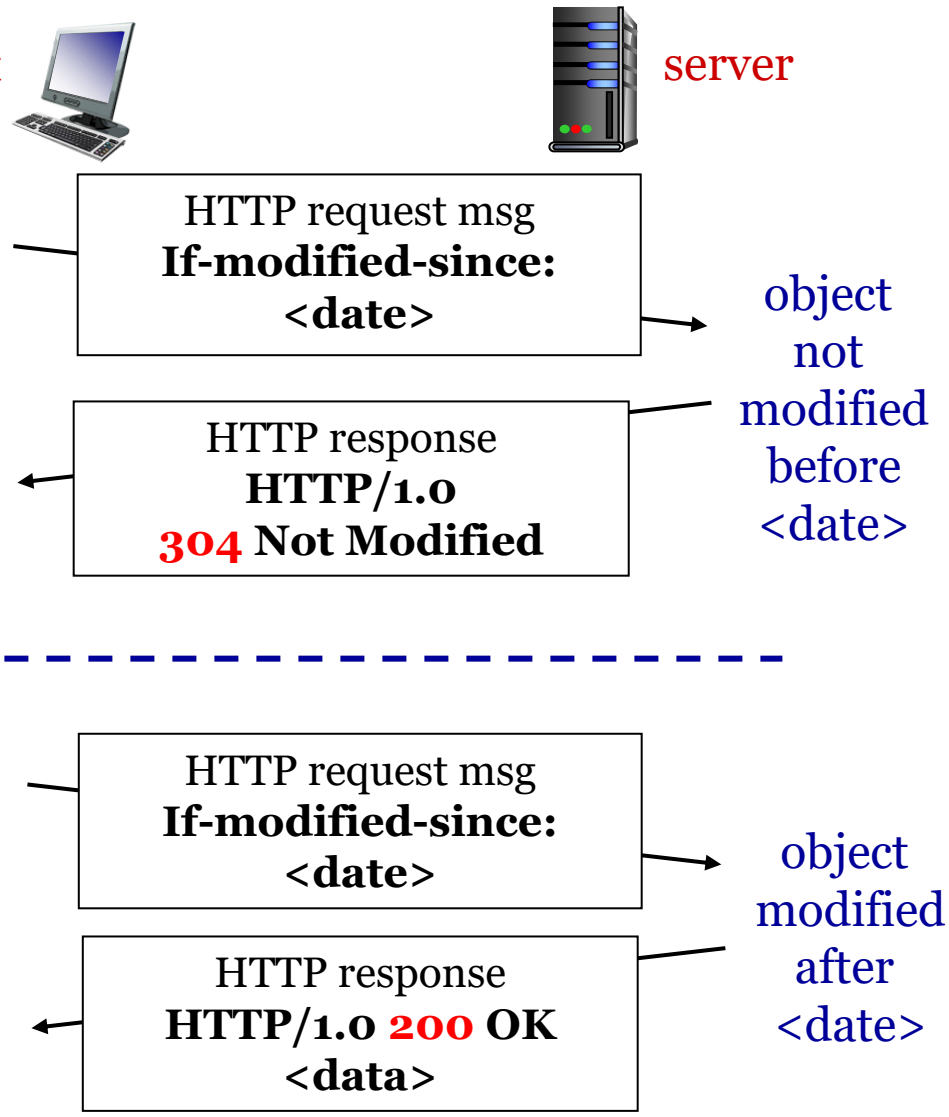
lower response time and doesn't require an upgrade to access link

Conditional GET

- *Goal:* don't send object if cache has up-to-date cached version
  - no object transmission delay
  - lower link utilization
- *client:* specify date of cached copy in HTTP request

  `If-modified-since: <date>`
- *server:* response contains no object if cached copy is up-to-date:

  `HTTP/1.0 304 Not Modified`



Browser requests and a proxy cache sends a request message to a Web server

```
GET /fruit/kiwi.gif HTTP/1.1
Host: www.exotiquecuisine.com
```

Then, the Web server sends a response message with the requested object to the cache

```
HTTP/1.1 200 OK
Date: Sat, 8 Oct 2011 15:39:29
Server: Apache/1.3.0 (Unix)
Last-Modified: Wed, 7 Sep 2011 09:23:24
Content-Type: image/gif
(data data data data data ...)
```

The cache forwards the object to the requesting browser and keeps a copy locally (including the last-modified date)

1 week later ➔ Browser requests the same object to cache

cache performs an up-to-date check to server (for modification) by issuing a conditional GET.

```
GET /fruit/kiwi.gif HTTP/1.1
Host: www.exotiquecuisine.com
If-modified-since: Wed, 7 Sep 2011 09:23:24
```

If not modified, Web server sends a response message

```
HTTP/1.1 304 Not Modified
Date: Sat, 15 Oct 2011 15:39:29
Server: Apache/1.3.0 (Unix)
(empty entity body)
```

The object in cache is forwarded to the browser

HTTP/2

Key goal: decreased delay in multi-object HTTP requests

HTTP1.1: introduced multiple, pipelined GETs over single TCP connection

- server responds *in-order* (FCFS: first-come-first-served scheduling) to GET requests
- with FCFS, small object may have to wait for transmission (head-of-line (HOL) blocking) behind large object(s)
- loss recovery (retransmitting lost TCP segments) stalls object transmission

HTTP/2

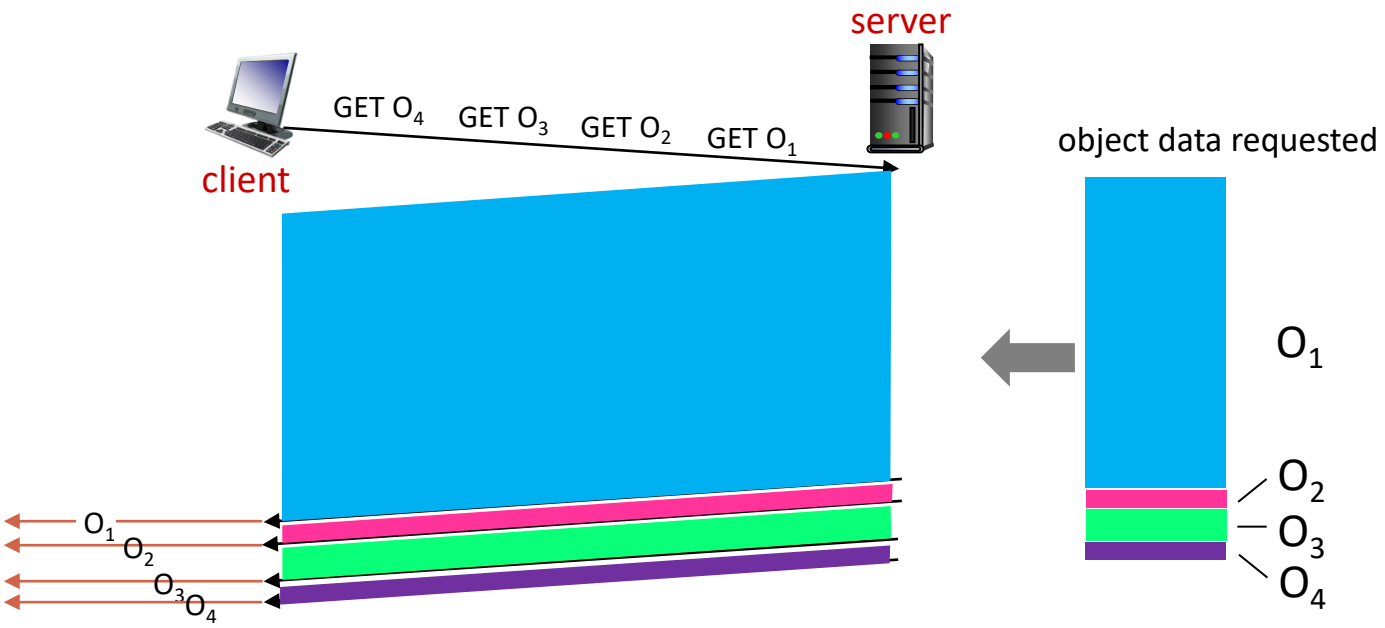
Key goal: decreased delay in multi-object HTTP requests

HTTP/2: [RFC 7540, 2015] increased flexibility at *server* in sending objects to client:

- methods, status codes, most header fields unchanged from HTTP 1.1
- transmission order of requested objects based on client-specified object priority (not necessarily FCFS)
- *push* unrequested objects to client
- divide objects into frames, schedule frames to mitigate HOL blocking

HTTP/2: mitigating HOL blocking

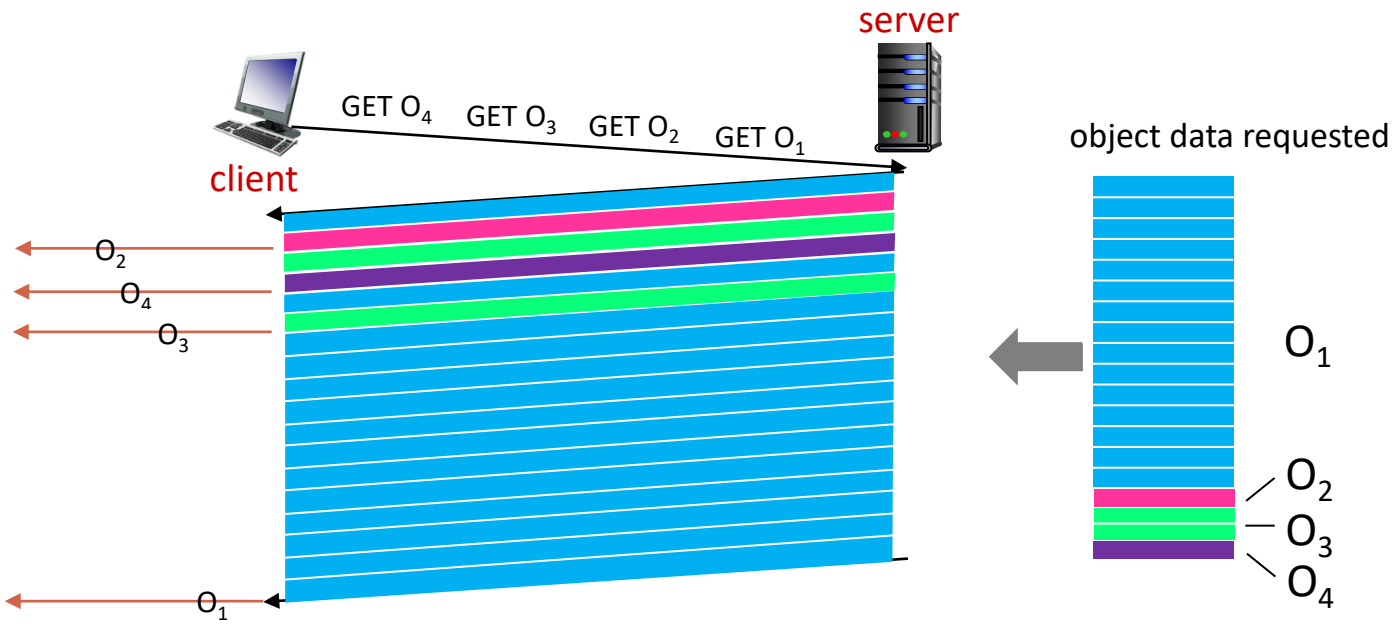
HTTP 1.1: client requests 1 large object (e.g., video file) and 3 smaller objects



objects delivered in order requested: O_2, O_3, O_4 wait behind O_1

HTTP/2: mitigating HOL blocking

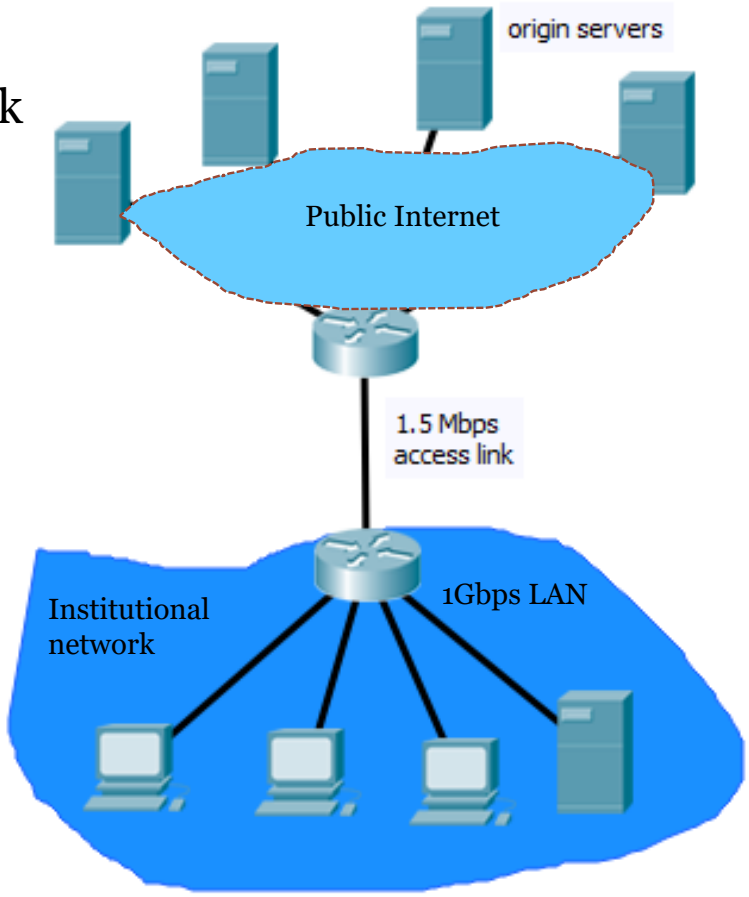
HTTP/2: objects divided into frames, frame transmission interleaved



O_2, O_3, O_4 delivered quickly, O_1 slightly delayed

Homework

- The diagram shows an institutional network connected to public Internet.
- Answer the following questions based on the assumption below:



Average object size: 100K bits

Average request rate from browsers to origin servers: 15/sec,

Access link rate: 1.5 Mbps,

Access LAN rate: 1Gbps

Do this on your own. Compare your answer with your friends. **Only ONE person from class email me with the agreed upon answer.

Homework



Average object size: 100K bits, average request rate from browsers to origin servers: 15/sec, access link rate: 1.5 Mbps, & access LAN: 1Gbps

a. Without web cache server, calculate access link utilization.
b. Assume the access link rate is increased to 3Mbps. Without web cache server, calculate the access link utilization.
c. Assume the link capacity is unchanged (1.5Mbps) and a web server cache is used which has a hit rate of 50%, find the access link utilization.
d. Discuss the results in b and c (in terms of utilization). What is your conclusion?

FTP, Email, DNS, P2P & CDN



FTP: File Transfer Protocol

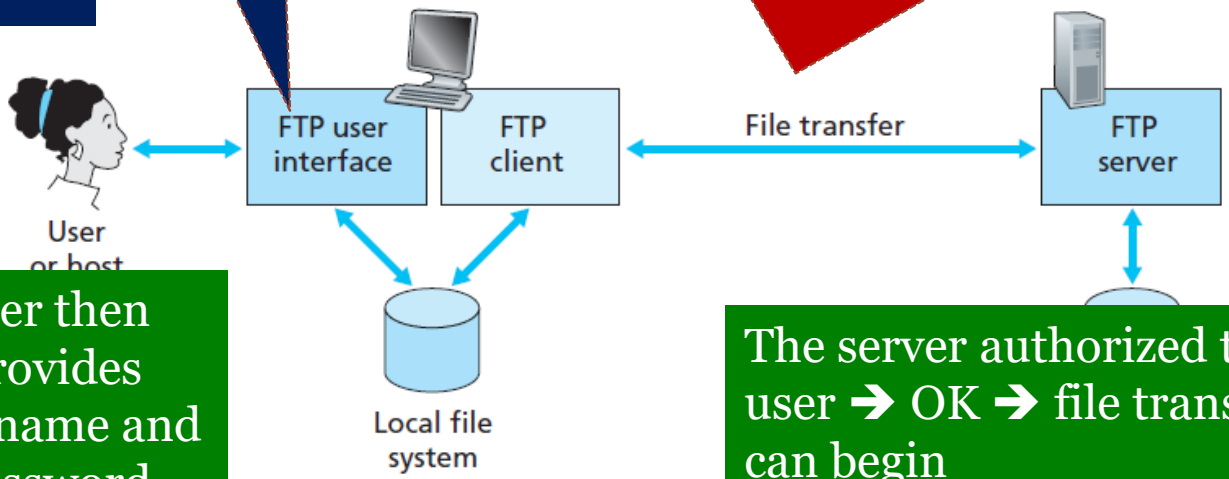
- In FTP, a user will either transfer files to or from a remote host.
- In order for the user to access the remote account, the user must provide a form of authentication – username and password.
  - Upon successful authorization, the user can transfer files from the local file system to the remote file system and vice versa.
- FTP uses a client/server model
  - client: side that initiates transfer (either to/from remote)
  - server: remote host
- FTP : runs on TCP
- FTP : RFC 959
- FTP server: port 21

FTP: File Transfer Protocol

user interacts with FTP through an FTP user agent

User provides the hostname of the remote host

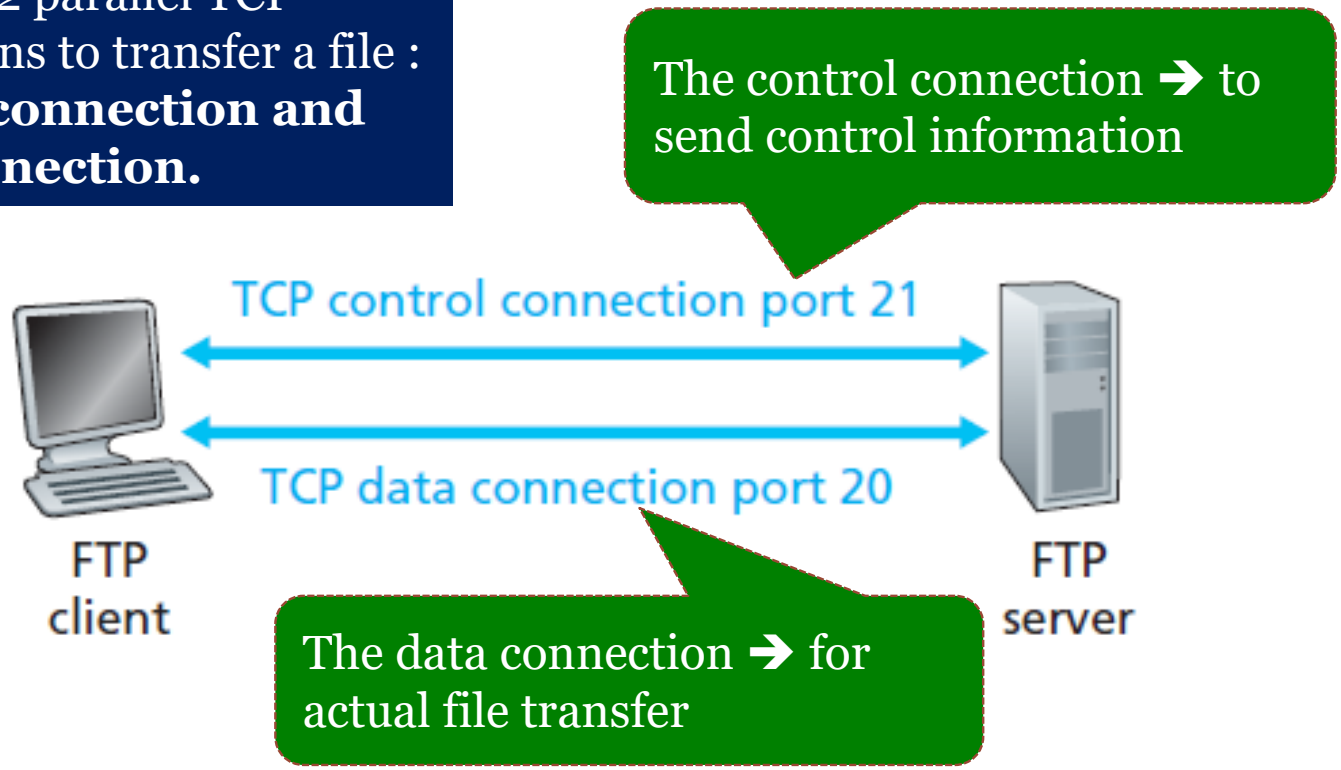
The FTP client process will establish a TCP connection with the FTP server process in the remote host



user then provides username and password

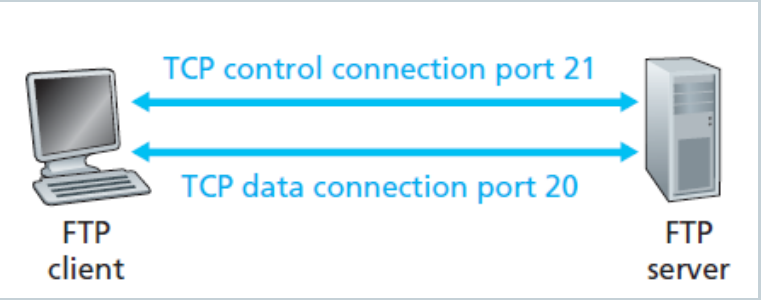
The server authorized the user ➔ OK ➔ file transfer can begin

FTP uses 2 parallel TCP connections to transfer a file : **control connection and data connection.**



FTP: separate control, data connections

- FTP client contacts FTP server at port 21, using TCP
- client authorized over control connection
- client browses remote directory, sends commands over control connection
- when server receives file transfer command, *server* opens *2nd* TCP data connection (for file) *to* client ➔ port 20
- after transferring exactly one file, server closes data connection



- the control connection remains open throughout the duration of the user session
- server opens another TCP data connection to transfer another file
- FTP server maintains "state" of the user ➔ current directory, earlier authentication

FTP commands, responses

sample commands:

- sent as ASCII text over control channel
- **`USER` *`username`***
- **`PASS` *`password`***
- **`LIST`** return list of file in current directory
- **`RETR filename`** retrieves (gets) file
- **`STOR filename`** stores (puts) file onto remote host

sample return codes

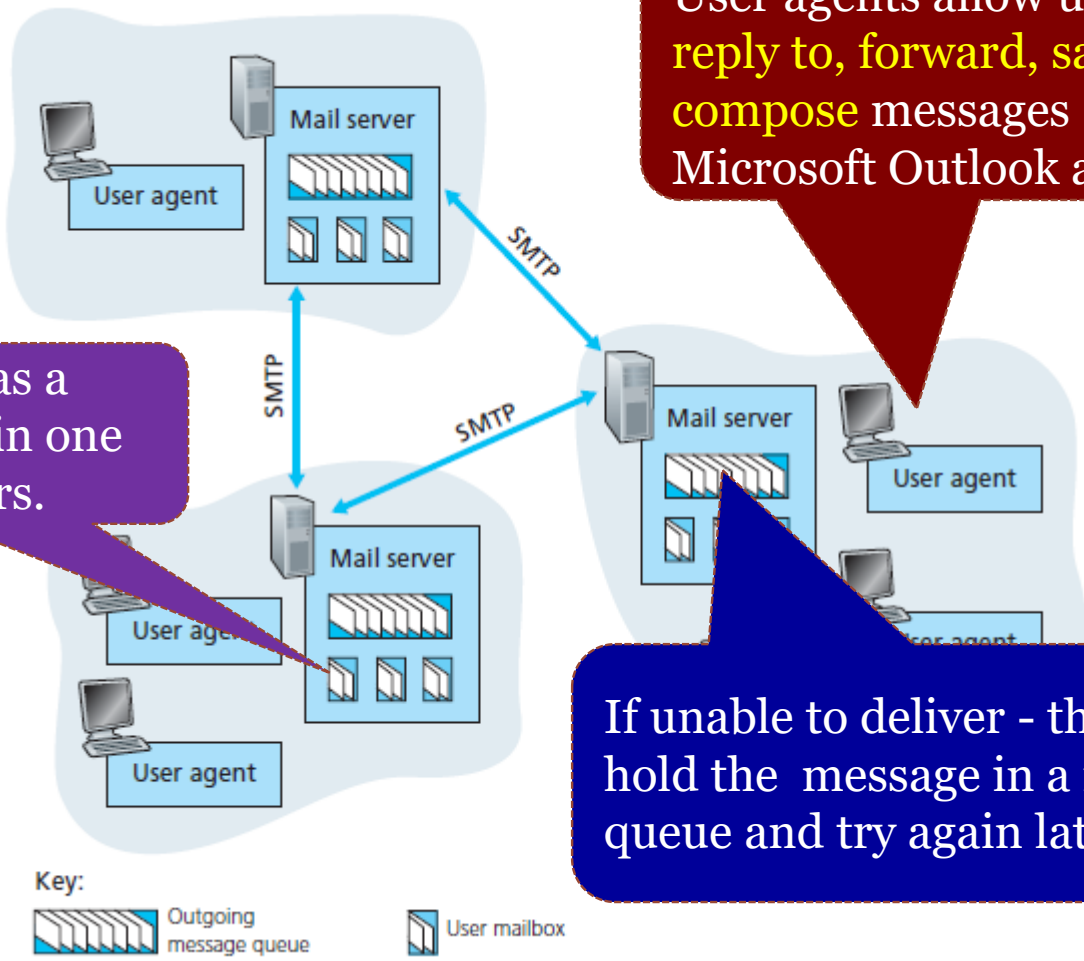
- status code and phrase (as in HTTP)
- **`331 Username OK, password required`**
- **`125 data connection already open; transfer starting`**
- **`425 Can't open data connection`**
- **`452 Error writing file`**

Electronic mail

- Email is an asynchronous communication medium
  - User can send and read messages at their own convenience → no need for coordination
- Email: uses TCP and port 25
- Email: uses client-server method
  - client side - which executes on the sender's mail server
  - server side - which executes on the recipient's mail server.
  - mail server can also be a client when a mail server sends mail to other mail servers

three major components: user agents, mail servers, the protocol (SMTP).

User agents allow users to read, reply to, forward, save, and compose messages ➔ e.g. Microsoft Outlook and Apple Mail



Each recipient has a mailbox located in one of the mail servers.

If unable to deliver - the server will hold the message in a message queue and try again later

Electronic mail

Three major components:

- user agents
- mail servers
- simple mail transfer protocol: SMTP

User Agent

- a.k.a. "mail reader"
- composing, editing, reading mail messages
- e.g., Outlook, Thunderbird, iPhone mail client
- outgoing, incoming messages stored on server

mail servers:

- *mailbox* contains incoming messages for user
- *message queue* of outgoing (to be sent) mail messages
- *SMTP protocol* between mail servers to send email messages
  - client: sending mail server
  - "server": receiving mail server

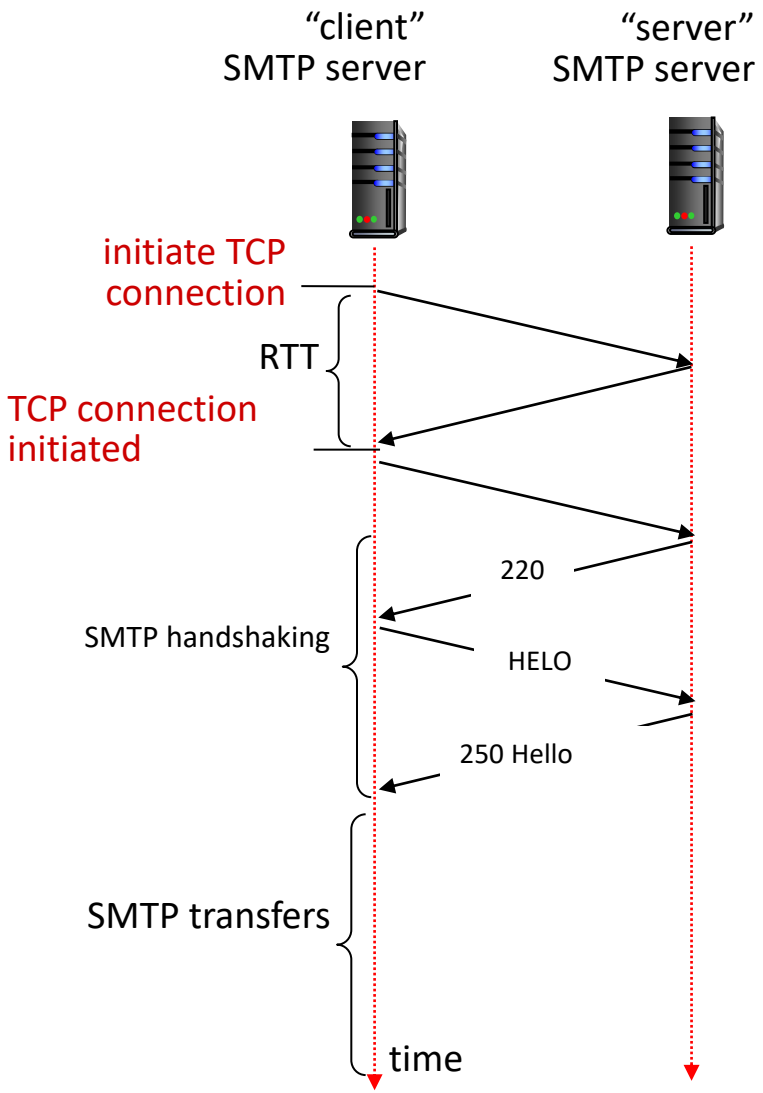
Electronic Mail: SMTP

- uses TCP to reliably transfer email message from client to server, port 25
- direct transfer: sending server to receiving server
- three phases of transfer
  - SMTP handshaking (greeting)
  - SMTP transfer of messages
  - SMTP closure
- command/response interaction (like HTTP)
  - commands: ASCII text
  - response: status code and phrase
- messages must be in 7-bit ASCII
- SMTP uses persistent connections

SMTP RFC (5321)

three phases of transfer

- SMTP handshaking (greeting)
- SMTP transfer of messages
- SMTP closure



1 Alice uses UA to compose message "to" bob@someschool.edu

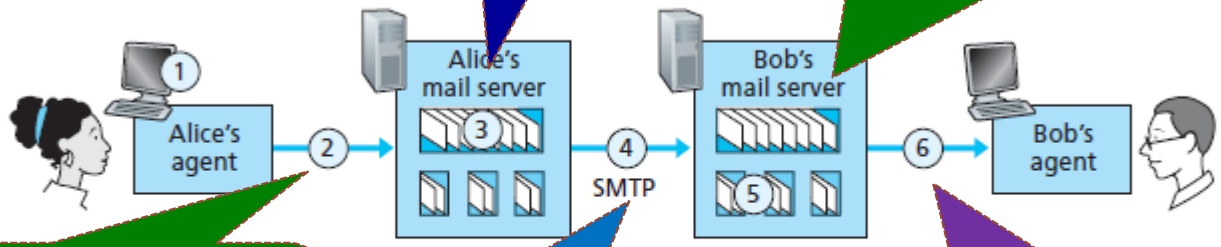
2 Alice's UA sends message to her mail server; message placed in message queue

3 client side of SMTP opens TCP connection with Bob's mail server

4 SMTP client sends Alice's message over the TCP connection

5 Bob's mail server places the message in Bob's mailbox

6 Bob invokes his user agent to read message

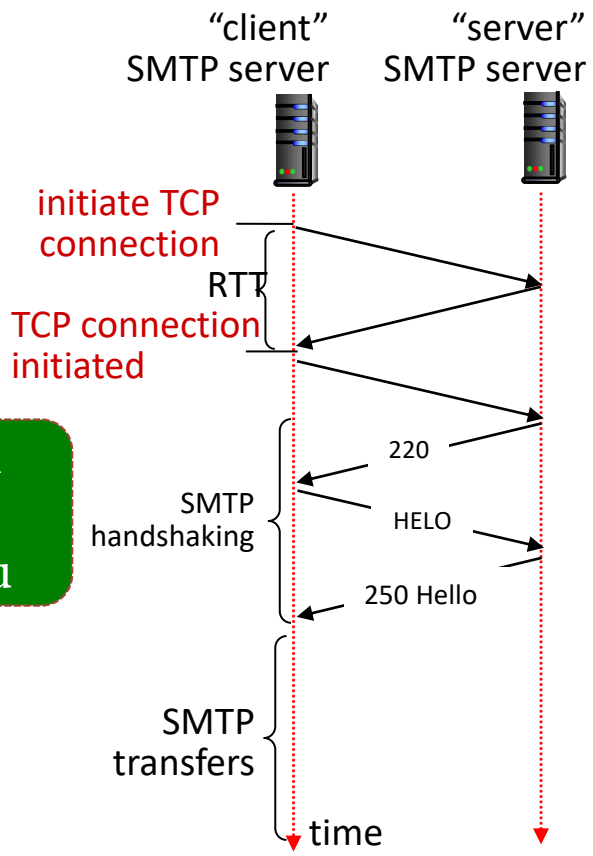


S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr ... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection

Server hostname

Client hostname

Messages from crepes.fr to hamburger.edu



SMTP: comparison with HTTP

- Both HTTP and SMTP transfer files and use persistent connection
- HTTP is mainly a pull protocol ➔ servers have information that clients pull at their convenience ➔ clients initiated service
- SMTP is mainly a push protocol ➔ the sending mail server pushes the file to the receiving mail server ➔server initiates service
- SMTP requires each message, including the body of each message, to be in 7-bit ASCII format ➔ HTTP does not.
- In sending messages ➔ HTTP encapsulates each object in its own HTTP response message. Internet mail places all of the message's objects into one message.
- SMTP: multiple objects sent in multipart message

Mail message format

SMTP: protocol for exchanging email messages

RFC 2822 defines syntax for e-mail message itself (like HTML defines syntax for web documents):

- header lines, e.g.,
  - To:
  - From:
  - Subject:

```
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Searching for the meaning of life.
```

These are part of the mail message itself.

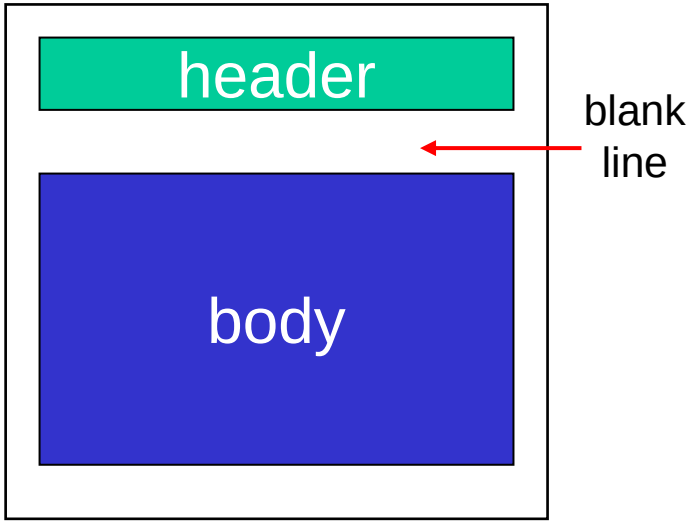
different from SMTP MAIL FROM, RCPT TO: commands!

- Body: the "message"
  - ASCII characters only

These are part of the SMTP handshaking protocol

- Must have a blank line between header and body

Mail message format



Mail access protocols

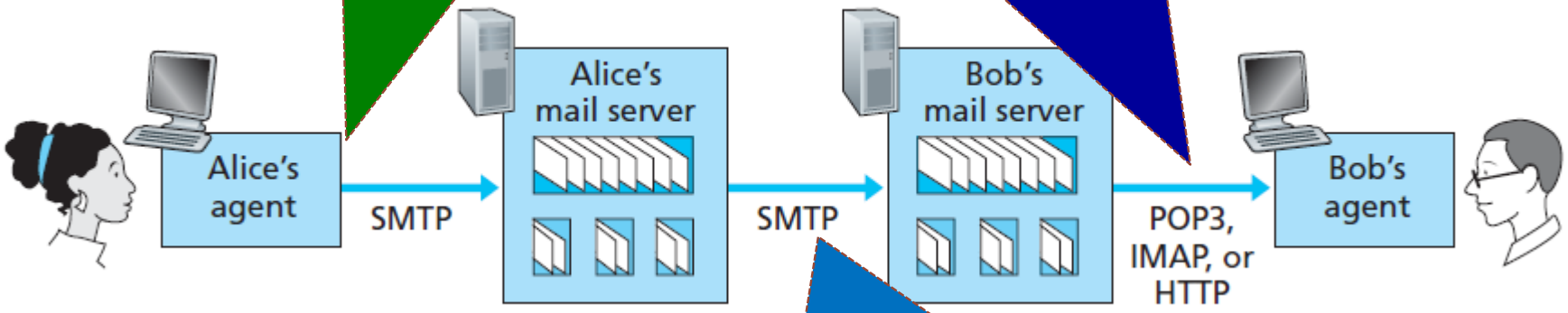
- Suppose you want to read your email from your PC
  - Advantage: Easy! Great!
  - Problem: Your PC is not always ON (like a server) ➔ Mail can't get to you.
    - Solution: Email sent is kept in the mail server until your user agent gets it from there.
  - Problem: User agent cannot run SMTP – because SMTP is push protocol, you need a pull protocol.
    - Solution: Use a special mail access protocol that transfers messages from the mail server to your local PC.
- There are currently a number of mail access protocols, including Post Office Protocol—Version 3 (POP3), Internet Mail Access Protocol (IMAP), and HTTP

Mail access protocols

- SMTP: delivery/storage to receiver's server
- mail access protocol: retrieval from server
  - POP: Post Office Protocol [RFC 1939]: authorization, download
  - IMAP: Internet Mail Access Protocol [RFC 3501]: messages stored on server, IMAP provides retrieval, deletion, folders of stored messages on server
  - HTTP: gmail, Hotmail, Yahoo!Mail, etc. provides web-based interface on top of STMP (to send), IMAP (or POP) to retrieve e-mail messages

Alice's user agent uses SMTP to push the e-mail message into her mail server

A special mail access protocol is used to transfers messages from Bob's mail server to his local PC.



Alice's mail server uses SMTP (as an SMTP client) to relay the e-mail message to Bob's mail server.

POP3 protocol

- POP3 is an extremely simple mail access protocol.
  - Due to its simplicity, its functionality is rather limited.
- POP3 begins when the user agent (the client) opens a TCP connection to the mail server (the server) on port 110.
- With the TCP connection established, POP3 goes through 3 phases:
  - Authorization → get username and password
  - Transaction → gets the email, delete email, etc
  - Update → server final update of actions (like delete) after closing
- When the user agent issues commands, the server responds with a reply (+OK or –ERR).
  - +OK ➔ the previous command was fine
  - -ERR ➔ something was wrong with the previous command.

Authorization phase:
- Notice password not hidden

transaction phase:
list: list message numbers
retr: retrieve message by number
dele: delete
quit

```
S: +OK POP3 server ready
C: user bob
S: +OK
C: pass hungry
S: +OK user successfully logged on
C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 1 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

POP3 (more…)

- POP3 can run on download-and-delete mode or download-and-keep mode
- download-and-delete mode ➔ user agent gets mail from mail server and deletes mail in server
  - Can only open in 1 device - restricting
- download-and-keep mode ➔ mail in server is not deleted after downloading
  - Can open with many different devices
- POP3 is stateless across sessions

IMAP

- More complex, and has more features than POP3
- Keeps all messages in one place: at server
- Allows user to organize messages in folders
- Keeps user state across sessions:
  - names of folders and mappings between message IDs and folder name
- Also has the ability to read part of the message
  - To accommodate in low bandwidth connections

Works well even over a slow connection such as a modem

HTTP Protocol

- The HTTP protocol is not a protocol dedicated for email communications, but it can be used for accessing your mailbox.
- Also called web based email, this protocol can be used to compose or retrieve emails from an your account.
- Hotmail is a good example of using HTTP as an email protocol.

Web-Based E-Mail

- Hotmail introduced Web-based access mid 1990s.
- The user agent is a Web browser, communicates with its remote mailbox via HTTP
- To access e-mail message, user read mailbox in browser.
- The e-mail message is sent from recipient's mail server to it's browser using HTTP protocol rather than POP3 or IMAP.
- To send e-mail message, it is sent from user's browser to mail server over HTTP rather than SMTP.

Chapter 2: Outline

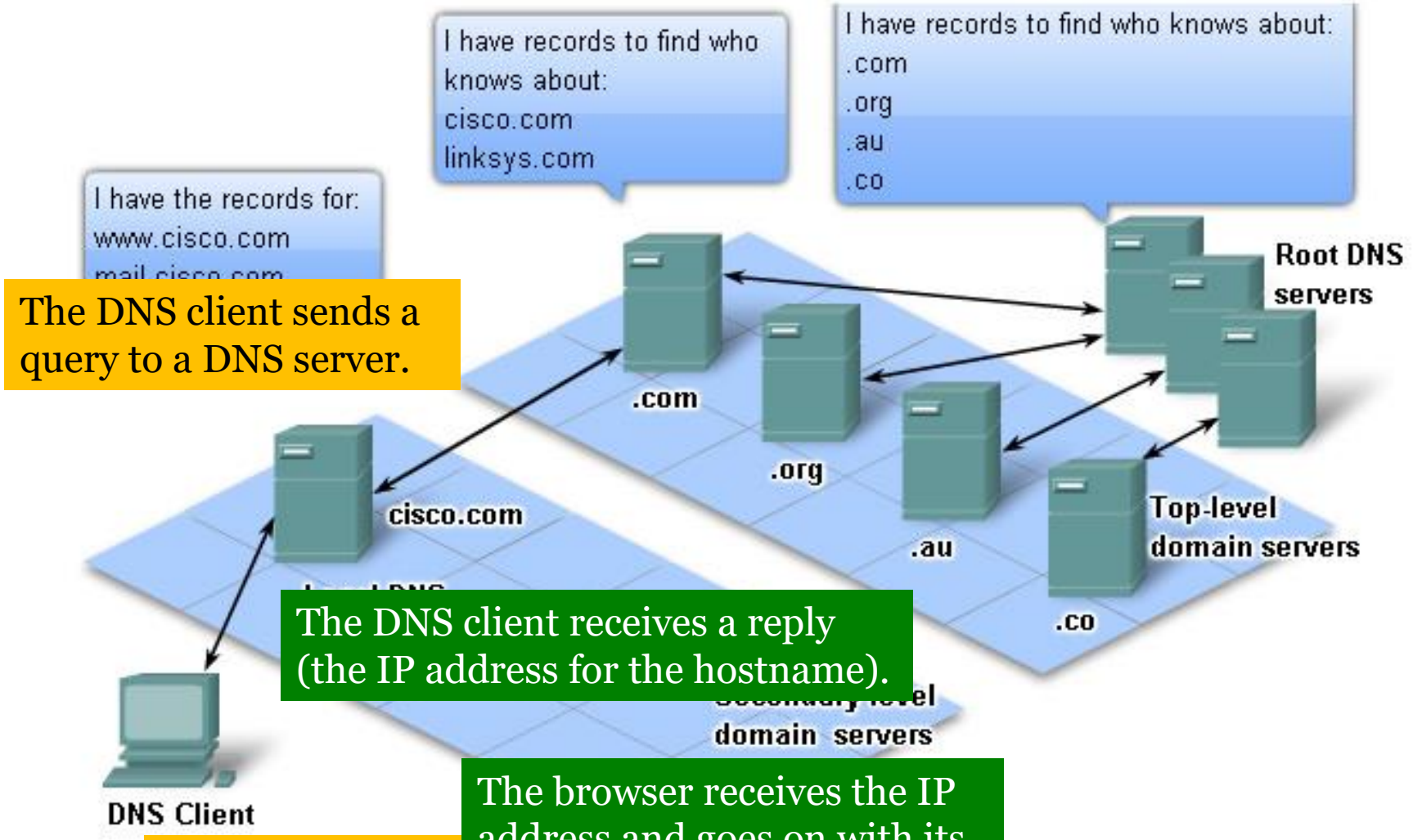
- Principles of network applications
  - app architectures
  - app requirements
- Web and HTTP
- Electronic mail
  - SMTP, POP3, IMAP
- DNS
- P2P applications
- video streaming and content distribution networks
- socket programming with UDP and TCP

DNS: Domain Name System

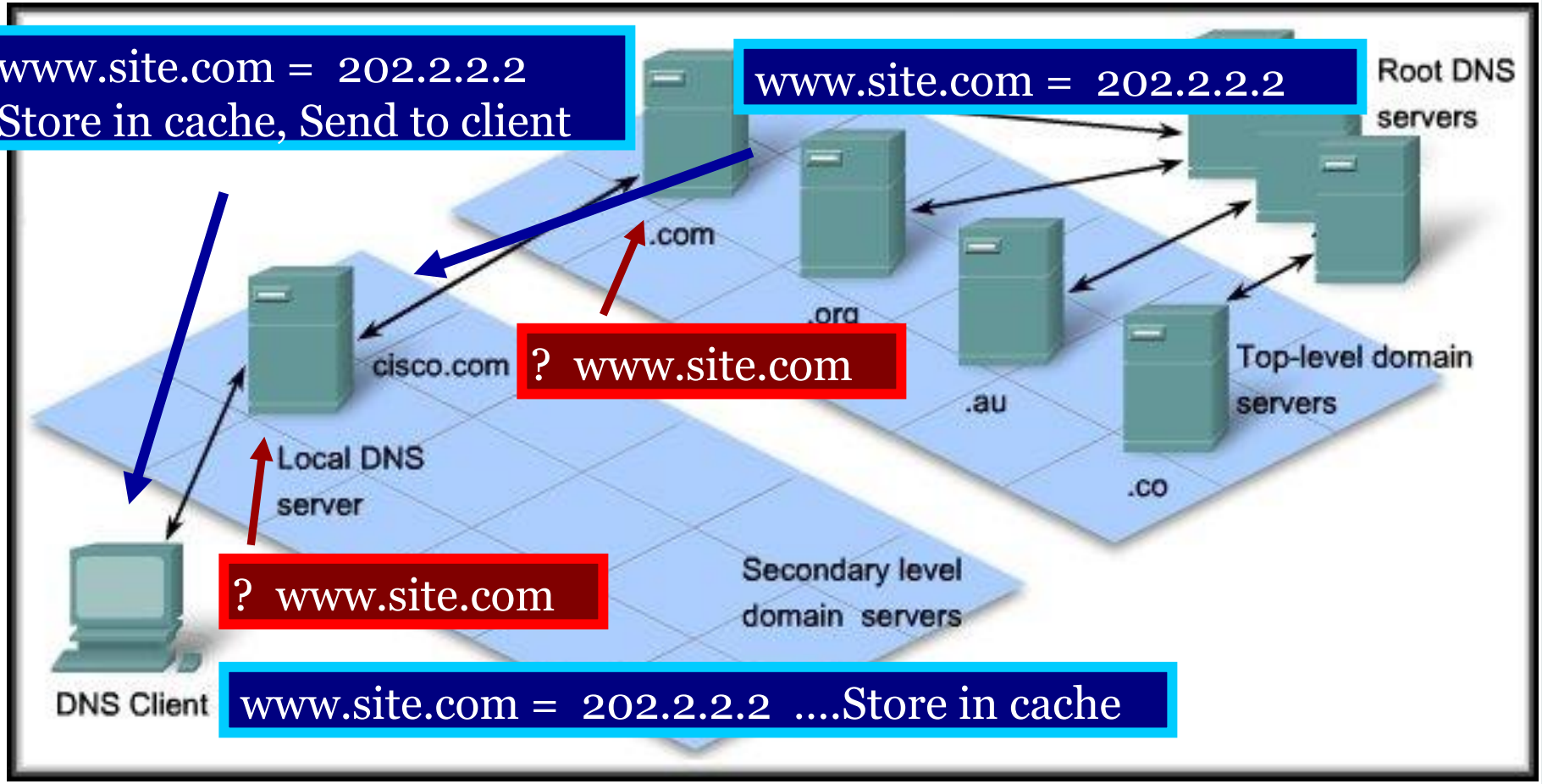
- People can be identified with many identifiers:
  - SSN, name, passport#, IC
- A host can be identified in 1 of 2 identifiers
  - IP address (32 bit)
    - used for addressing datagrams
    - Example: 192.168.1.1, 166.55.34.244
    - Other machines like this identifier
  - "name"
    - Example: www.yahoo.com , www.uni.edu
    - Preferred by humans
- Which identifier to use? – why not both
- How to resolve between them? - DNS

DNS: Domain Name System

- DNS translates hostnames to IP addresses.
- The DNS is
  - a distributed database implemented in a hierarchy of name servers,
  - an application-layer protocol that allows hosts to query the distributed database to resolve names.
- The DNS protocol runs over UDP and uses port 53.
- DNS is commonly used by other application-layer protocols—including HTTP, SMTP, and FTP
  - *note:* core Internet function, implemented as application-layer protocol
  - complexity at network's "edge"


I have records to find who knows about:
cisco.com
linksys.com
I have records to find who knows about:
.com
.org
.au
.co
I have the records for:
www.cisco.com
The DNS client sends a query to a DNS server.
Root DNS servers
.com
.org
.au
.co
Top-level domain servers
cisco.com
The DNS client receives a reply (the IP address for the hostname).
domain servers
DNS Client
The browser receives the IP address and goes on with its service
The browser extra
URL and pass to
names with addresses.

Domain Name System (DNS)



DNS: services

- Hostname to IP address translation
- Host aliasing
  - A complicated hostname (the canonic… …nay have a mnemonic (the alias hostname)
  - For example, a hostname relay1.west-coast.enterprise.co… …may have 2 two aliases such as enterprise.com and www.enterprise.com.



- Mail server aliasing
  - A simple email (you@hotmail.com) may actually have a more complicated hostname (relay1.west-coast.hotmail.com).
- Load distribution
  - Busy servers have load distribution
  - replicated Web servers: many IP addresses correspond to one name
  - content distribution companies such as Akamai have used DNS in more sophisticated ways

DNS: structure

- Q: Why not centralize DNS?
  - Single point of failure
  - Traffic volume – Internet is VERY LARGE
  - Distant centralized database – is it the same being in US and in Malaysia? Oh the delays!
  - Maintenance – Updates, search algorithms

A: doesn't scale!

- Comcast DNS servers alone: 600B DNS queries/day
- Akamai DNS servers alone: 2.2T DNS queries/day

Thinking about the DNS

humongous distributed database:

- ~ billion records, each simple

handles many *trillions* of queries/day:

- *many* more reads than writes
- *performance matters:* almost every Internet transaction interacts with DNS - msecs count!

organizationally, physically decentralized:

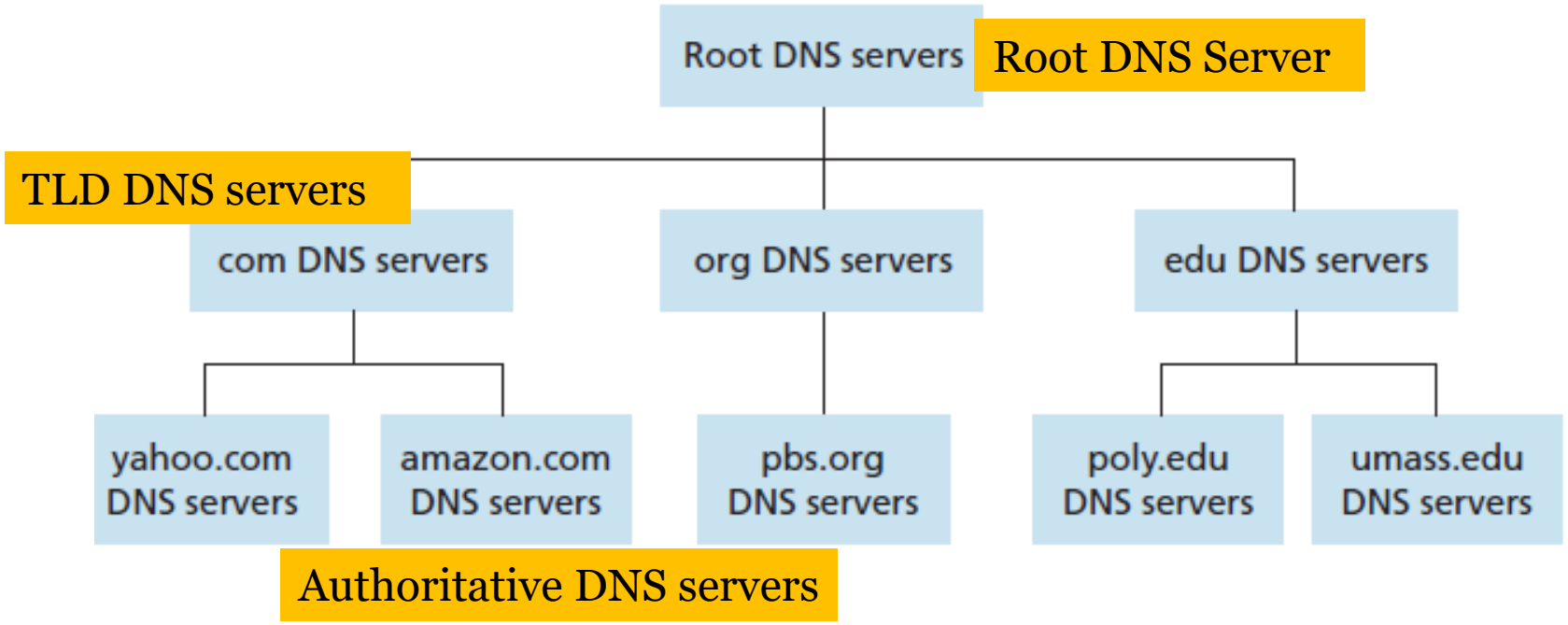
- millions of different organizations responsible for their records



"bulletproof": reliability, security

DNS: a distributed, hierarchical database

- For better performance, the DNS servers are organized in a hierarchical fashion and distributed around the world
- There are three classes of DNS servers
  - Root DNS servers
    - In the Internet there are 13 root DNS servers (labelled A through M) ➔ each "server" is actually a network of replicated servers
  - Top-level domain (TLD) DNS servers
    - Responsible for top-level domains such as com, org, net, edu, and gov, and all of the country top-level domains such as uk, fr, ca, and jp.
  - Authoritative DNS servers
    - An organization DNS server – web server, mail server
    - Example: Cisco ➔ www.cisco.com, mail.cisco.com


Root DNS servers
Root DNS Server
TLD DNS servers
com DNS servers
org DNS servers
edu DNS servers
yahoo.com DNS servers
amazon.com DNS servers
pbs.org DNS servers
poly.edu DNS servers
umass.edu DNS servers
Authoritative DNS servers

DNS: a distributed, hierarchical database

- Client wants IP for www.amazon.com; 1st approx:
  - client queries root server to find com DNS server
  - client queries .com DNS server to get amazon.com DNS server
  - client queries amazon.com DNS server to get IP address for www.amazon.com

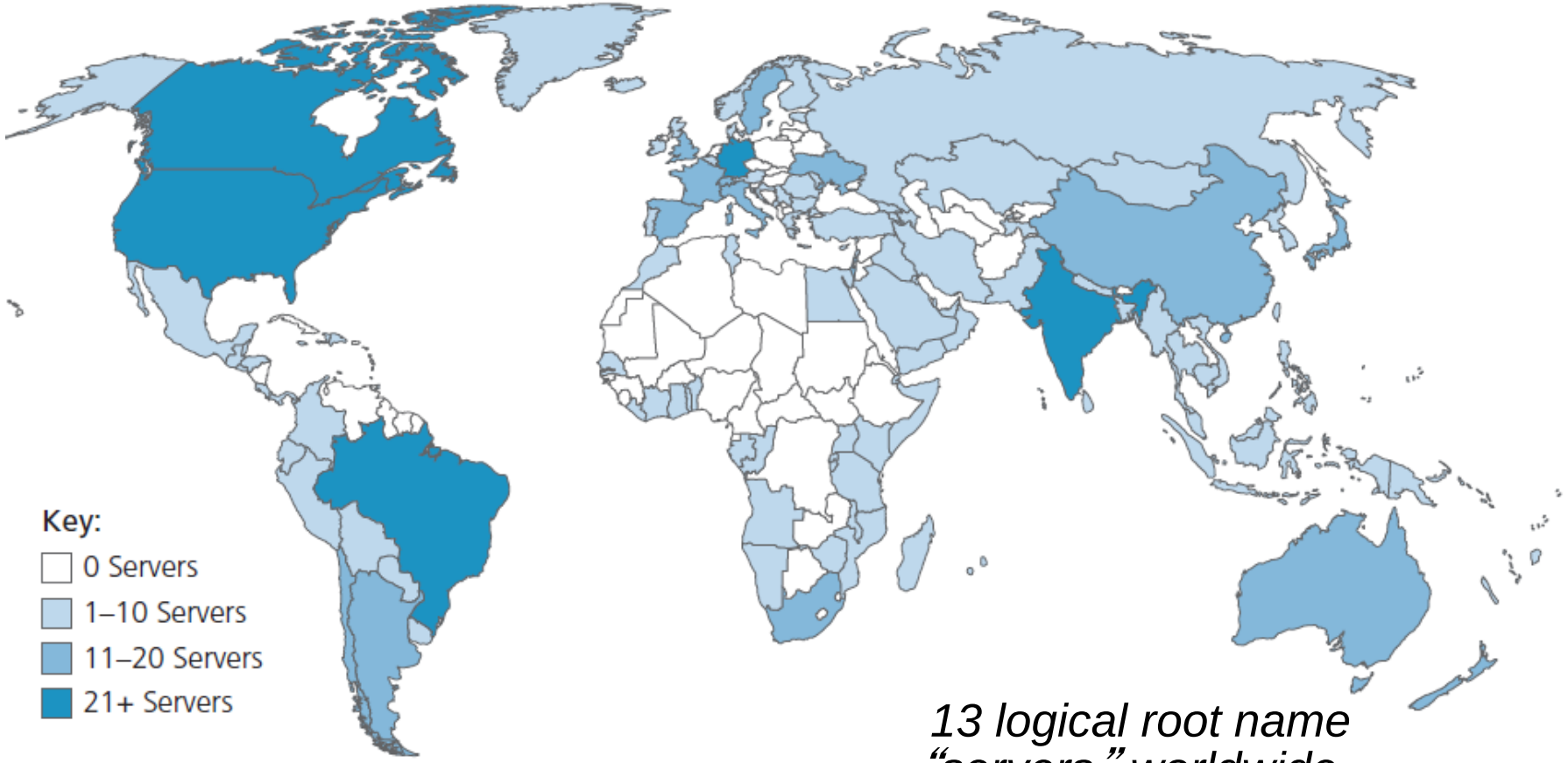
DNS: root name servers

- official, contact-of-last-resort by name servers that can not resolve name
- root name server:
  - contacts authoritative name server if name mapping not known
  - gets mapping
  - returns mapping to local name server

- *incredibly important* Internet function
  - Internet couldn't function without it!
  - DNSSEC – provides security (authentication, message integrity)
- ICANN (Internet Corporation for Assigned Names and Numbers) manages root DNS domain

DNS root servers in 2020

There are more than 1000 root servers instances scattered all over the world



13 logical root name "servers" worldwide

- *each "server" replicated many times*

TLD and authoritative servers

top-level domain (TLD) servers:

- responsible for com, org, net, edu, aero, jobs, museums, and all top-level country domains, e.g.: uk, fr, ca, jp
- Network Solutions : authoritative registry for .com , .net TLD
- Educause : for .edu TLD

authoritative DNS servers:

- organization's own DNS server(s), providing authoritative hostname to IP mappings for organization's named hosts
- can be maintained by organization or service provider

Local DNS name server

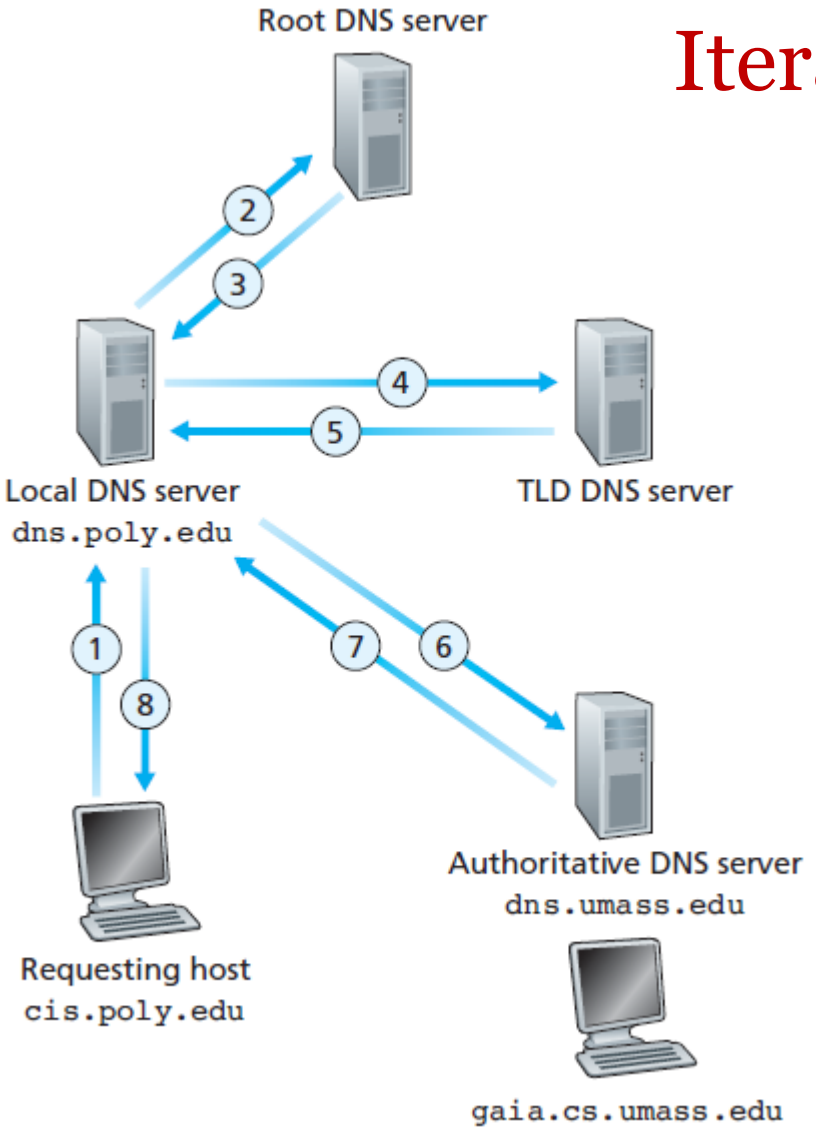
- does not strictly belong to hierarchy
- each ISP (residential ISP, company, university) has one
  - also called "default name server"
- when host makes DNS query, query is sent to its local DNS server
  - Local DNS server returns reply, answering:
    - From its local cache of recent name-to-address translation pairs (but may be out of date!)
    - acts as proxy, forwards query into hierarchy

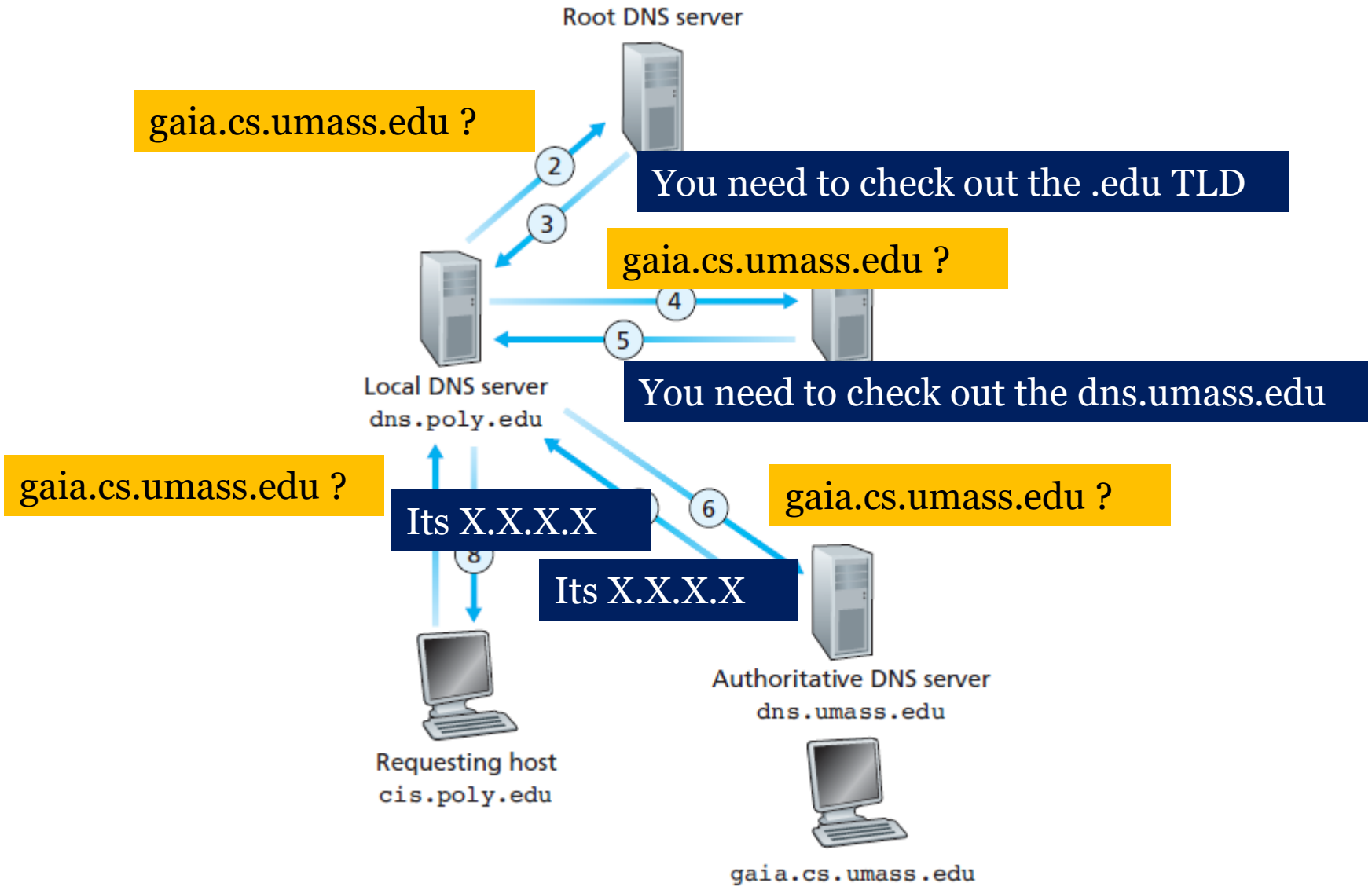
- each ISP has local DNS name server; to find yours:
  - MacOS: `% scutil --dns`
  - Windows: `>ipconfig /all`

DNS name resolution example

- DNS name resolution can be
  - Iterative :
    - the contacted server guides the clients to the next path to take
    - "I don't know this name, but ask this server"
  - Recursive:
    - the contacted name server resolves the name on the clients behalf
    - "the IP address is x.x.x.x"

Iterative queries in DNS

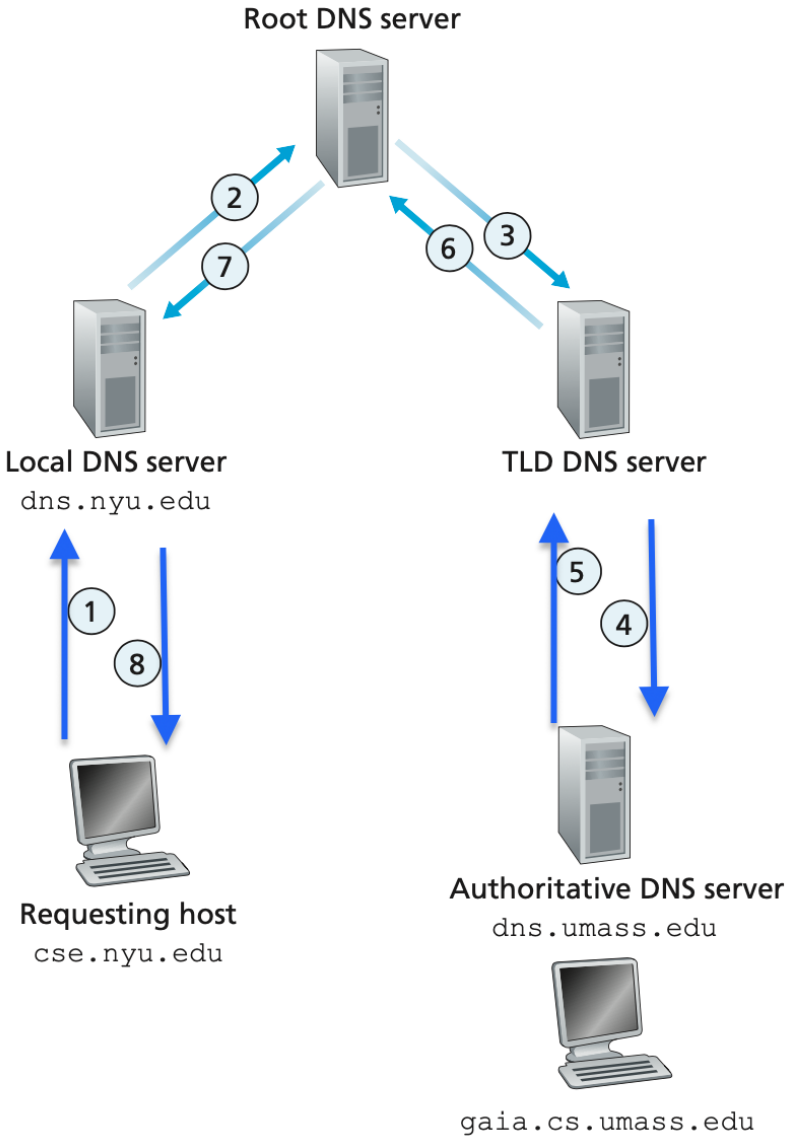



Root DNS server
gaia.cs.umass.edu ?
2
You need to check out the .edu TLD
3
gaia.cs.umass.edu ?
4
5
Local DNS server
dns.poly.edu
You need to check out the dns.umass.edu
gaia.cs.umass.edu ?
Its X.X.X.X
gaia.cs.umass.edu ?
6
8
Its X.X.X.X
Authoritative DNS server
dns.umass.edu
Requesting host
cis.poly.edu
gaia.cs.umass.edu

Recursive queries in DNS

Recursive query:

- puts burden of name resolution on contacted name server
- heavy load at upper levels of hierarchy?



DNS: caching

- DNS caching is used
  - to improve performance (↓delay)
  - to reduce the number of DNS messages going around the Internet (save bandwidth)
- once (any) name server learns mapping, it *caches* mapping in local memory
  - cache entries timeout (disappear) after some time (TTL)
  - TLD servers typically cached in local name servers
    - thus root name servers not often visited

DNS: caching

- cached entries may be *out-of-date* (best effort name-to-address translation!)
  - if name host changes IP address, may not be known Internet-wide until all TTLs expire (e.g. keep for 2 days)
- update/notify mechanisms proposed IETF standard
  - RFC 2136

DNS: record

- The DNS server stores different types of resource records (RR) used to resolve names. These records contain the name, address, and type of record.
- Some of these record types are: RR Format: (Name, Value, Type, TTL)
  - Type=A - an end device address ➔ Value is the IP address for the hostname.
  - Type=NS - an authoritative name server ➔ Value is the hostname of an authoritative DNS server that knows how to get the IP addresses for hostname (e.g. foo.com)
  - Type= CNAME ➔ Value is a canonical hostname (or Fully Qualified Domain Name) for the alias hostname Name.

    www.ibm.com is really servereast.backup2.ibm.com
  - Type= MX ➔ Value is the canonical name of a mail server that has an alias hostname Name.

DNS protocol, messages

- There are *query* and *reply* messages ➔ both with same *message form*

❖ identification: 16 bit number for query, reply to query uses same number

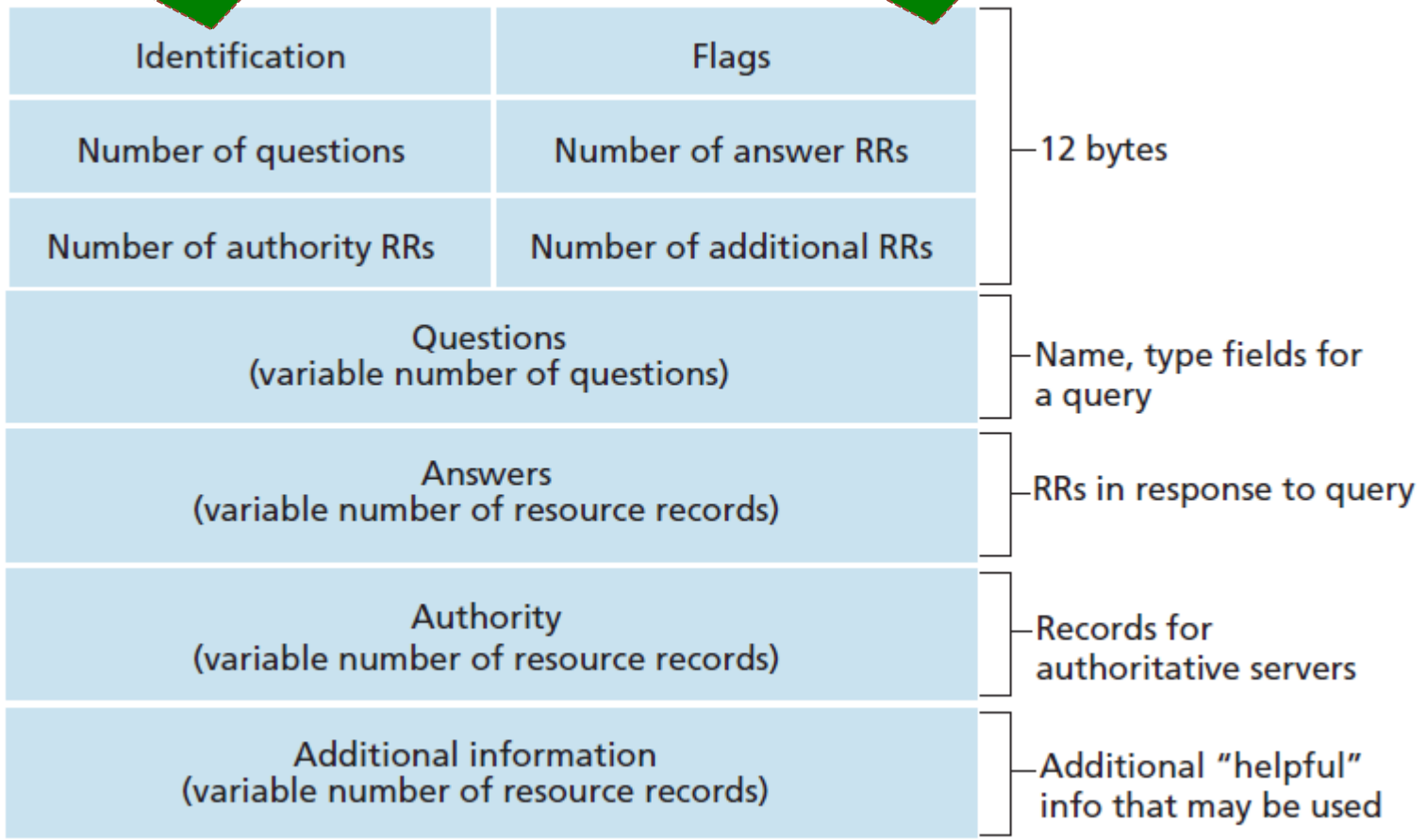
❖ flags:

- query or reply
- recursion desired
- recursion available
- reply is authoritative

DNS: messages

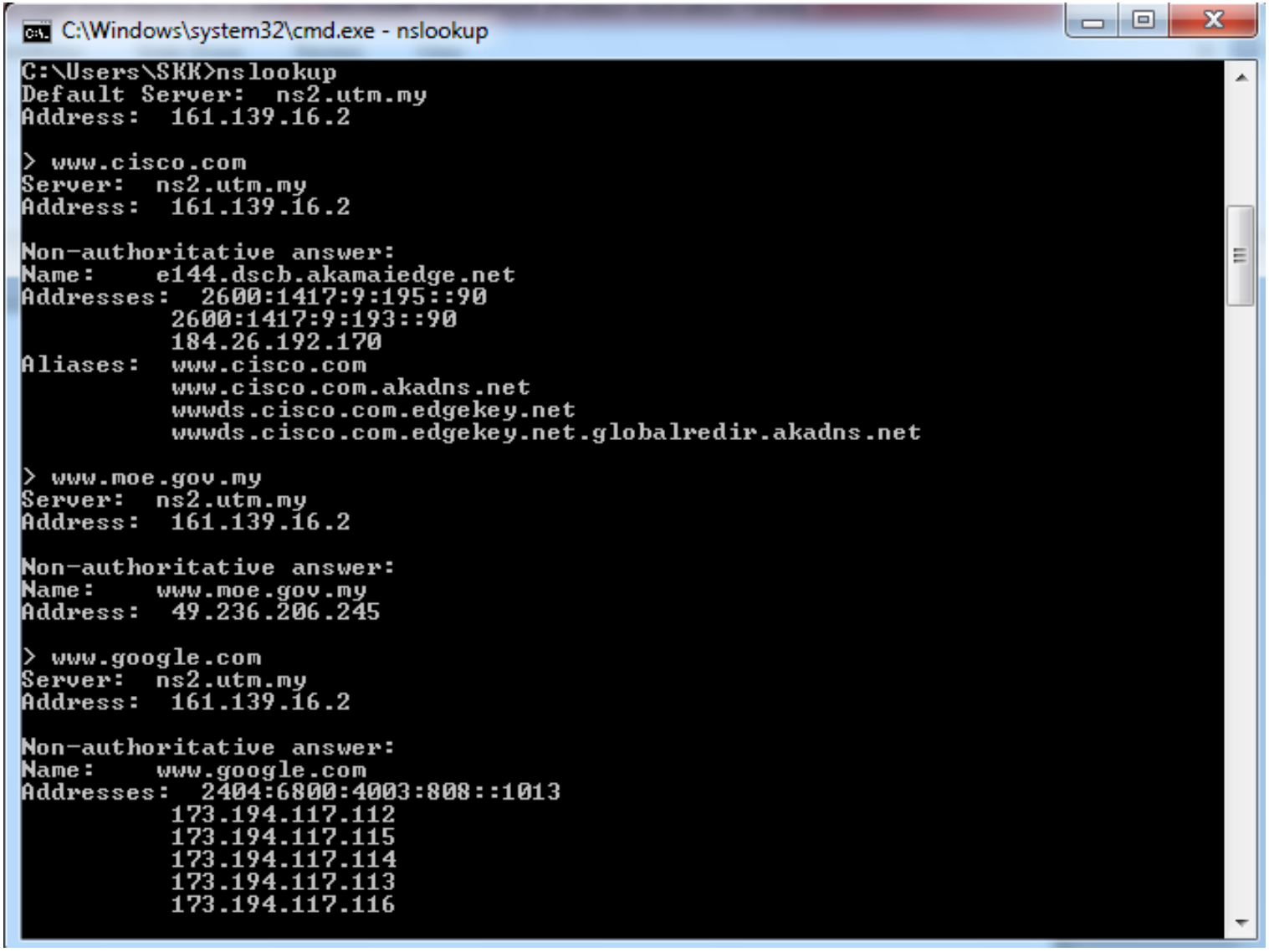
A 16-bit number that identifies the query.

Indicators: query/reply etc.



RR – Resource Records

Class activity: Using nslookup



Getting your info into the DNS

example: new startup “Network Utopia”

- register name networkuptopia.com at *DNS registrar* (e.g., Network Solutions)
  - provide names, IP addresses of authoritative name server (primary and secondary)
  - registrar inserts NS, A RRs into .com TLD server:

    `(networkutopia.com, dns1.networkutopia.com, NS)`

    `(dns1.networkutopia.com, 212.212.212.1, A)`
- create authoritative server locally with IP address `212.212.212.1`
  - type A record for www.networkuptopia.com
  - type MX record for networkutopia.com

DNS security

DDoS attacks

- **bombard root servers with traffic**
  - not successful to date
  - traffic filtering
  - local DNS servers cache IPs of TLD servers, allowing root server bypass
- **bombard TLD servers**
  - potentially more dangerous

Spoofing attacks

- **intercept DNS queries, returning bogus replies**
  - DNS cache poisoning
    - Send bogus replies to DNS server, which caches
  - RFC 4033: DNSSEC authentication services

exploit DNS for DDoS

- send queries with spoofed source address: target IP
- requires amplification

Chapter 2: Outline

- Principles of network applications
  - app architectures
  - app requirements
- Web and HTTP
- Electronic mail
  - SMTP, POP3, IMAP
- DNS

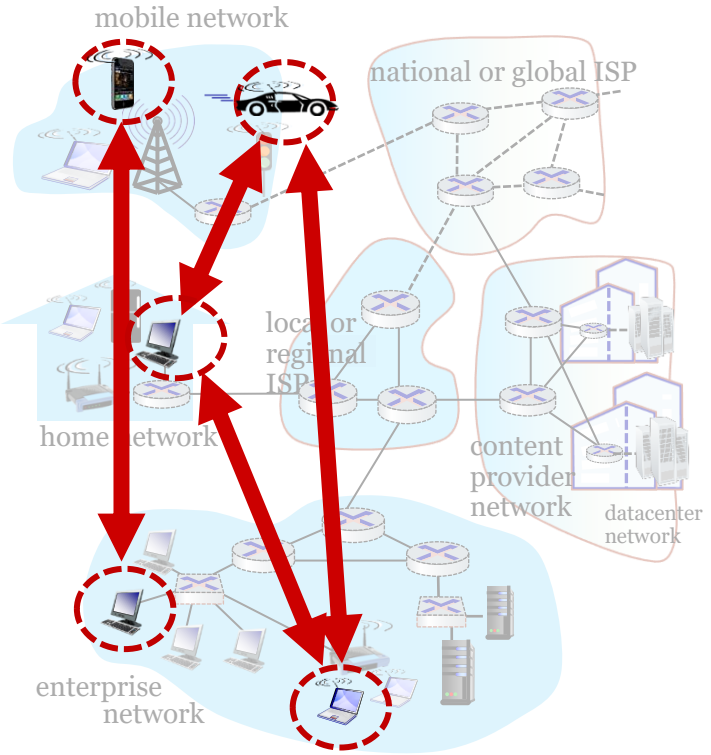
- P2P applications
- video streaming and content distribution networks
- socket programming with UDP and TCP

P2P & CDN



Peer-to-peer (P2P) architecture

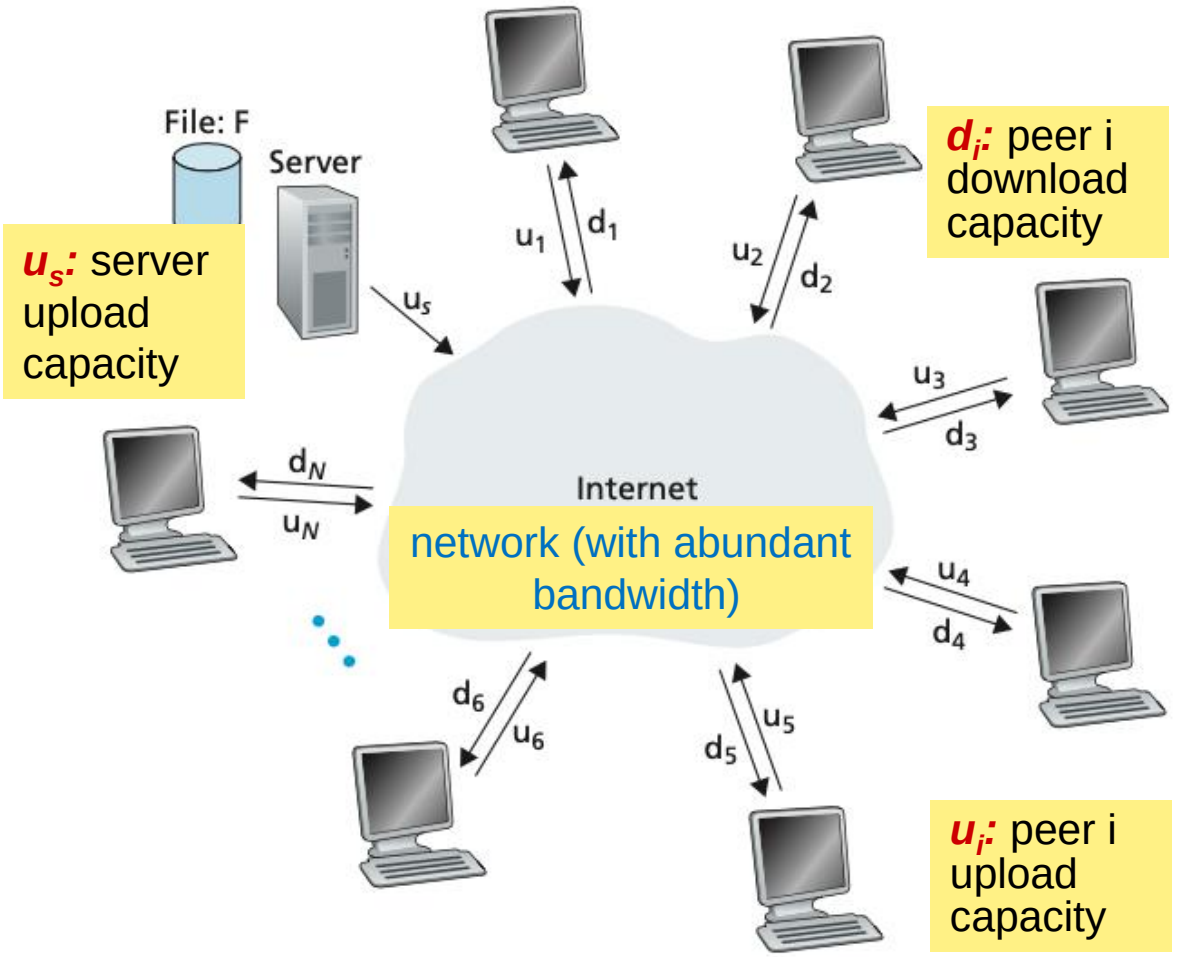
- *no* always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
  - *self scalability* – new peers bring new service capacity, and new service demands
- peers are intermittently connected and change IP addresses
- examples: P2P file sharing (BitTorrent), streaming (KanKan), VoIP (Skype)



File distribution: client-server vs P2P

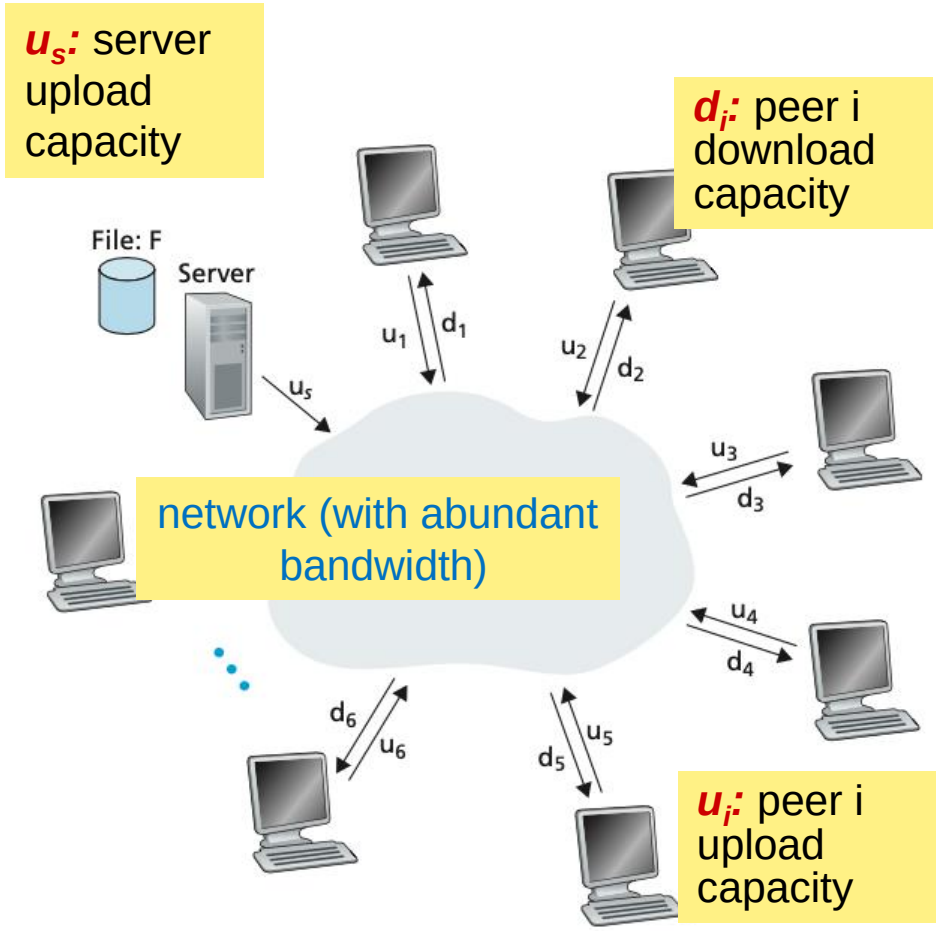
Question: how much time to distribute file (size *F*) from one server to *N* *peers*?

peer upload/download capacity is limited resource



File distribution: client-server vs P2P

- TWO assumptions are made
  1. the Internet core has abundant bandwidth, implying that all of the bottlenecks are in access networks.
  2. the server and clients are not participating in any other network applications



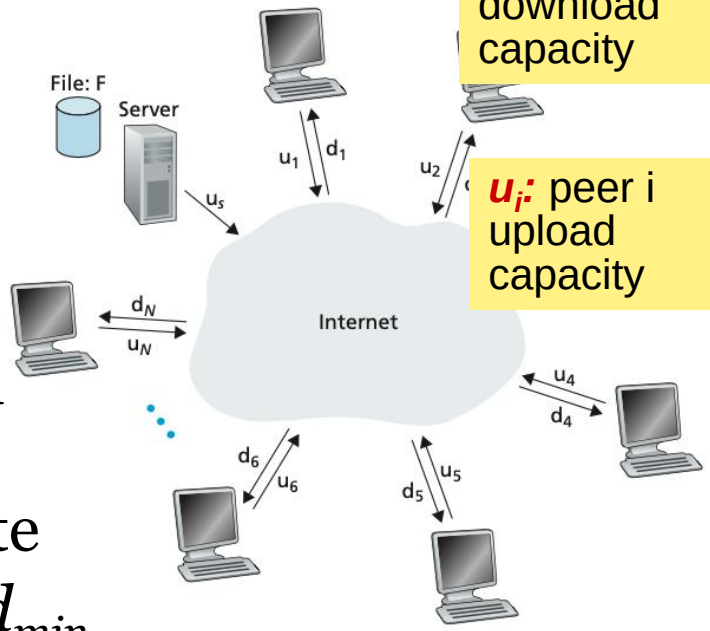
File distribution: client-server

u_s: server upload capacity

d_i: peer i download capacity

u_i: peer i upload capacity

- *server transmission:* must sequentially send (upload) N file copies:
  - time to send one copy: F/u_s
  - time to send N copies: NF/u_s
- *client:* each client must download file copy
  - d_{min} = min client download rate
  - min client download time: F/d_{min}



time to distribute F to N clients using client-server approach

$$D_{c\text{-}s} > max\{NF/u_s, F/d_{min}\}$$

increases linearly in N

$$D_{cs} \geq \max\left\{\frac{NF}{u_s}, \frac{F}{d_{\min}}\right\}.$$

$$d_{\min} = \min\{d_1, d_p, \ldots, d_N\}$$

D_{cs} = the distribution time for the client-server architecture
N = the number of peers that the server service (each needs a copy)
NF = the total transmitted bits to all peers
u_s = server's upload rate
d_{min} = the download rate of the peer with the lowest download rate
F/d_{min} = the minimum distribution time

the distribution time increases linearly with the number of peers *N*

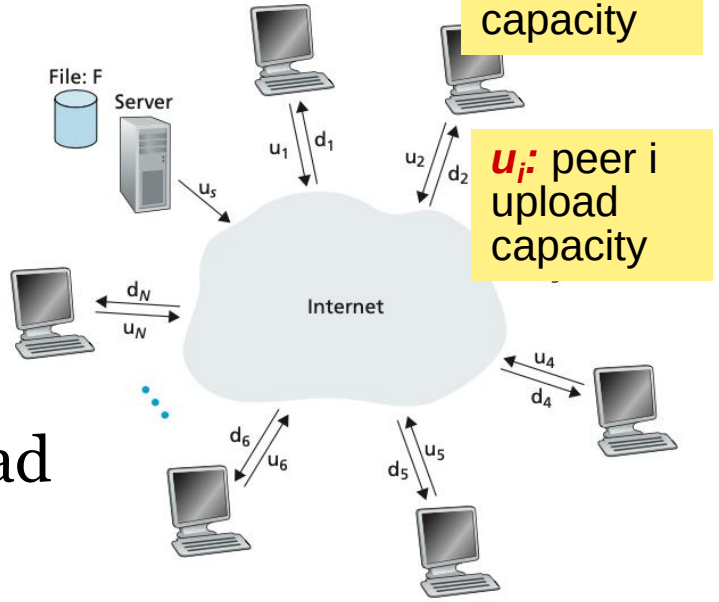
File distribution: P2P

- *server transmission:* must upload at least one copy
  - time to send one copy: F/u_s
- *client:* each client must download file copy
  - min client download time: F/d_{min}
- *clients:* as aggregate must download NF bits
  - max upload rate (limiting max download rate) is $u_s + \Sigma u_i$

u_s: server upload capacity

d_i: peer i download capacity

u_i: peer i upload capacity



time to distribute F to N clients using P2P approach

$$D_{P2P} > max\{F/u_s, F/d_{min}, NF/(u_s + \Sigma u_i)\}$$

File distribution: P2P

increases linearly in *N*

time to distribute F to N clients using P2P approach

$$D_{P2P} > max\{F/u_s, F/d_{min}, NF/(u_s + \Sigma u_i)\}$$

Increases as each peer brings service capacity

$$D_{P2P} \geq \max\left\{\frac{F}{u_s}, \frac{F}{d_{min}}, \frac{NF}{u_s + \sum_{i=1}^{N} u_i}\right\}$$

the minimum distribution time

F/u_s = the min distribution time for the client-server (at beginning)
N = the number of peers that the server service (each needs a copy)
NF = the total transmitted bits to all peers
u_s = server's upload rate
d_{min} = the download rate of the peer with the lowest download rate
F/d_{min} = the minimum distribution time

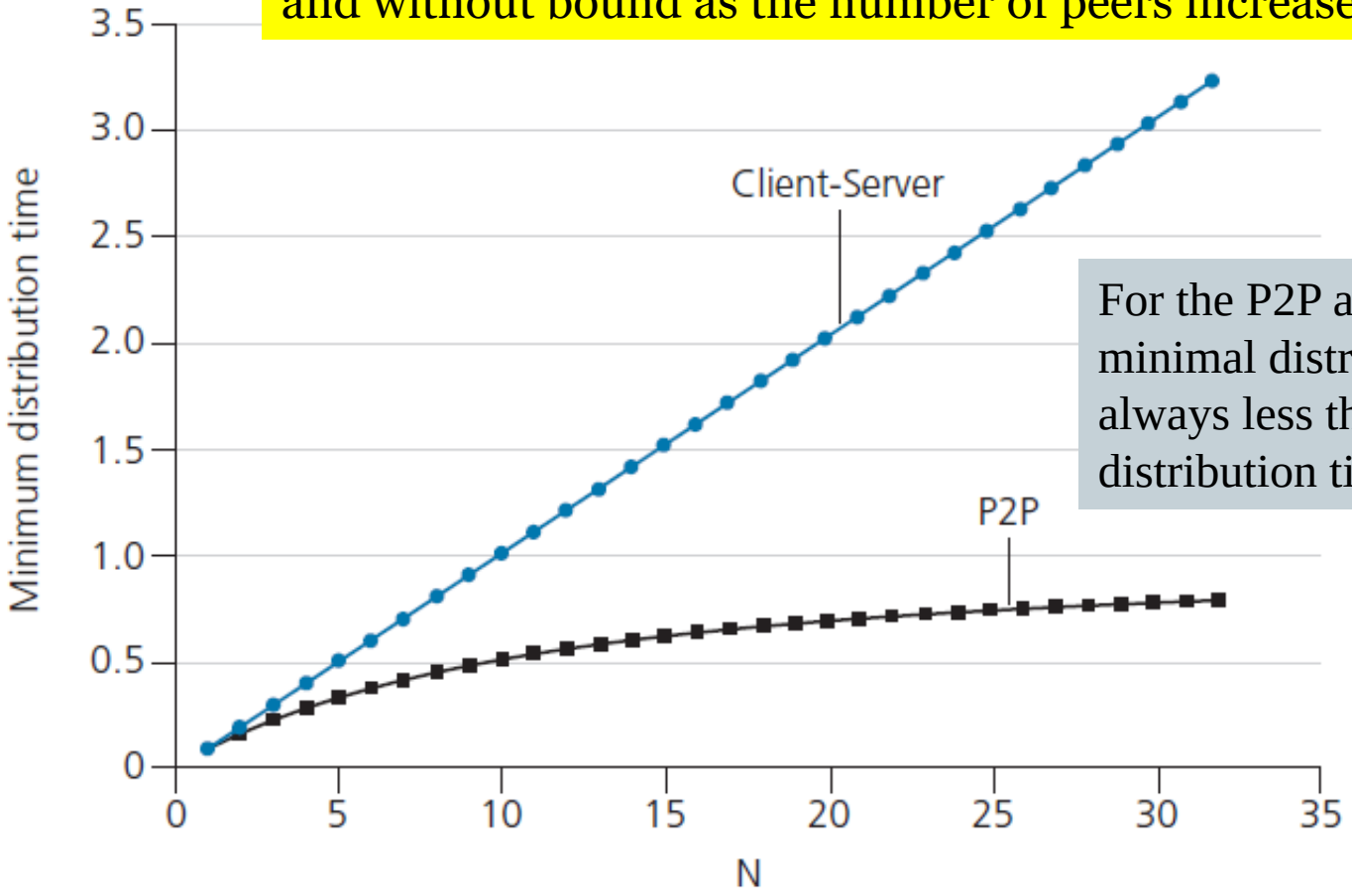
Σu_i = total upload rates of each of the individual peers

the total upload capacity of the system as a whole is equal to the upload rate of the server plus the upload rates of each of the individual peers.

$$u_{total} = u_s + u_1 + \cdots + u_N$$

Distribution time for P2P and client-server architectures

For the CS architecture: the distribution time increases linearly and without bound as the number of peers increases



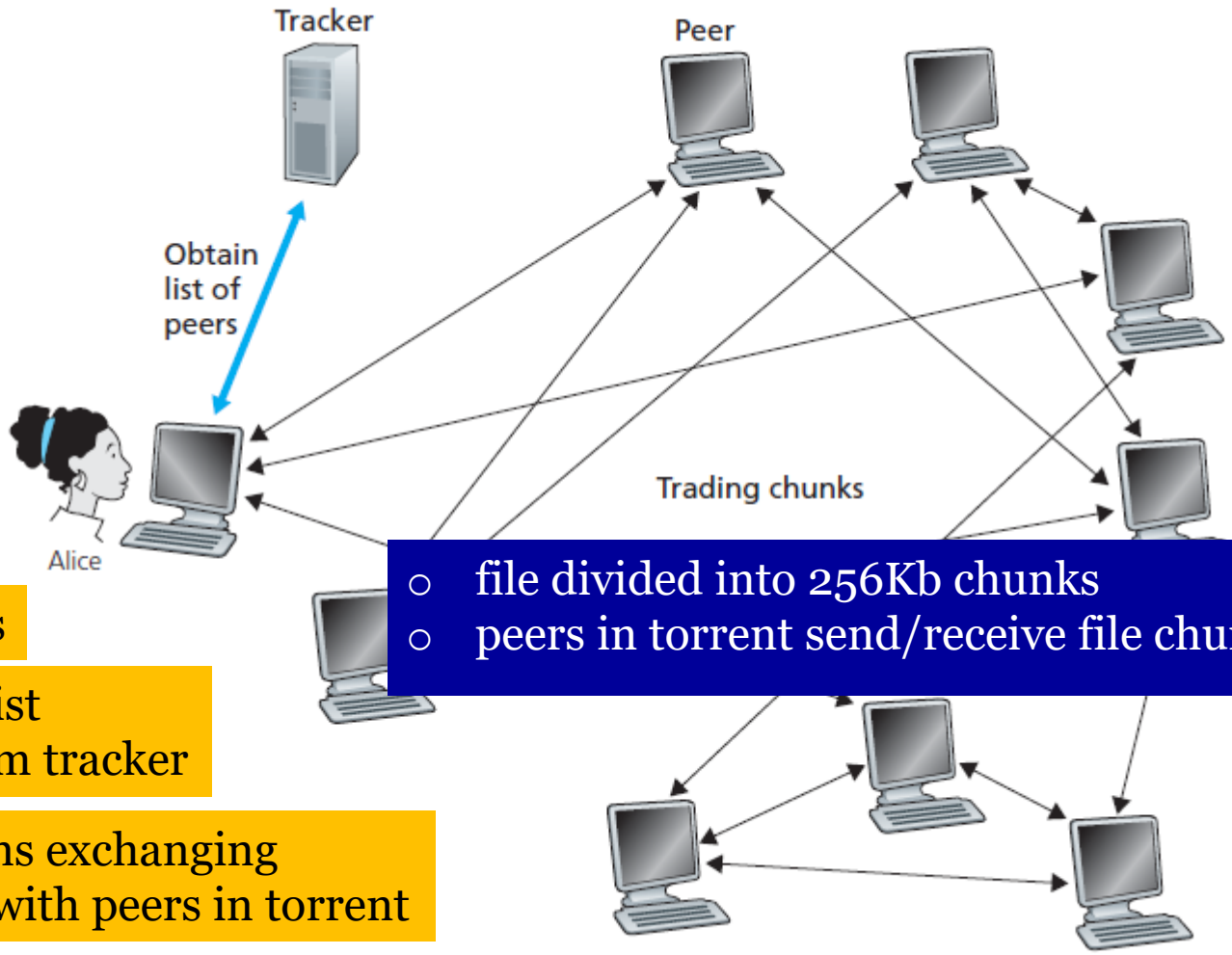
For the P2P architecture: the minimal distribution time is always less than the distribution time of the CS

client upload rate = u, F/u = 1 hour, $u_s = 10u$, $d_{min} \geq u_s$

P2P file distribution: BitTorrent

tracker: tracks peers participating in torrent

torrent: group of peers exchanging chunks of a file



- file divided into 256Kb chunks
- peers in torrent send/receive file chunks

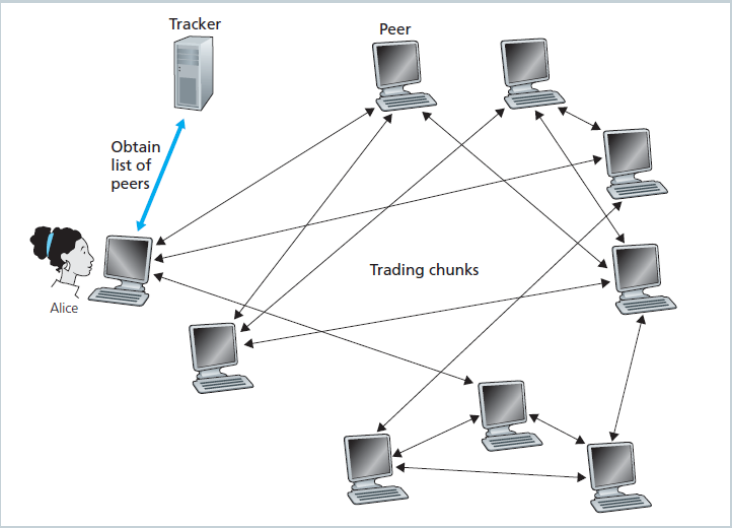
Alice arrives

→ obtains list of peers from tracker

→ and begins exchanging file chunks with peers in torrent

P2P file distribution: BitTorrent

- peer joining torrent:
  - has no chunks, but will accumulate them over time from other peers
  - registers with tracker to get list of peers, connects to subset of peers ("neighbors")



- while downloading, peer uploads chunks to other peers
- peer may change peers with whom it exchanges chunks
- *churn:* peers may come and go
- once peer has entire file, it may (selfishly) leave or (altruistically) remain in torrent

BitTorrent: requesting, sending file chunks

requesting chunks:

- at any given time, different peers have different subsets of file chunks
- periodically, Alice asks each peer for list of chunks that they have
- Alice requests missing chunks from peers, rarest first

sending chunks: tit-for-tat

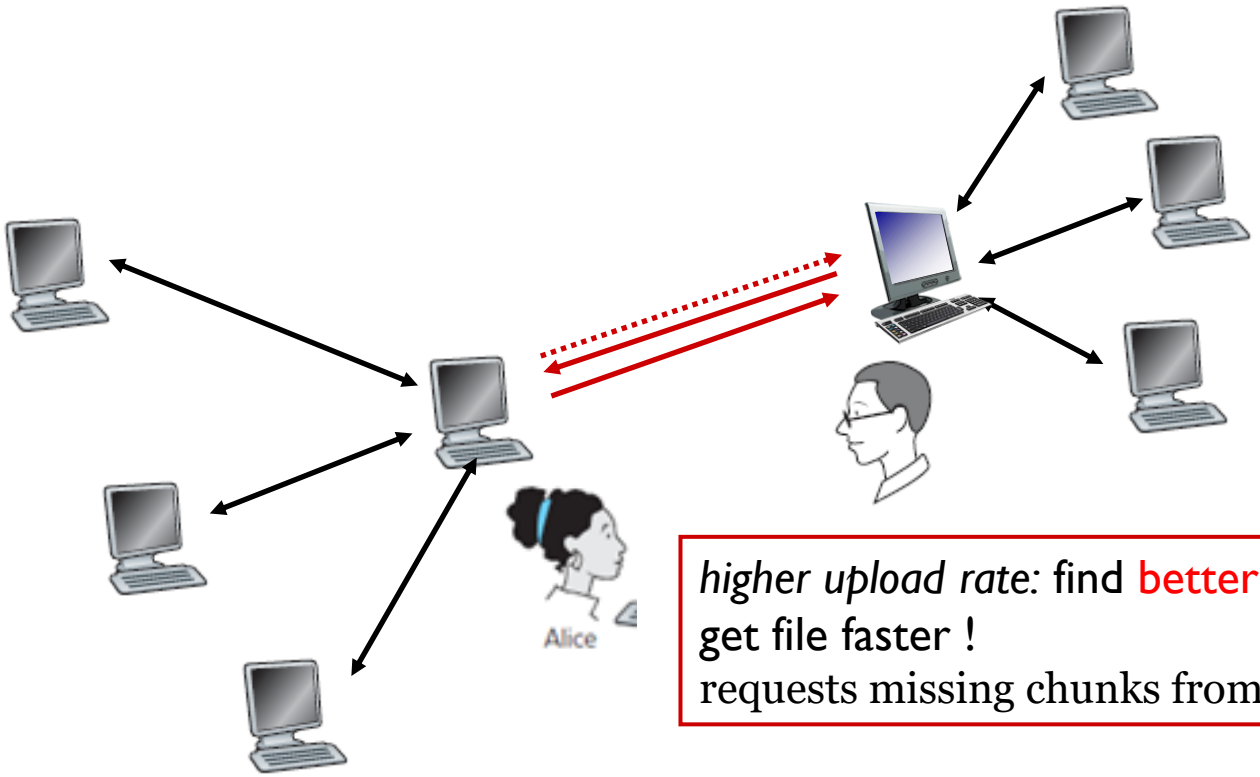
- Alice sends chunks to those four peers currently sending her chunks *at highest rate*
  - other peers are choked by Alice (do not receive chunks from her)
  - re-evaluate top 4 every10 secs
- every 30 secs: randomly select another peer, starts sending chunks
  - "optimistically unchoke" this peer
  - newly chosen peer may join top 4

BitTorrent: tit-for-tat

(1) Alice "optimistically unchokes" Bob

(2) Alice becomes one of Bob's top-four providers; Bob reciprocates

(3) Bob becomes one of Alice's top-four providers



higher upload rate: find better trading partners, get file faster !

requests missing chunks from peers: rarest first

Chapter 2: outline

- 2.1 principles of network applications
- 2.2 Web and HTTP
- 2.3 electronic mail
- SMTP, POP3, IMAP
- 2.4 DNS
- 2.5 P2P applications
- 2.6 video streaming and content distribution networks (CDNs)
- 2.7 socket programming with UDP and TCP

Video Streaming and CDNs: context

- stream video traffic: major consumer of Internet bandwidth
  - Netflix, YouTube, Amazon Prime: account for 80% of residential ISP traffic (2020)
  - Netflix, YouTube: 37%, 16% of downstream residential ISP traffic
  - ~1B YouTube users, ~75M Netflix users



- *challenge:* scale - how to reach ~1B users?



- *challenge:* heterogeneity
  - different users have different capabilities (e.g., wired versus mobile; bandwidth rich versus bandwidth poor)





- *solution:* distributed, application-level infrastructure



Multimedia: video

- video: sequence of images displayed at constant rate
  - e.g., 24 images/sec
- digital image: array of pixels
  - each pixel represented by bits
- coding: use redundancy *within* and *between* images to decrease # bits used to encode image
  - spatial (within image)
  - temporal (from one image to next)

spatial coding example: instead of sending *N* values of same color (all purple), send only two values: color value (*purple*) *and number of repeated values* (N)



frame *i*



frame *i+1*

temporal coding example: instead of sending complete frame at i+1, send only differences from frame i

Multimedia: video

- CBR: (constant bit rate): video encoding rate fixed
- VBR: (variable bit rate): video encoding rate changes as amount of spatial, temporal coding changes
- examples:
  - MPEG 1 (CD-ROM) 1.5 Mbps
  - MPEG2 (DVD) 3-6 Mbps
  - MPEG4 (often used in Internet, < 1 Mbps)

use compression to create multiple versions of the same video, each at a different quality level.

spatial coding example: instead of sending *N* values of same color (all purple), send only two values: color value (*purple) and number of repeated values (*N)



frame *i*

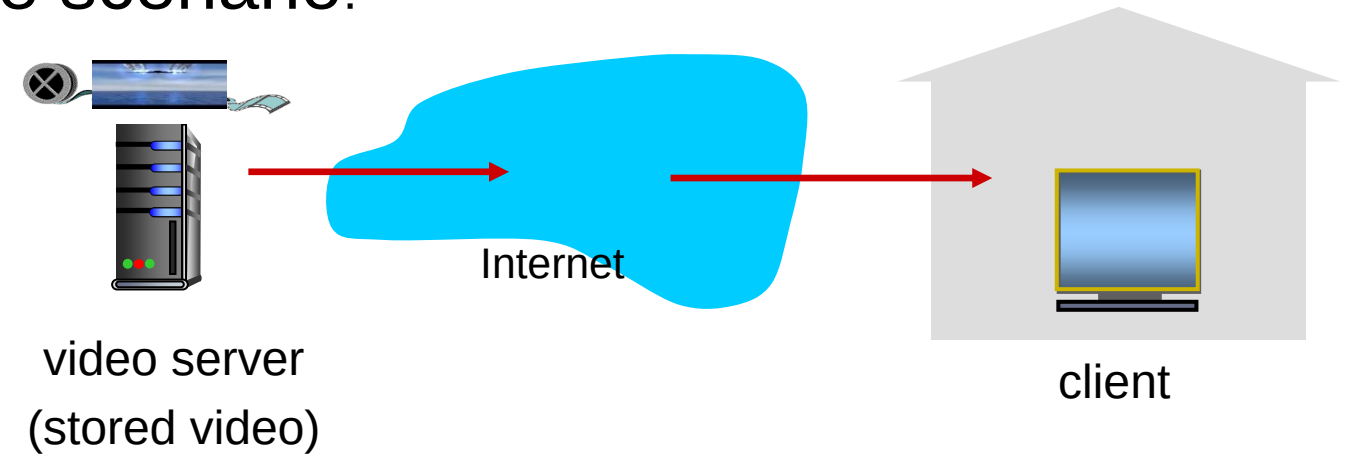


frame *i+1*

temporal coding example: instead of sending complete frame at i+1, send only differences from frame i

Streaming stored video:

simple scenario:



Have a major shortcoming: All clients receive the same encoding of the video, despite the large variations in the amount of bandwidth available to a client

➔across different clients

➔over time for the same client.

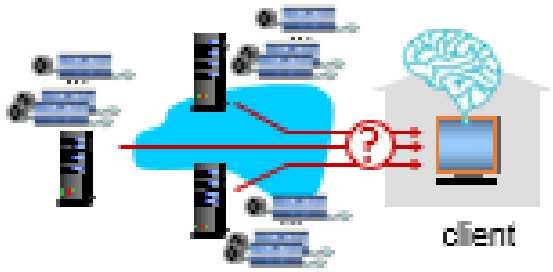
Streaming multimedia: DASH

- *DASH: D*ynamic, *A*daptive *S*treaming over *H*TTP
- *server:*
  - divides video file into multiple chunks
  - each chunk stored, encoded at different rates
  - *manifest file:* provides URLs for different chunks
- *client:*
  - periodically measures server-to-client bandwidth
  - consulting manifest, requests one chunk at a time
    - chooses maximum coding rate sustainable given current bandwidth
    - can choose different coding rates at different points in time (depending on available bandwidth at time)

Streaming multimedia: DASH

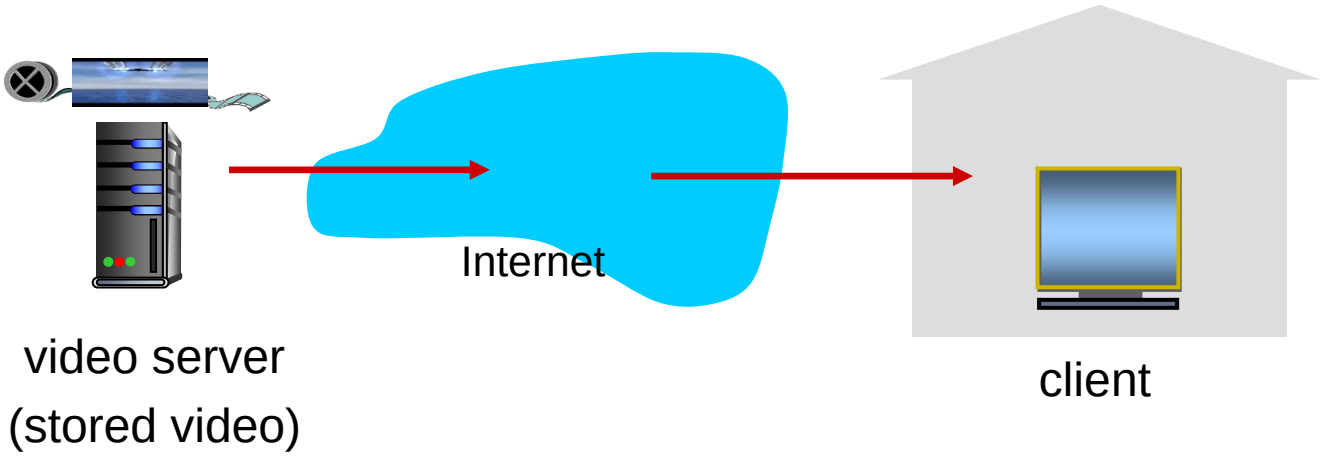
- *DASH: D*ynamic, *A*daptive *S*treaming over *H*TTP
- *"intelligence"* at client: client determines
  - *when* to request chunk (so that buffer starvation, or overflow does not occur)
  - *what encoding rate* to request (higher quality when more bandwidth available)
  - *where* to request chunk (can request from URL server that is "close" to client or has high available bandwidth)

Streaming video
= encoding + DASH + playout buffering



Streaming stored video:

simple scenario:



With DASH: Dynamic, Adaptive Streaming over HTTP

use compression to create multiple versions of the same video, each at a different quality level.

Users decide which version they want to depending on their current available bandwidth.

DASH allows the client to freely switch among different quality levels.

Content distribution networks (CDNs)

challenge: how to stream content (selected from millions of videos) to hundreds of thousands of *simultaneous* users?

- *option 1:* single, large "mega-server"
  - single point of failure
  - point of network congestion
  - long (and possibly congested) path to distant clients
  - multiple copies of video sent over outgoing link

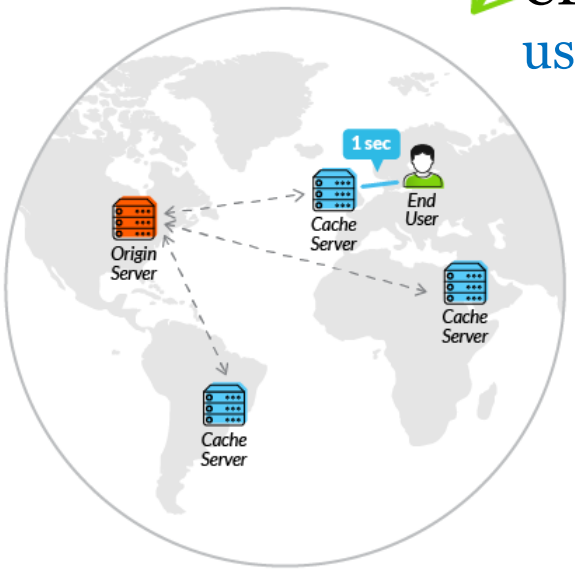
....quite simply: this solution *doesn't scale*

- *option 2:* store/serve multiple copies of videos at multiple geographically distributed sites *(CDN)*

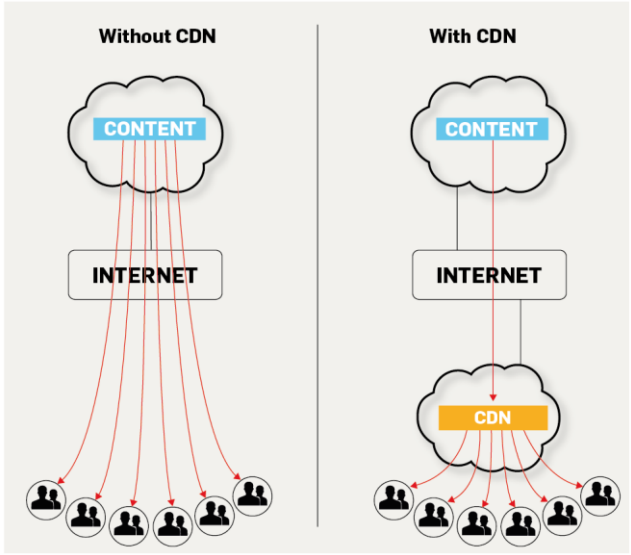
CDN



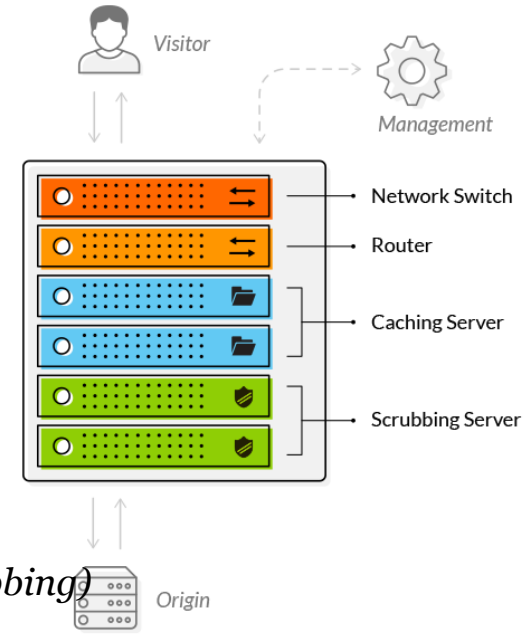
A CDN is a system of distributed servers (network) that deliver pages and other Web content to a user, based on the geographic locations of the user, the origin of the webpage and the content delivery server.



It attempts to direct each user request to a CDN location that will provide the best user experience



- A CDN's job is to enhance your regular hosting by
  - reducing bandwidth consumption
  - minimizing latency
  - providing the scalability needed to handle abnormal traffic loads.
- Requires a robust network architecture



***scrubbing server : for security (DDoS scrubbing)*

CDN



- Servers in multiple geographic locations
- May be
  - **private CDN** (owned by content provider) - Google's CDN distributes YouTube videos and other types of content
  - **a 3-rd party CDN** (pay someone else to do it) -Akamai, Limelight and Level-3 all operate third-party CDNs
- CDN placements:
  - Enter Deep
  - Bring Home
- CDN replicates content across clusters
  - Not all – just popular ones
  - Uses pull strategy
    - Clients request → cluster retrieve → cluster caches → cluster clears when cache is full

CDN placements



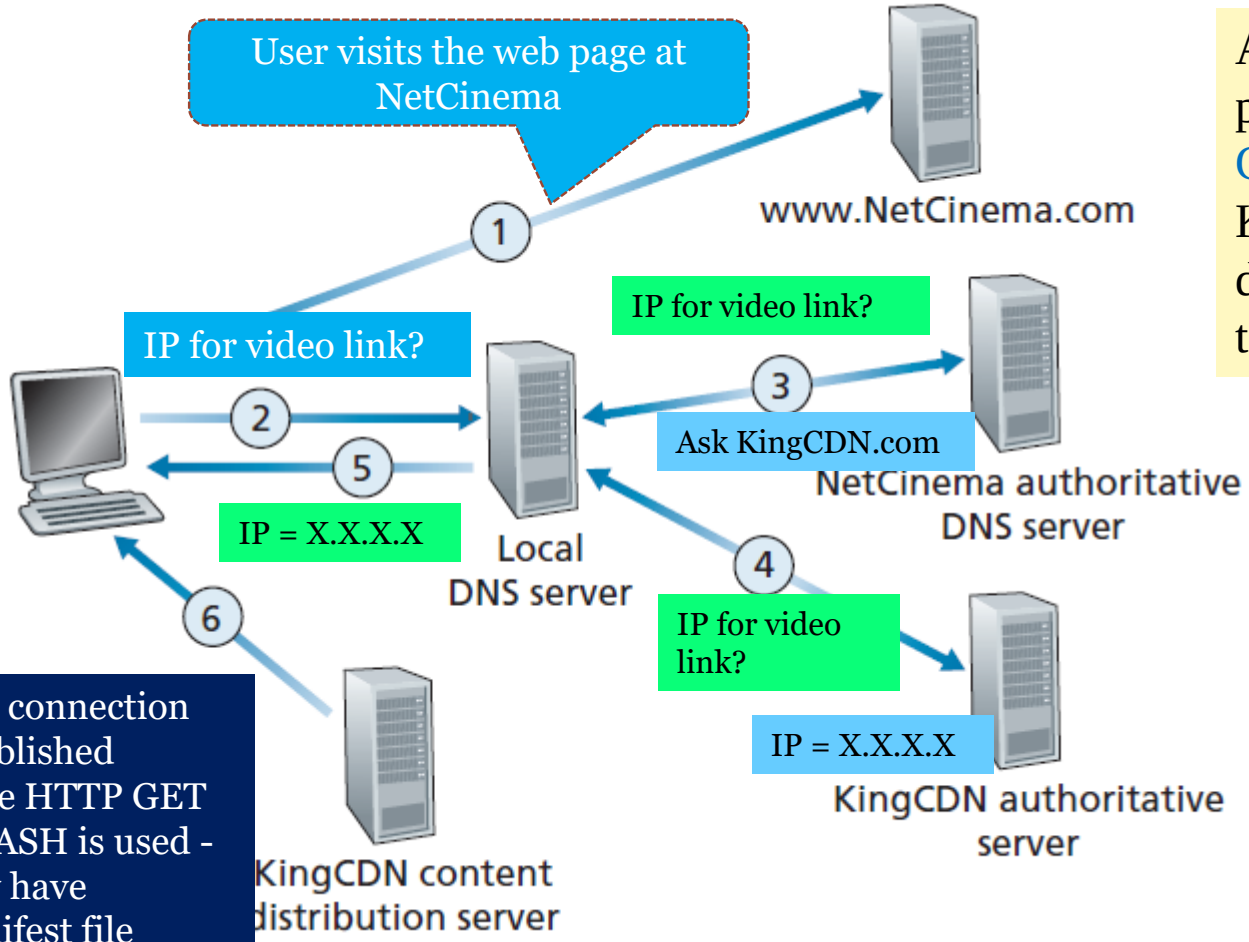
- Enter Deep
  - Goal: get close to end users (into access networks) → improve user-perceived delay and throughput
  - Deploy server clusters in access ISP all over the world
    - Akamai has about 240,000 locations in >120 countries (2015)
  - Issue: maintain and manage these clusters
- Bring Home
  - Goal: get close to ISPs→ lower maintenance and management overhead
  - Deploy server clusters in IXP (Internet Xchange Point)
    - Instead of getting inside the access ISPs, these CDNs typically place each cluster at a location that is simultaneously near the point of presence (POP) of many tier-1 ISPs.
    - Smaller number (10's) of larger clusters – connected using private high-speed network
  - Issue: higher delay, lower throughput

Homework

- CASE STUDY: GOOGLE'S NETWORK INFRASTRUCTURE
- Read this in Textbook (pg. 179)

Video link ➔ http://video.netcinema.com/6Y7B23V

Most CDNs take advantage of DNS to intercept and redirect requests



Assume NetCinema pays the 3rd party CDN company, KingCDN, to distribute its videos to its customers.

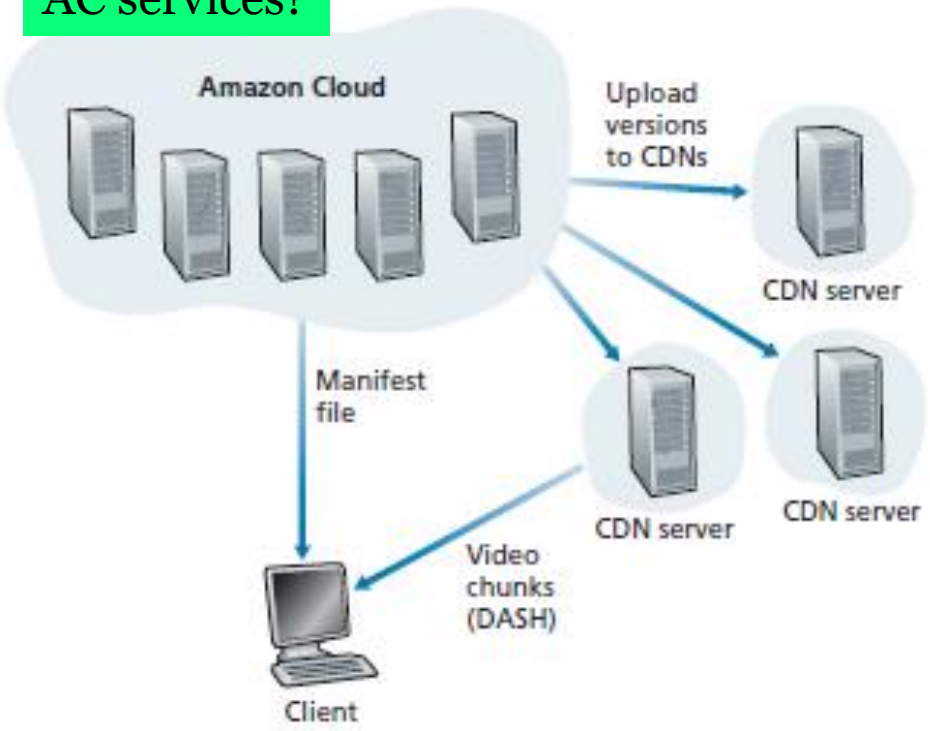
- TCP connection established
- Issue HTTP GET
- If DASH is used - May have manifest file

Which CDN servers to refer to?

- **Cluster selection strategy** is employed.
  - This is a mechanism for dynamically directing clients to a server cluster or a data center within the CDN
- Approaches:
  - Geographically closest
    - Easiest, can cater many users (not all)
    - Tend to use the same CDN servers (do not consider variation in delay and available bandwidth)
  - Real-time measurements
    - Determine best cluster based on *current traffic conditions*
    - How: send probes periodically to measure
    - Issue: Many LDNS servers are configured to not respond to probes

Netflix video streaming platform

AC services?



Netflix video distribution has two major components:

1. the Amazon cloud and
2. its own private CDN infrastructure.

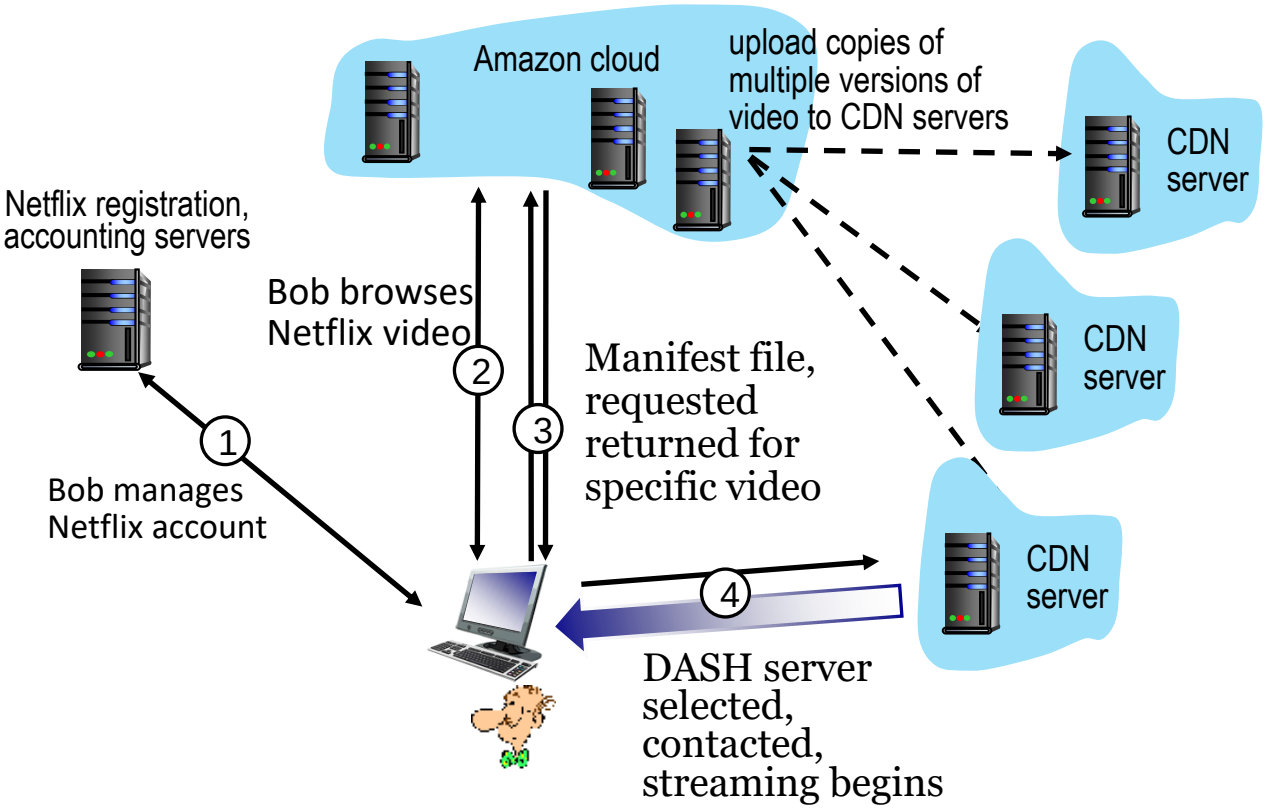
Netflix employs many of the techniques, including

- video distribution using a CDN (actually multiple CDNs)
- adaptive streaming over HTTP

Netflix: Amazon Cloud services

- User side: User accounts, registration, payment, movie catalogue, search, recommendation
- Content ingestion
  - Netflix get studio master versions of movies → uploads and processes them to hosts in the Amazon cloud
- Content processing
  - create many different formats for each movie, suitable for a diverse array of client video players running on different end devices
  - A different version is created for each of these formats and at multiple bit rates, allowing for adaptive streaming over HTTP using DASH.
- Uploading versions to its CDN
  - the hosts in the Amazon cloud upload these versions to its CDN servers

Case study: Netflix



Chapter 2: outline

2.1 principles of network applications

2.2 Web and HTTP

2.3 electronic mail

SMTP, POP3, IMAP

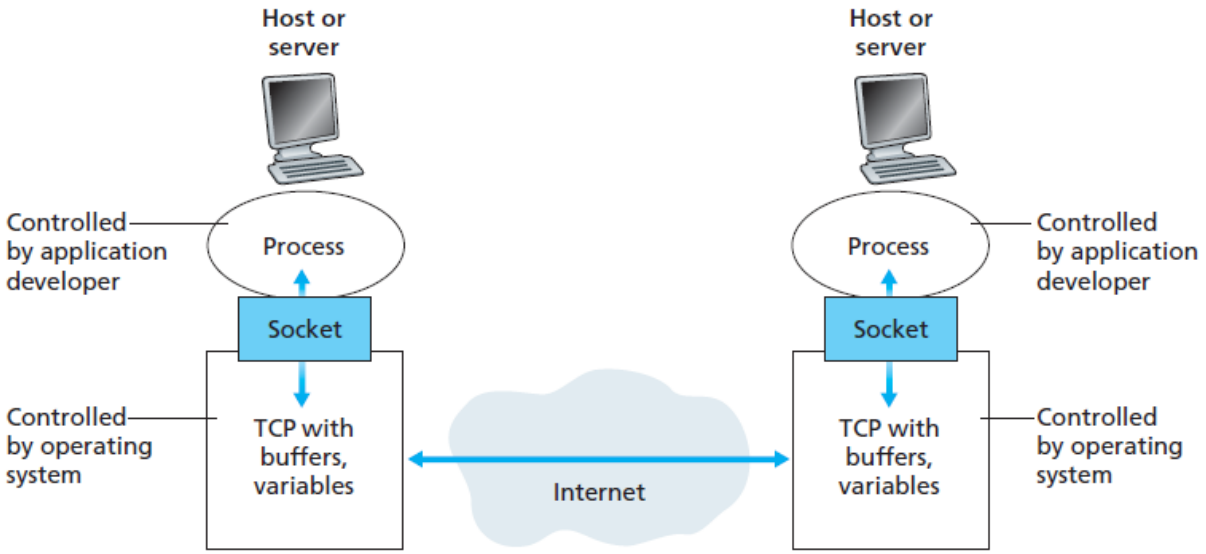
2.4 DNS

- 2.5 P2P applications
- 2.6 video streaming and content distribution networks (CDNs)
- 2.7 socket programming with UDP and TCP

Socket programming

goal: learn how to build client/server applications that communicate using sockets

socket: door between application process and end-end-transport protocol



Socket programming

Two socket types for two transport services:

- *UDP:* unreliable datagram
- *TCP:* reliable, byte stream-oriented

Application Example:

1. client reads a line of characters (data) from its keyboard and sends data to server
2. server receives the data and converts characters to uppercase
3. server sends modified data to client
4. client receives modified data and displays line on its screen

Socket programming *with UDP*



UDP: no "connection" between client & server

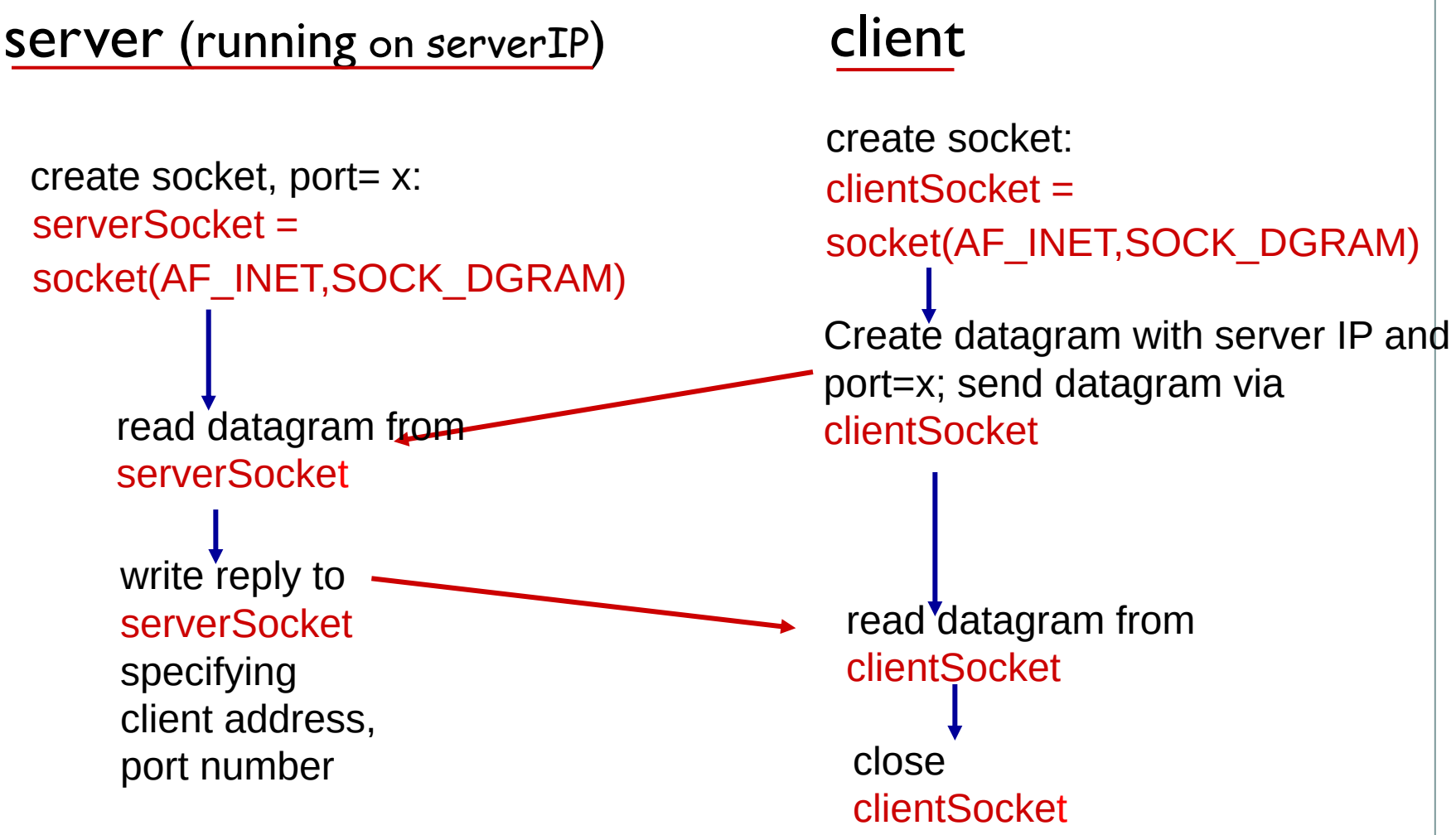
- no handshaking before sending data
- sender explicitly attaches IP destination address and port # to each packet
- receiver extracts sender IP address and port# from received packet

UDP: transmitted data may be lost or received out-of-order

Application viewpoint:

- UDP provides *unreliable* transfer of groups of bytes ("datagrams") between client and server

Client/server socket interaction: UDP



Example app: UDP client

Python UDPClient

include Python's socket library

```
from socket import *
serverName = 'hostname'
serverPort = 12000
clientSocket = socket(AF_INET,
                      SOCK_DGRAM)
message = raw_input('Input lowercase sentence:')
clientSocket.sendto(message.encode(),
                    (serverName, serverPort))
modifiedMessage, serverAddress =
                    clientSocket.recvfrom(2048)
print modifiedMessage.decode()
clientSocket.close()
```

- IP address (128.138.32.126) or the
- hostname (abc.uniA.edu) of the server

create UDP socket for server

- AF = Address family
- AF_INET = IPv4

get user keyboard input

Attach server name, port to message; send into socket

read reply characters from socket into string

print out received string and close socket

Example app: UDP server

Python UDPServer

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))
print ("The server is ready to receive")
while True:
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(),
                        clientAddress)
```

create UDP socket

bind socket to local port number 12000

loop forever

Read from UDP socket into message, getting client's address (client IP and port)

send upper case string back to this client

Socket programming *with TCP*

client must contact server

- server process must first be running
- server must have created socket (door) that welcomes client's contact

client contacts server by:

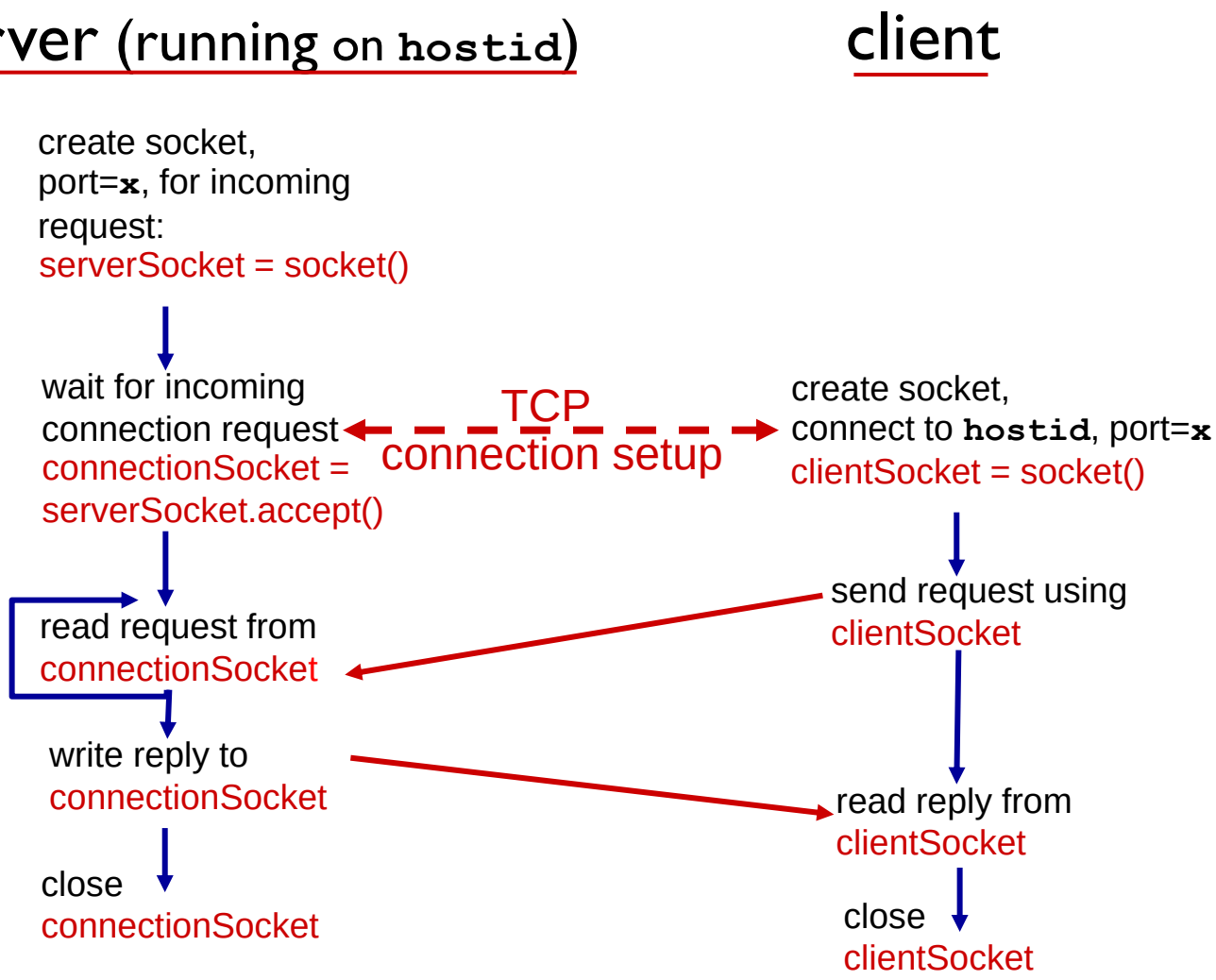
- Creating TCP socket, specifying IP address, port number of server process
- *when client creates socket:* client TCP establishes connection to server TCP

- when contacted by client, *server TCP creates new socket* for server process to communicate with that particular client
  - allows server to talk with multiple clients
  - source port numbers used to distinguish clients (more in Chap 3)

application viewpoint:

TCP provides reliable, in-order byte-stream transfer ("pipe") between client and server

Client/server socket interaction: TCP



Example app: TCP client

Python TCPClient

```
from socket import *
serverName = 'servername'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName,serverPort))
sentence = raw_input('Input lowercase sentence:')
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print ('From Server:', modifiedSentence.decode())
clientSocket.close()
```

Create a client TCP socket

create TCP socket for server, remote port 12000

sends the sentence through the client's socket and into the TCP connection. No need to attach server name, port

Example app: TCP server

Python TCPServer

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind(('',serverPort))
serverSocket.listen(1)
print 'The server is ready to receive'
while True:
    connectionSocket, addr = serverSocket.accept()

    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.
                                        encode())
    connectionSocket.close()
```

create TCP welcoming socket

server begins listening for incoming TCP requests

loop forever

server waits on accept() for incoming requests, new socket created on return

read bytes from socket (but not address as in UDP)

close connection to this client (but *not* welcoming socket)

Chapter 2: summary

- application architectures
  - client-server
  - P2P
- application service requirements:
  - reliability, bandwidth, delay
- Internet transport service model
  - connection-oriented, reliable: TCP
  - unreliable, datagrams: UDP

our study of network apps now complete!

- specific protocols:
  - HTTP
  - FTP
  - SMTP, POP, IMAP
  - DNS
  - P2P: BitTorrent
  - video streaming, CDNs
  - socket programming: TCP, UDP sockets

Chapter 2: summary

most importantly: learned about protocols!

- typical request/reply message exchange:
  - client requests info or service
  - server responds with data, status code
- message formats:
  - headers: fields giving info about data
  - data: info being communicated

important themes:

- control vs. data messages
  - in-band, out-of-band
- centralized vs. decentralized
- stateless vs. stateful
- reliable vs. unreliable message transfer
- "complexity at network edge"